



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Specifica Tecnica

Stato	In lavorazione
Versione	0.12.0
Autori	Aldo Bettega Davide Lorenzon Ana Maria Draghici Filippo Guerra Felician Mario Necsulescu Davide Testolin
Verificatori	Davide Lorenzon Filippo Guerra Felician Mario Necsulescu
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin BlueWind srl

Vers.	Data	Autore	Verificatore	Descrizione
0.13.0	2026-05-19	Aldo Bettega		Scrittura sezione MVVM
0.12.0	2026-05-16	Aldo Bettega		Rivisitazione complessiva dei diagrammi e creazione dei diagrammi di dominio, asset, device e generazione report
0.11.0	2026-05-14	Felician Mario Necsulescu	Aldo Bettega	Riviste classi di importazione ed esportazione: <u>Sezione 5.4</u> e <u>Sezione 5.5</u>
0.10.0	2026-05-12	Felician Mario Necsulescu	Aldo Bettega	Riviste classi di sessione e valutazione: <u>Sezione 5.6</u> e <u>Sezione 5.7.1</u>
0.9.0	2026-05-06	Ana Maria Draghici	Aldo Bettega	Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Import, ValutazioneRequisiti, Dashboard, Export e Frontend
0.8.0	2026-05-03	Ana Maria Draghici	Aldo Bettega	Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Device, Asset e Session
0.7.3	2026-04-22	Davide Testolin	Ana Maria Draghici	Migliorata sezione <u>Sezione 3.5</u>
0.7.2	2026-04-20	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern comportamentali e architetturali
0.7.1	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern: creazionali e strutturali <u>Sezione 4.2</u> , <u>Sezione 4.3</u>

Vers.	Data	Autore	Verificatore	Descrizione
0.7.0	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei principi di design <u>Sezione 4.1</u>
0.6.4	2026-04-19	Davide Lorenzon	Aldo Bettega	Revisione architeturale della sezione relativa alla modifica degli asset
0.6.3	2026-04-19	Davide Lorenzon	Aldo Bettega	Bozza delle classi relative alla generazione del report di conformità
0.6.2	2026-04-19	Davide Lorenzon	Aldo Bettega	Bozza delle classi relative a import ed export tramite file di dispositivo e modello
0.6.1	2026-04-17	Filippo Guerra	Aldo Bettega	Bozza della classe valutazione <u>Sezione 5.6</u>
0.6.0	2026-04-17	Filippo Guerra	Davide Lorenzon	Stesura vista_dati
0.5.0	2026-04-16	Ana Maria Draghici	Davide Lorenzon	Stesura vista_dati <u>Sezione 3.5</u>
0.4.0	2026-04-16	Aldo Bettega	Davide Lorenzon	Stesura diagramma dominio e <u>Sezione 5.2</u>
0.3.0	2026-04-15	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.4</u>
0.2.0	2026-04-10	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.1</u> e <u>Sezione 3.2</u>
0.1.2	2026-04-09	Davide Lorenzon	Filippo Guerra	Bozza iniziale della <u>Sezione 3.3</u> Architettura di deployment.
0.1.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Bozza iniziale della <u>Sezione 2</u> Tecnologie.

Vers.	Data	Autore	Verificatore	Descrizione
0.1.0	2026-04-08	Aldo Bettega	Felician Mario Necsulescu	Stesura introduzio- ne
0.0.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Creazione del docu- mento

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	2
1.4.1) Riferimenti normativi	2
1.4.2) Riferimenti informativi	2
2) Tecnologie	3
2.1) Backend	3
2.2) Frontend	5
2.3) Persistenza dei dati	5
3) Architettura di Sistema	7
3.1) Premessa: approccio di lavoro usato	7
3.2) Architettura Logica	7
3.2.1) Stile architetturale	7
3.2.2) Motivazioni della scelta architetturale	8
3.2.3) Limiti dell'architettura	8
3.2.4) Diagramma dei package	9
3.3) Architettura di Deployment	10
3.3.1) Motivazioni della scelta	10
3.3.2) Realizzazione Pratica e Topologia	10
3.4) Diagrammi di sequenza	11
3.4.1) UC05 - Importazione dispositivo	12
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato	14
3.5) Schema dati	16
3.5.1) Scelte Architettureali e Formalismo	16
3.5.2) Schema Dati Device	16
3.5.3) Schema ComplianceStandard	18
3.5.4) Gestione degli Esiti e Storicità	21
4) Design Pattern	22
4.1) Principi di Design	22
4.1.1) Dependency Injection	22
4.2) Design Pattern Creazionali	22
4.2.1) Factory	22
4.3) Design Pattern strutturali	23
4.3.1) Adapter	23
4.4) Design Pattern Comportamentali	23
4.4.1) Template Method	23
4.5) Pattern MVVM nel Frontend	24
4.5.1) Model	24

4.5.2)	ViewModel	24
4.5.3)	View	24
5)	Diagrammi delle classi	25
5.1)	Dominio	25
5.1.1)	Device	26
5.1.2)	Asset	27
5.1.3)	AssetAnagraphic	27
5.1.4)	AssetProperties	28
5.1.5)	AssetType	29
5.1.6)	AssetEvidence	29
5.1.7)	ComplianceStandard	30
5.1.8)	Requirement	31
5.1.9)	DecisionTree	32
5.1.10)	Node	32
5.1.11)	DecisionNode	33
5.1.12)	LeafNode	34
5.1.13)	StandardVerdict	34
5.1.14)	EvaluationEngine	35
5.1.15)	EvaluationState	36
5.1.16)	RequirementEvaluationResult	36
5.1.17)	AssetEvaluationResult	37
5.1.18)	DeviceEvaluationResult	38
5.1.19)	NodeDetail	39
5.1.20)	RequirementEvaluationDetail	40
5.1.21)	AssetEvaluationDetail	41
5.1.22)	DeviceEvaluationDetail	41
5.1.23)	EvaluationSession	42
5.1.24)	SessionHandler	43
5.1.25)	ExportedFile	43
5.1.26)	DeviceSummary	44
5.2)	Device	45
5.2.1)	CreateDevice	45
5.2.1.1)	FlaskWriteDeviceController	45
5.2.1.2)	CreateDeviceUseCase	46
5.2.1.3)	CreateDeviceCommand	46
5.2.1.4)	CreateDeviceService	47
5.2.1.5)	RegisterDevicePort	47
5.2.1.6)	MongoDeviceAdapter	48
5.2.2)	DeleteDevice	49
5.2.2.1)	DeleteDeviceUseCase	49
5.2.2.2)	DeleteDeviceCommand	50

5.2.2.3)	DeleteDeviceService	50
5.2.2.4)	DeleteDevicePort	51
5.2.3)	UpdateDevice	51
5.2.3.1)	UpdateDeviceUseCase	52
5.2.3.2)	UpdateDeviceCommand	52
5.2.3.3)	UpdateDeviceService	53
5.2.3.4)	SaveDevicePort	53
5.2.3.5)	FindDevicePort	54
5.2.4)	GetDeviceDetail	54
5.2.4.1)	FlaskQueryDeviceController	55
5.2.4.2)	GetDeviceDetailUseCase	55
5.2.4.3)	GetDeviceDetailService	56
5.2.4.4)	GetDeviceDetailCommand	56
5.2.4.5)	GetComplianceStandardUseCase	57
5.2.4.6)	GetComplianceStandardService	57
5.2.4.7)	GetComplianceStandardCommand	58
5.2.4.8)	FindStandardPort	58
5.2.4.9)	MongoStandardAdapter	59
5.2.5)	GetDeviceList	59
5.2.5.1)	GetDeviceListUseCase	60
5.2.5.2)	GetDeviceListService	60
5.2.5.3)	DeviceSummary	61
5.2.5.4)	FindAllDevicesPort	61
5.2.6)	GetDeviceEvaluationDetail	62
5.2.6.1)	FlaskDeviceEvaluationDetailController	62
5.2.6.2)	GetDeviceEvaluationDetailCommand	63
5.2.6.3)	GetDeviceEvaluationDetailUseCase	63
5.2.6.4)	GetDeviceEvaluationDetailService	64
5.2.6.5)	DTO	64
	5.2.6.5.1) DeviceEvaluationDTO	65
	5.2.6.5.2) AssetEvaluationSummaryDTO	65
5.3)	Asset	66
5.3.1)	CreateAsset	66
5.3.1.1)	FlaskWriteAssetController	67
5.3.1.2)	CreateAssetUseCase	67
5.3.1.3)	CreateAssetCommand	68
5.3.1.4)	CreateAssetService	69
5.3.1.5)	InMemoryEvaluationSessionCache	69
5.3.1.6)	SaveEvaluationSessionPort	70
5.3.1.7)	GetEvaluationSessionPort	70
5.3.2)	UpdateAsset	71

5.3.2.1)	UpdateAssetUseCase	72
5.3.2.2)	UpdateAssetCommand	72
5.3.2.3)	UpdateAssetService	73
5.3.3)	DeleteAsset	74
5.3.3.1)	DeleteAssetCommand	75
5.3.3.2)	DeleteAssetUseCase	75
5.3.3.3)	DeleteAssetService	76
5.3.4)	GetAssetAnagraphic	77
5.3.4.1)	FlaskQueryAssetController	78
5.3.4.2)	GetAssetAnagraphicCommand	78
5.3.4.3)	GetAssetAnagraphicUseCase	79
5.3.4.4)	GetAssetAnagraphicService	79
5.3.5)	GetAssetEvaluationDetail	80
5.3.5.1)	FlaskAssetEvaluationDetailController	80
5.3.5.2)	GetAssetEvaluationDetailCommand	81
5.3.5.3)	GetAssetEvaluationDetailUseCase	81
5.3.5.4)	GetAssetEvaluationDetailService	82
5.3.5.5)	DTO	82
5.3.5.5.1)	AssetEvaluationDTO	83
5.3.5.5.2)	RequirementEvaluationSummaryDTO	83
5.3.6)	GetRequirement	84
5.3.6.1)	FlaskRequirementEvaluationDetailController	84
5.3.6.2)	GetRequirementEvaluationDetailUseCase	85
5.3.6.3)	GetRequirementEvaluationDetailCommand	86
5.3.6.4)	GetRequirementEvaluationDetailService	86
5.3.6.5)	DTO	87
5.3.6.5.1)	RequirementEvaluationDTO	87
5.3.6.5.2)	DependencySummaryDTO	88
5.3.6.5.3)	DecisionTreeDTO	88
5.3.6.5.4)	LeafNodeDTO	89
5.3.6.5.5)	DecisionNodeDTO	89
5.3.6.5.6)	NodeBaseDTO	90
5.4)	Import	90
5.4.1)	ImportDevice	90
5.4.1.1)	UploadFileController	91
5.4.1.2)	FlaskFlaskImportDeviceController	91
5.4.1.3)	ImportDeviceUseCase	92
5.4.1.4)	ImportDeviceCommand	92
5.4.1.5)	AllowedDeviceFileExtension	93
5.4.1.6)	ImportDeviceService	93
5.4.1.7)	FileDeviceImporterPort	94

5.4.1.8)	FileDeviceImporter	94
5.4.1.9)	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	95
5.4.1.10)	FileDeviceImporterFactoryPort	96
5.4.1.11)	ConcreteFileDeviceImporterFactory	96
5.5)	Export	97
5.5.1)	ExportDevice	97
5.5.1.1)	FlaskExportDeviceController	98
5.5.1.2)	ExportDeviceUseCase	98
5.5.1.3)	ExportDeviceCommand	99
5.5.1.4)	ExportDeviceService	99
5.5.1.5)	FileDeviceExporterFactoryPort	100
5.5.1.6)	FileDeviceExporterPort	100
5.5.1.7)	FileDeviceExporter	101
5.5.1.8)	ConcreteFileDeviceExporterFactory	101
5.5.1.9)	CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter	102
5.5.2)	ExportReport	103
5.5.2.1)	FlaskExportReportController	103
5.5.2.2)	GenerateReportUseCase	104
5.5.2.3)	GenerateReportCommand	105
5.5.2.4)	ReportFormat	105
5.5.2.5)	GenerateReportService	105
5.5.2.6)	ReportGeneratorPort	106
5.5.2.7)	PdfReportGenerator	107
5.6)	Session	108
5.6.1)	OpenEvaluationSession	108
5.6.1.1)	EvaluationSessionController	109
5.6.1.2)	OpenEvaluationSessionCommand	109
5.6.1.3)	OpenEvaluationSessionUseCase	110
5.6.1.4)	OpenEvaluationSessionService	110
5.6.1.5)	SessionCoordinator	111
5.6.1.6)	EvaluationSessionExistPort	112
5.6.1.7)	CreateEvaluationSessionPort	112
5.6.2)	CommitEvaluationSession	113
5.6.2.1)	CommitEvaluationSessionUseCase	114
5.6.2.2)	CommitEvaluationSessionService	114
5.6.2.3)	CommitEvaluationSessionCommand	115
5.6.3)	CloseEvaluationSession	116
5.6.3.1)	CloseEvaluationSessionUseCase	116
5.6.3.2)	CloseEvaluationSessionService	117

5.6.3.3)	CloseEvaluationSessionCommand	117
5.6.3.4)	DeleteSessionPort	118
5.7)	Area navigazione	119
5.7.1)	EvaluateDecisionNode	119
5.7.1.1)	EvaluateDecisionNodeController	120
5.7.1.2)	EvaluateDecisionNodeUseCase	120
5.7.1.3)	EvaluateDecisionNodeCommand	121
5.7.1.4)	EvaluateDecisionNodeService	121
5.7.2)	InsertJustification	122
5.7.2.1)	FlaskInsertJustificationController	123
5.7.2.2)	InsertJustificationUseCase	123
5.7.2.3)	InsertJustificationCommand	124
5.7.2.4)	InsertJustificationService	124
5.8)	Architettura Frontend: Widget Semplici	125
5.8.1)	Shared	126
5.8.1.1)	Validation Rule	126
5.8.1.2)	Field Definition	127
5.8.1.3)	Use Form Model	127
5.8.1.4)	Definizioni di Dominio (Constants)	128
5.8.2)	Component	129
5.8.2.1)	AsyncButton	129
5.8.2.2)	BaseModal	130
5.8.2.3)	FormField	131
5.8.2.4)	Toast	132
5.8.2.5)	FileDropZone	133
5.8.2.6)	Device Form	134
5.8.2.7)	Asset Form	135
5.8.3)	Widget	136
5.8.3.1)	AssetCreateWidget	136
5.8.3.2)	AssetUpdateWidget	137
5.8.3.3)	AssetDeleteWidget	137
5.9)	Architettura Frontend: Modulo Decision Tree	137
5.9.1)	Architettura Interattiva della Sidebar	138
5.9.1.1)	DecisionTreeStore	139
5.9.1.2)	TreeSidebar	140
5.9.1.3)	NodeDTO	140
5.9.2)	Architettura Visiva del Canvas (Tree Canvas)	141
5.9.2.1)	LayoutResult e i DTO Geometrici	142
5.9.2.2)	LayoutEngine e D3LayoutEngine	142
5.9.2.3)	TreeCanvas	143
5.9.2.4)	UiDecisionNode e UiLeafNode	144

5.9.3)	Core di Valutazione (Domain Logic Client-side)	145
5.9.3.1)	Interfaccia Node	145
5.9.3.2)	DecisionNode	146
5.9.3.3)	LeafNode	146
5.9.3.4)	TreeStructure	147
5.9.3.5)	EvaluationEngine	147
5.9.4)	DecisionTreeStore	148
5.9.5)	EvaluationApiClient	149
6)	Tracciamento	150
7)	Gestione degli Errori	151
7.0.1)	Il Flusso di Traduzione degli Errori (Exception Mapping)	151

Lista delle tabelle

Tabella 1	Backend	3
Tabella 2	Frontend	5
Tabella 3	Schema dati: Root — Device	17
Tabella 4	Schema dati: Sub-documento — assets[N]	18
Tabella 5	Schema dati: Sub-documento — evaluations[N]	18
Tabella 6	Schema dati: Root — ComplianceStandard	20
Tabella 7	Schema dati: Sub-documento — requirements[N]	20
Tabella 8	Schema dati: Sub-documento — DecisionTree	20
Tabella 9	Schema dati: Sub-documento — DecisionNode	21
Tabella 10	Schema dati: Sub-documento — LeafNode	21

Lista delle immagini

Figura 1	Schema logico: Documento Device	17
Figura 2	Schema logico: ComplianceStandard	19
Figura 3		25
Figura 4	Modello di Dominio	25
Figura 5	Device	26
Figura 6	Asset	27
Figura 7	AssetAnagraphic	27
Figura 8	AssetProperties	28
Figura 9	AssetType	29
Figura 10	AssetEvidence	29
Figura 11	ComplianceStandard	30
Figura 12	Requirement	31
Figura 13	DecisionTree	32
Figura 14	Node	32
Figura 15	DecisionNode	33
Figura 16	LeafNode	34
Figura 17	StandardVerdict	34
Figura 18	EvaluationEngine	35
Figura 19	EvaluationState	36
Figura 20	RequirementEvaluationResult	36
Figura 21	AssetEvaluationResult	37
Figura 22	DeviceEvaluationResult	38
Figura 23	NodeDetail	39
Figura 24	RequirementEvaluationDetail	40
Figura 25	AssetEvaluationDetail	41
Figura 26	DeviceEvaluationDetail	41
Figura 27	EvaluationSession	42
Figura 28	SessionHandler	43
Figura 29	ExportedFile	43
Figura 30	DeviceSummary	44
Figura 31	Caso d'uso CreateDevice	45
Figura 32	FlaskWriteDeviceController	45
Figura 33	CreateDeviceUseCase	46
Figura 34	CreateDeviceCommand	46
Figura 35	CreateDeviceService	47
Figura 36	RegisterDevicePort	47
Figura 37	MongoDeviceAdapter	48
Figura 38	Caso d'uso DeleteDevice	49
Figura 39	DeleteDeviceUseCase	49
Figura 40	DeleteDeviceCommand	50

Figura 41	DeleteDeviceService	50
Figura 42	DeleteDevicePort	51
Figura 43	Caso d'uso UpdateDevice	51
Figura 44	UpdateDeviceUseCase	52
Figura 45	UpdateDeviceCommand	52
Figura 46	UpdateDeviceService	53
Figura 47	SaveDevicePort	53
Figura 48	FindDevicePort	54
Figura 49	GetDeviceDetail	54
Figura 50	FlaskQueryDeviceController	55
Figura 51	GetDeviceDetailUseCase	55
Figura 52	GetDeviceDetailService	56
Figura 53	GetDeviceDetailCommand	56
Figura 54	GetComplianceStandardUseCase	57
Figura 55	GetComplianceStandardService	57
Figura 56	GetComplianceStandardCommand	58
Figura 57	FindStandardPort	58
Figura 58	MongoStandardAdapter	59
Figura 59	Caso d'uso GetDeviceList	59
Figura 60	GetDeviceListUseCase	60
Figura 61	GetDeviceListService	60
Figura 62	DeviceSummary	61
Figura 63	FindAllDevicesPort	61
Figura 64	GetDeviceEvaluationDetail	62
Figura 65	FlaskDeviceEvaluationDetailController	62
Figura 66	GetDeviceEvaluationDetailCommand	63
Figura 67	GetDeviceEvaluationDetailUseCase	63
Figura 68	GetDeviceEvaluationDetailService	64
Figura 69	DeviceEvaluationDTO	65
Figura 70	Caso d'uso CreateAsset	66
Figura 71	FlaskWriteAssetController	67
Figura 72	CreateAssetUseCase	67
Figura 73	CreateAssetCommand	68
Figura 74	CreateAssetService	69
Figura 75	InMemoryEvaluationSessionCache	69
Figura 76	SaveEvaluationSessionPort	70
Figura 77	GetEvaluationSessionPort	70
Figura 78	Caso d'uso UpdateAsset	71
Figura 79	UpdateAssetUseCase	72
Figura 80	UpdateAssetCommand	72
Figura 81	UpdateAssetService	73

Figura 82	Caso d'uso DeleteAsset	74
Figura 83	DeleteAssetCommand	75
Figura 84	DeleteAssetUseCase	75
Figura 85	DeleteAssetService	76
Figura 86	Caso d'uso GetAssetAnagraphic	77
Figura 87	FlaskQueryAssetController	78
Figura 88	GetAssetAnagraphicCommand	78
Figura 89	GetAssetAnagraphicUseCase	79
Figura 90	GetAssetAnagraphicService	79
Figura 91	GetAssetEvaluationDetail	80
Figura 92	GetAssetEvaluationDetail	80
Figura 93	GetAssetEvaluationDetailCommand	81
Figura 94	GetAssetEvaluationDetailUseCase	81
Figura 95	GetAssetEvaluationDetailService	82
Figura 96	AssetEvaluationDTO	83
Figura 97	Caso d'uso GetRequirement	84
Figura 98	FlaskRequirementEvaluationDetailController	84
Figura 99	GetRequirementEvaluationDetailUseCase	85
Figura 100	GetRequirementEvaluationDetailCommand	86
Figura 101	GetRequirementEvaluationDetailService	86
Figura 102	RequirementEvaluationDTO	87
Figura 103	Caso d'uso ImportDevice	90
Figura 104	UploadFileController	91
Figura 105	FlaskImportDeviceController	91
Figura 106	ImportDeviceUseCase	92
Figura 107	ImportDeviceCommand	92
Figura 108	AllowedDeviceFileExtension	93
Figura 109	ImportDeviceService	93
Figura 110	FileDeviceImporterPort	94
Figura 111	FileDeviceImporter	94
Figura 112	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	95
Figura 113	FileDeviceImporterFactoryPort	96
Figura 114	ConcreteFileDeviceImporterFactory	96
Figura 115	Caso d'uso ExportDevice	97
Figura 116	FlaskExportDeviceController	98
Figura 117	ExportDeviceUseCase	98
Figura 118	ExportDeviceCommand	99
Figura 119	ExportDeviceService	99
Figura 120	FileDeviceExporterFactoryPort	100
Figura 121	FileDeviceExporterPort	100

Figura 122	FileDeviceExporter	101
Figura 123	ConcreteFileDeviceExporterFactory	101
Figura 124	CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter	102
Figura 125	ExportReport	103
Figura 126	ExportReportController	103
Figura 127	GenerateReportUseCase	104
Figura 128	GenerateReportCommand	105
Figura 129	GenerateReportService	105
Figura 130	ReportGeneratorPort	106
Figura 131	PdfReportGenerator	107
Figura 132	Caso d'uso OpenEvaluationSession	108
Figura 133	EvaluationSessionController	109
Figura 134	OpenEvaluationSessionCommand	109
Figura 135	OpenEvaluationSessionUseCase	110
Figura 136	OpenEvaluationSessionService	110
Figura 137	SessionCoordinator	111
Figura 138	EvaluationSessionExistPort	112
Figura 139	CreateEvaluationSessionPort	112
Figura 140	Caso d'uso CommitEvaluationSession	113
Figura 141	CommitEvaluationSessionUseCase	114
Figura 142	CommitEvaluationSessionService	114
Figura 143	CommitEvaluationSessionCommand	115
Figura 144	Caso d'uso CloseEvaluationSession	116
Figura 145	CloseEvaluationSessionUseCase	116
Figura 146	CloseEvaluationSessionService	117
Figura 147	CloseEvaluationSessionCommand	117
Figura 148	DeleteSessionPort	118
Figura 149	Caso d'uso EvaluateDecisionNode	119
Figura 150	EvaluateDecisionNodeController	120
Figura 151	EvaluateDecisionNodeUseCase	120
Figura 152	EvaluateDecisionNodeCommand	121
Figura 153	EvaluateDecisionNodeService	121
Figura 154	Caso d'uso InsertJustification	122
Figura 155	FlaskInsertJustificationController	123
Figura 156	InsertJustificationUseCase	123
Figura 157	InsertJustificationCommand	124
Figura 158	InsertJustificationService	124
Figura 159	Shared - ValidationRule	126
Figura 160	Shared - FieldDefinition	127
Figura 161	Shared - UseFormModel	127

Figura 162	Shared - Form Fields Configurations	128
Figura 163	Componente - AsyncButton	129
Figura 164	Componente - BaseModal	130
Figura 165	Componente - FormField	131
Figura 166	Componente - Toast	132
Figura 167	Componente - FileDropZone	133
Figura 168	Component - DeviceForm	134
Figura 169	Component - AssetForm	135
Figura 170	Widget - AssetCreateWidget	136
Figura 171	Architettura del componente TreeSidebar e del suo Store	138
Figura 172	DecisionTreeStore	139
Figura 173	TreeSidebar	140
Figura 174	NodeDTO	140
Figura 175	Architettura del componente TreeCanvas e del Layout Engine	141
Figura 176	LayoutResult, NodeDTO ed EdgeDTO	142
Figura 177	D3LayoutEngine	142
Figura 178	TreeCanvas	143
Figura 179	UiDecisionNode e UiLeafNode	144
Figura 180	Diagramma delle classi del Motore di Valutazione	145
Figura 181	Interfaccia Node	145
Figura 182	DecisionNode	146
Figura 183	LeafNode	146
Figura 184	TreeStructure	147
Figura 185	EvaluationEngine	147
Figura 186	Dettaglio del DecisionTreeStore (Pinia)	148
Figura 187	Dettaglio dell'EvaluationApiClient	149

1) Introduzione

1.1) Scopo del documento

Il presente documento intende fornire una descrizione approfondita dell'architettura del prodotto software, delineando con chiarezza le componenti che lo costituiscono, le loro responsabilità e le interazioni reciproche all'interno del sistema. La Specifica Tecnica funge da riferimento principale per la fase di progettazione e codifica, garantendo coerenza con i requisiti analizzati e consolidando la maturità architetturale del prodotto.

Nello specifico, questo documento si propone di:

- **Definire l'architettura logica e i design pattern:** descrivere l'organizzazione dei moduli e le logiche di interazione, evidenziando come i pattern adottati garantiscano un codice modulare e testabile.
- **Motivare lo stack tecnologico:** giustificare la scelta delle tecnologie utilizzate in funzione della scalabilità del sistema e della qualità complessiva del prodotto finale.
- **Delineare l'architettura di deployment:** illustrare la distribuzione delle componenti negli ambienti di esecuzione e le strategie di rilascio adottate.
- **Fornire un framework per l'evoluzione del software :** stabilire le linee guida necessarie per facilitare la comprensione del sistema, agevolando futuri interventi di estensione e manutenzione.

1.2) Scopo del prodotto

Il prodotto si prefigge di automatizzare e digitalizzare il processo di verifica della conformità alla normativa EN 18031, sostituendo le attuali procedure manuali, spesso onerose e soggette a errore umano, con una soluzione software interattiva.

Il sistema è progettato per guidare l'utente attraverso l'esecuzione di alberi decisionali (Decision Tree) complessi, permettendo la valutazione sistematica dei requisiti di sicurezza per dispositivi IoT e la generazione di esiti certi (Pass, Fail, N.A.). L'integrazione di funzionalità per l'importazione di dati strutturati e una dashboard di monitoraggio in tempo reale mira a garantire la massima tracciabilità del processo, ottimizzando i tempi operativi della proponente e assicurando un elevato standard di affidabilità e manutenibilità dei risultati ottenuti.

1.3) Glossario

Per garantire precisione terminologica senza appesantire la lettura, in questo documento i termini tecnici presenti nel Glossario sono segnalati con un pedice "G", ad esempio:

termine_G: indica che il termine è definito nel Glossario e può essere consultato per chiarimenti.

Questo metodo consente di mantenere il testo chiaro e tecnicamente corretto, permettendo al lettore di riferirsi al Glossario solo quando necessario, senza interrompere il flusso della lettura.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl](#)
Ultima consultazione: 8 aprile 2026;
- [Regolamento del progetto](#)
Ultima consultazione: 8 aprile 2026;
- [Standard ISO/IEC/IEEE 12207:2017](#)
Ultima consultazione: 10 gennaio 2026;
- [Standard ISO/IEC 9126](#)
Ultima consultazione: 10 gennaio 2026;

1.4.2) Riferimenti informativi

- [Glossario del gruppo](#)
Ultima consultazione: 8 aprile 2026;
- [Software Engineering, Ian Sommerville](#)
Ultima consultazione: 10 gennaio 2026;
- [Documentazione del gruppo GroupRubberDuck](#)
Ultima consultazione: 8 aprile 2026;

2) Tecnologie

In questa sezione vengono descritte le tecnologie utilizzate nello sviluppo del progetto **Automated EN18031 Compliance Verification**.

Per ogni tecnologia è riportata la versione di riferimento e il relativo ruolo all'interno del progetto

2.1) Backend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
Python	3.12.X	Linguaggio interpretato scelto per la rapidità di sviluppo, l'alta leggibilità e il vasto ecosistema.
Framework Principale		
Flask	3.1.3	Micro-framework web utilizzato come infrastruttura principale di backend. Consente una gestione flessibile e leggera del routing per esporre le interfacce API. Buona documentazione e possibilità di confronto tecnico con la proponente.
Dipendenze principali		
Waitress	3.0.2	Server WSGI leggero. Gestisce la concorrenza delle richieste in modo affidabile.
Fpdf		Libreria per la generazione dinamica di documenti e reportistica in formato PDF direttamente lato server.
Pymongo		Driver ufficiale per l'integrazione con MongoDB.
Pydantic		Utilizzato per la validazione rigida e type-safe dei dati in ingresso e delle variabili d'ambiente di sistema.
Python-dotenv		Gestione semplificata delle configurazioni e delle variabili d'ambiente.
Jinja2		Template engine HTML con un'ottima integrazione con Flask.

Nome	Versione	Descrizione
Werkzeug		Server di sviluppo locale.
Watchdog		Permette l'hot reloading, passando le modifiche al server di sviluppo senza necessità di riavvio e preservando lo stato dell'applicazione.
Dynamic Testing		
Pytest		Framework di testing adottato per la sua sintassi concisa. Utilizzato per automatizzare i «Sanity Tests» e validare l'integrazione di sistema alla radice del progetto.
Static Testing		
Ruff		Lint e formater estremamente veloce (scritto in Rust). Impone e garantisce standard di codice puliti e uniformi senza rallentare lo sviluppo.
Mypy		Analizzatore statico basato sul Type Hinting. Previene i bug e gli errori di tipo a tempo di sviluppo prima ancora dell'esecuzione
Sviluppo		
Poetry		Gestore moderno delle dipendenze e degli ambienti virtuali (.venv). Garantisce build riproducibili tra i vari sviluppatori tramite il file di blocco (poetry.lock).

Tabella 1: Backend

2.2) Frontend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
JavaScript		Linguaggio client-side universale.
Framework Principale		
Vue.js		Framework progressivo con una curva di apprendimento morbida e un'ottima implementazione della reattività. L'adozione della Composition API permette di scrivere logica modulare e riutilizzabile.
Dipendenze principali		
pinia		Gestore dello stato ufficiale e leggero per Vue.
tailwindcss		Framework CSS utility-first per un'agile stilizzazione dell'interfaccia.
Dynamic Testing		
vitest		Framework di unit testing veloce e nativo per Vite. Utilizzato per validare la logica isolata dei componenti.
vue-test-utils		Libreria ufficiale affiancata a Vitest per montare e simulare le interazioni dell'utente sulle singole isole Vue durante i test.
Sviluppo		
vite		Tool di build per minificare e raggruppare i file JS/CSS corrispondenti ai componenti Vue, ottimizzandoli per la produzione.

Tabella 2: Frontend

2.3) Persistenza dei dati

MongoDB è un database NoSQL orientato ai documenti. A differenza dei classici database relazionali (che usano tabelle e colonne rigide), archivia le informazioni in documenti flessibili molto simili al formato JSON.

La versione utilizzata è la 7.XX

Permette di aggiungere o rimuovere campi dai dati senza riscrivere l'intera organizzazione delle informazioni del database.

Nel progetto **Automated EN18031 Compliance Verification** viene utilizzato come sistema per la persistenza dei dati, in particolare per la rappresentazione dei dispositivi sottoposti alle valutazioni e per la rappresentazione del modello di standard tramite configurazioni esterne.

Il suo utilizzo permette di rappresentare facilmente strutture dati annidate, tramite una sintassi JSON.

Evita la gestione di join complessi tipici di un database relazionale, offre una maggiore sicurezza rispetto alla gestione manuale di file di rappresentazione interni.

È facilmente integrabile nel sistema backend

3) Architettura di Sistema

3.1) Premessa: approccio di lavoro usato

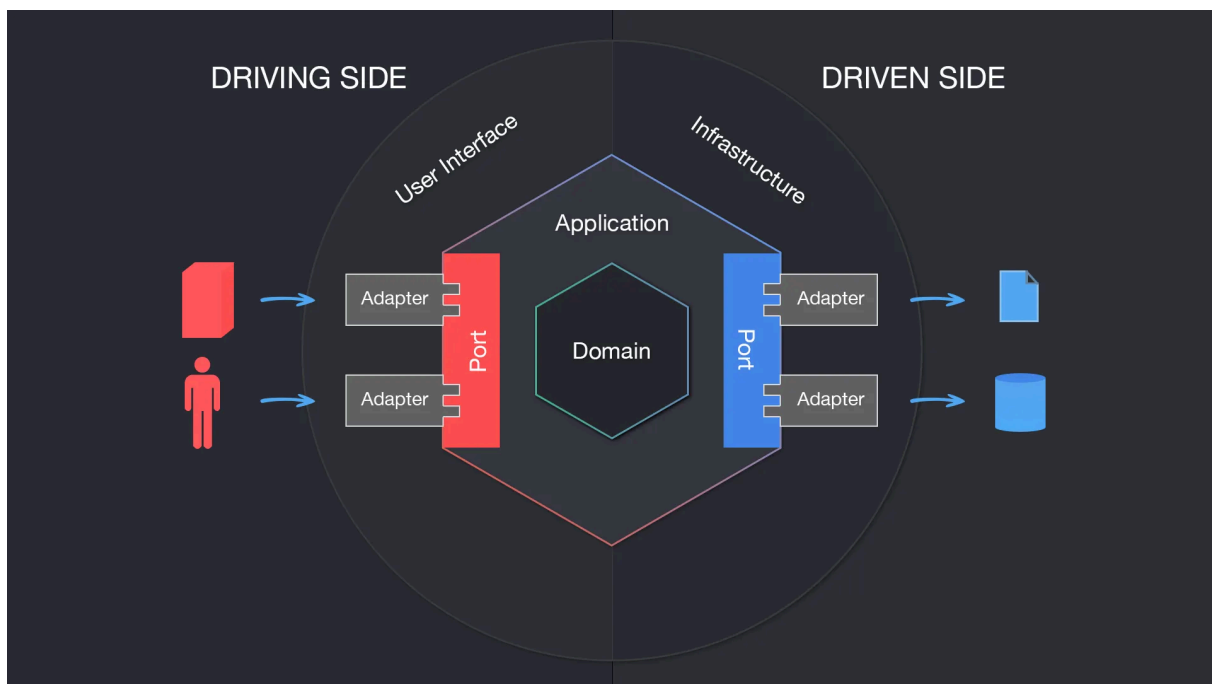
L'obiettivo della fase di progettazione è definire un'architettura software in grado di soddisfare pienamente i requisiti di sistema, operando secondo criteri di sostenibilità e ottimizzazione delle risorse (design-to-cost).

La metodologia adottata per la definizione dell'architettura è di tipo Top-Down. Il sistema viene inizialmente concepito nella sua interezza come un'unica entità astratta e, attraverso un processo di esplorazione funzionale, viene progressivamente decomposto in sotto-sistemi logici più piccoli e gestibili.

3.2) Architettura Logica

3.2.1) Stile architetturale

L'architettura adottata per realizzare il prodotto è l'architettura esagonale. Questo tipo di architettura ha come obiettivo primario isolare la logica di business dal resto. Questo approccio garantisce un'elevata testabilità del sistema e rende il nucleo del software completamente agnostico rispetto ai dettagli implementativi (framework, database o interfacce). Il sistema è strutturato in livelli concentrici, organizzati come segue:



- **Domain (Core):** Rappresenta il nucleo centrale dell'applicazione. Contiene la logica di business pura ed è rigorosamente privo di dipendenze esterne.
- **Services:** Definiscono i casi d'uso dell'applicazione (Application Services) e fungono da orchestratori. Implementano le Inbound Ports per gestire le richieste in ingresso, coordinano gli oggetti del Domain affinché eseguano la logica di business pura

e utilizzano le Outbound Ports per delegare all'esterno operazioni infrastrutturali (come il salvataggio su database).

- **Ports:** Costituiscono il punto di connessione tra il nucleo e il mondo esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in Inbound Ports (definiscono i casi d'uso accessibili dall'esterno) e Outbound Ports (permettono al nucleo di definire interfacce per interagire con i servizi esterni).
- **Adapters:** Rappresentano lo strato più esterno e fungono da traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in Driving/Input Adapters (adattatori in entrata, che guidano l'applicazione ricevendo input e invocando le Inbound Ports) e Driven/Output Adapters (adattatori in uscita, che vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le Outbound Ports).

3.2.2) Motivazioni della scelta architetturale

Le motivazioni per le quali questa architettura è stata scelta sono le seguenti:

- **Testabilità:** l'assenza di dipendenze esterne nel livello di Dominio permette di scrivere Unit Test in puro Python in modo semplice e veloce. È possibile testare la logica di business in modo isolato, iniettando mock o stub per simulare le interfacce, senza la necessità di avviare il server Flask o connettersi al database MongoDB.
- **Divisione del nucleo:** il nucleo dell'applicazione è totalmente disaccoppiato dall'infrastruttura. Essendo ignaro del livello di presentazione (l'interfaccia utente in HTML, CSS e Vue.js) e del livello di persistenza (il database NoSQL MongoDB), il sistema garantisce un'alta tolleranza ai cambiamenti. È possibile sostituire o aggiornare le tecnologie esterne senza dover alterare la logica di business.
- **Sviluppo parallelo:** la rigorosa definizione dei contratti di comunicazione (le Porte) nelle fasi iniziali di progettazione permette al team di procedere in parallelo. Frontend, backend e integrazione con il database possono essere sviluppati simultaneamente da membri diversi del gruppo, riducendo i colli di bottiglia.
- **Inversione delle dipendenze:** l'architettura applica rigorosamente questo principio poiché è la parte interna dell'esagono a dettare i contratti (le interfacce) a cui i servizi esterni devono sottostare, e non viceversa. In questo modo, il nucleo dell'applicazione mantiene il controllo assoluto del flusso ed è protetto dai potenziali effetti collaterali (side effects) derivanti dall'infrastruttura.

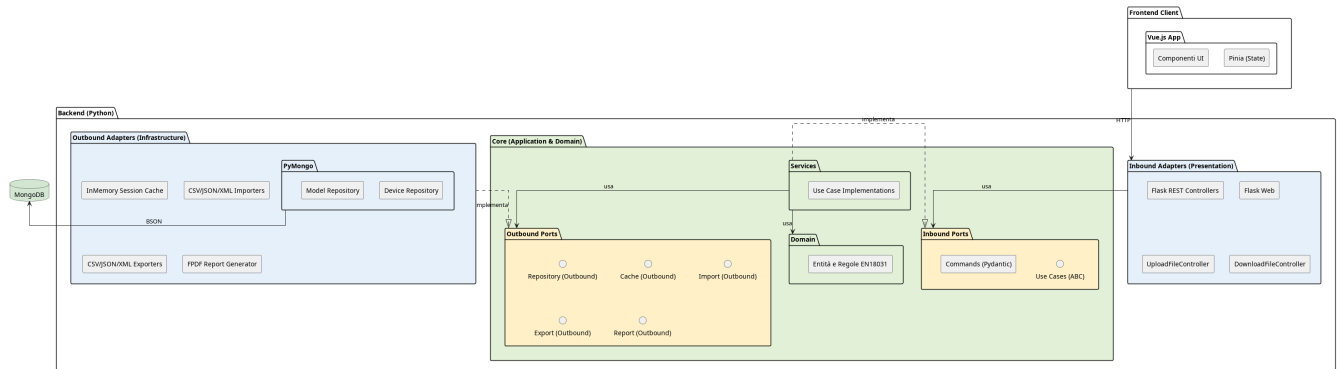
3.2.3) Limiti dell'architettura

Gli aspetti negativi di questa scelta sono:

- **Ripida curva di apprendimento:** l'architettura esagonale richiede una profonda comprensione dei principi SOLID, in particolare la Dependency Injection e usare questo pattern per la prima volta richiede profondo studio. Questo rischio sarà mitigato da una precisa fase di progettazione che semplificherà la codifica.
- **Elevato overhead iniziale:** La ferrea separazione dei livelli impone la stesura di un'abbondante quantità di codice infrastrutturale (boilerplate). Risulta necessario

definire contratti astratti (Porte), implementazioni concrete (Adattatori) e orchestratori (Servizi), allungando i tempi di sviluppo nelle prime fasi del progetto. Il gruppo ha tuttavia accettato questo costo iniziale, ritenendolo un investimento necessario a fronte del drastico abbattimento dei futuri costi di manutenzione e della massima testabilità garantita al nucleo applicativo.

3.2.4) Diagramma dei package



Il diagramma illustra l'organizzazione logica del sistema Automated EN18031 Compliance Verification, fondata sull'architettura esagonale. Il sistema è strutturalmente ripartito in un livello di presentazione esterno (Frontend Client), sviluppato tramite il framework Vue.js, e un nucleo applicativo (Backend), implementato in Python.

Per garantire una rigorosa separazione delle responsabilità e il pieno rispetto del principio di Inversione delle Dipendenze, il Backend si articola nei seguenti livelli tipici dell'architettura esagonale:

1. **Adapters:** Costituisce il confine tecnologico del sistema, responsabile della comunicazione con l'esterno. Si divide in:
 - **Inbound Adapters** (Driving Adapters): Agiscono da punto di ingresso per il sistema. Intercettano gli input esterni (es. le richieste HTTP/REST inviate dal Frontend), ne effettuano il parsing e traducono tali direttive in invocazioni dirette verso le Inbound Ports, senza mai accedere alla logica di business.
 - **Outbound Adapters** (Driven Adapters): Rappresentano le tecnologie concrete di uscita (es. il driver MongoDB o le librerie di generazione PDF). Il loro compito è implementare materialmente le interfacce definite nel livello Outbound Ports, traducendo i comandi astratti in operazioni specifiche per l'infrastruttura.
2. **Core** (Nucleo Applicativo): È il cuore del software, totalmente agnostico e privo di dipendenze verso framework web o database. Al suo interno è stratificato in:
 - **Ports:** Funge da barriera di isolamento. Definisce le interfacce in puro Python, disaccoppiando la tecnologia dall'applicazione:
 - **Inbound Ports** (Primary Ports): Definiscono le interfacce (i contratti) relative ai Casi d'Uso (Use Cases) che il sistema offre. Specificano «cosa» il sistema è

in grado di fare, permettendo agli Inbound Adapters di invocare le operazioni senza dover conoscere l'implementazione sottostante.

- **Outbound Ports** (Secondary Ports): Definiscono i contratti astratti di cui il dominio ha bisogno per comunicare con l'esterno (ad esempio, le interfacce del Repository Pattern per l'accesso ai dati). Queste porte vengono usate dal livello Services e implementate dagli Outbound Adapters.
- **Services**: Costituiscono il livello di orchestrazione. Implementano concretamente i Casi d'Uso definiti nelle Inbound Ports, coordinano il flusso di esecuzione richiamando le entità di dominio, e si avvalgono delle Outbound Ports per la persistenza dei dati.
- **Domain**: Incapsula le regole di business pure e la logica della norma EN18031. Contiene la modellazione formale delle entità e la validazione strutturale dei dati (supportata da Pydantic), operando in totale e completo isolamento.

3.3) Architettura di Deployment

Il sistema adotta un'architettura a Monolite Modulare.

L'intera applicazione è concepita, sviluppata e rilasciata come un'unica unità eseguibile.

3.3.1) Motivazioni della scelta

Questa scelta è coerente con lo sviluppo di una web app locale.

In questo contesto un'architettura a monolite modulare, rispetto all'architettura a microservizi, offre i seguenti vantaggi:

- **Semplicità di Deployment**: tutte le funzionalità sono contenute nella singola unità operativa;
- **Latenze ridotte**: lo scambio di informazioni avviene completamente in locale.

Non si è optato per un'architettura monolitica tradizionale in cui spesso il codice è fortemente accoppiato.

La rigorosa divisione in moduli logici isolati permette di coniugare la semplicità di rilascio dell'applicazione come singola unità con alcuni dei vantaggi organizzativi tipici delle architetture a microservizi:

- **Separazione delle responsabilità** tra i vari ambiti del dominio.
- **Semplicità di sviluppo e manutenibilità** a lungo termine.
- **Facilità di testing**, permettendo di collaudare i singoli moduli in totale isolamento.

3.3.2) Realizzazione Pratica e Topologia

Per la distribuzione viene usato Docker come tool di containerizzazione e Docker Compose come tool di orchestrazione.

1. Nodi di Esecuzione:

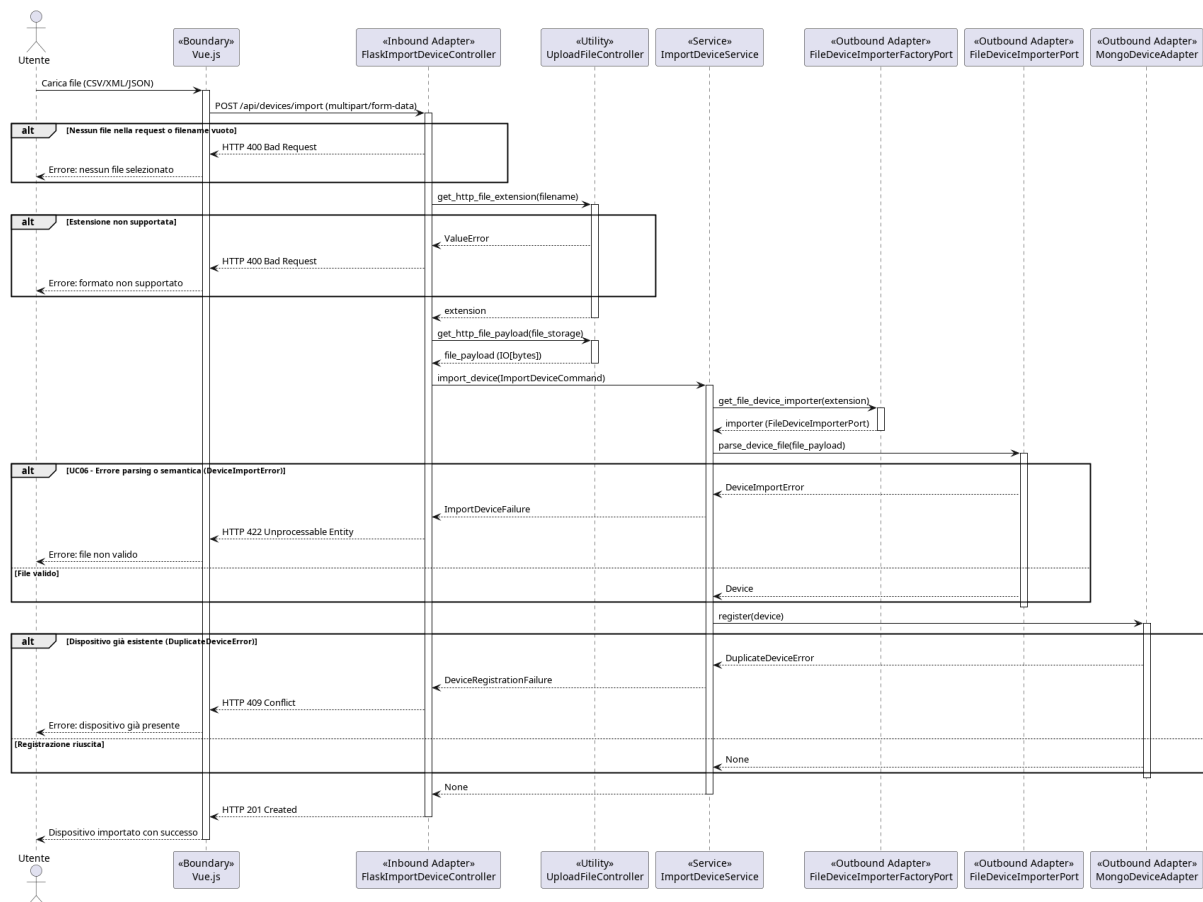
- **Browser dell'utente**, l'interfaccia utente è realizzata tramite browser;
 - **Server Backend**, contiene la core logic dell'applicazione e risponde alle richieste dell'utente;
 - **Database MongoDB**, sistema di permanenza;
2. **Packaging e isolamento**. L'infrastruttura è orchestrata tramite Docker Compose e prevede i seguenti container:
- **Web-app**
Contiene l'ambiente Python, il framework Flask, gli asset statici del frontend e altre dipendenze¹
 - **MongoDB**
Basato sull'immagine ufficiale di MongoDB. Per prevenire la perdita dei dati questo container è agganciato a un Docker Volume locale.
3. **Comunicazioni e protocolli di rete**
- I nodi comunicano attraverso i seguenti canali e protocolli:
- **Tra Browser e Backend:**
La comunicazione avviene in chiaro tramite protocollo HTTP/REST
 - **Tra Backend e Database:** Flask comunica con MongoDB utilizzando il protocollo binario proprietario.
- La porta del database non è esposta verso il browser né verso l'esterno, garantendo l'integrità dei dati.
4. **Gestione del Traffico Web**
- Tra il browser dell'utente e l'applicazione Python è interposto un server **WSGI**.
- Questo strato intermedio garantisce stabilità, gestisce la concorrenza delle richieste e le instrada in modo sicuro verso la logica applicativa.

3.4) Diagrammi di sequenza

La seguente sezione illustra il comportamento dinamico del sistema tramite diagrammi di sequenza, focalizzandosi sui casi d'uso di maggiore interesse. Questi modelli descrivono l'ordine cronologico dei messaggi scambiati tra gli attori esterni, i componenti infrastrutturali e il nucleo applicativo.

¹Vedi sezione [Tecnologie - Backend](#)

3.4.1) UC05 - Importazione dispositivo



Il caso d'uso consente all'utente di caricare un file (CSV, JSON o XML) contenente i dati di un dispositivo e di registrarlo nel sistema. Il flusso coinvolge tre componenti principali: il controller HTTP, il servizio applicativo e il repository.

Flusso Principale

1. Ricezione della richiesta HTTP:

il FlaskImportDeviceController riceve una richiesta POST /devices/import. Prima di delegare al servizio, verifica la presenza del file nella richiesta e l'estensione dello stesso. Se il file è assente o l'estensione non è supportata, restituisce immediatamente HTTP 400 senza propagare la richiesta al layer applicativo.

2. Costruzione del Command:

Se la validazione HTTP supera il controllo, il controller costruisce un ImportDeviceCommand contenente il percorso del file e il formato dichiarato, e lo passa all'ImportDeviceService.

3. Parsing del file:

il servizio delega il parsing al FileDeviceImporterPort tramite il metodo parse_device_file. Il port è implementato dall'adapter corretto in base al formato del file (CSV, JSON, XML). Il parser legge il file e, se il formato è valido e i dati sono semanticamente corretti, restituisce direttamente un oggetto Device del dominio.

4. Registrazione del dispositivo:

il Device restituito dal parser viene passato al DeviceRepositoryPort tramite il metodo register. Il repository persiste il dispositivo nel database.

5. Risposta di successo:

Al termine del flusso, il controller restituisce HTTP 201 Created.

Flussi alternativi

- **[File non valido o formato non supportato]**

Se il file è assente nella richiesta o l'estensione non rientra tra quelle accettate (csv, json, xml), il FlaskImportDeviceController restituisce HTTP 400 Bad Request. Il servizio non viene mai invocato.

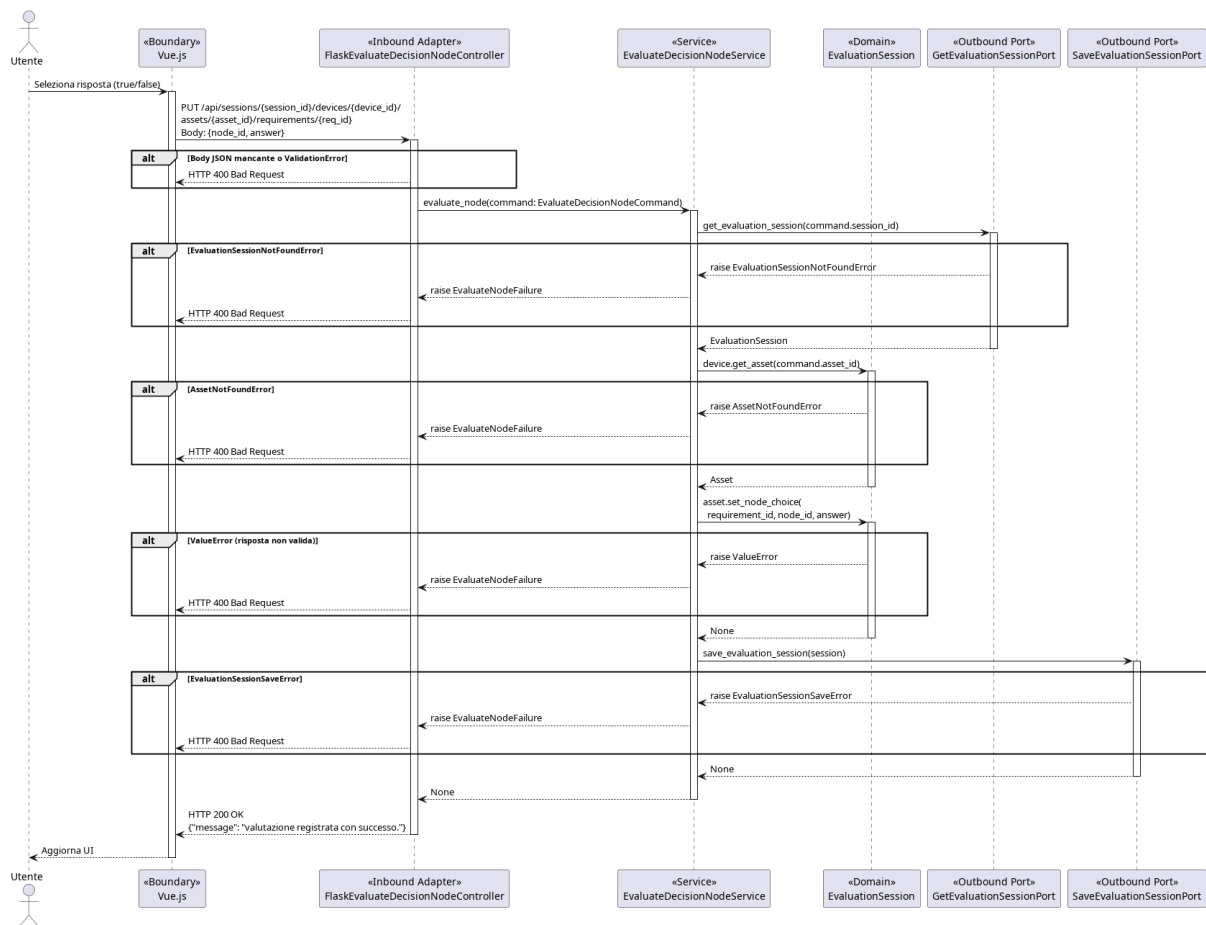
- **[Errore di parsing o dati non validi]**

Se il file non è leggibile, ha una struttura malformata, o contiene dati semanticamente non validi (es. campo obbligatorio mancante, tipo non riconosciuto), il FileDeviceImporterPort solleva una DeviceImportError. Il servizio cattura l'eccezione e la traduce in un ImportErrorFailure, che il controller mappa in HTTP 422 Unprocessable Entity.

- **[Dispositivo già registrato]**

Se un dispositivo con lo stesso identificatore è già presente nel repository, register() solleva una DuplicateDeviceError. Il servizio la traduce in un DeviceRegistrationFailure, che il controller mappa in HTTP 409 Conflict.

3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato



Il caso d'uso consente all'utente di registrare la risposta a un nodo decisionale nell'albero di valutazione di un requisito. Il flusso coinvolge il controller HTTP, il servizio applicativo, due port distinte per la sessione e gli oggetti di dominio EvaluationSession e Asset.

Flusso principale:

1. Ricezione della richiesta HTTP:

Il FlaskEvaluateDecisionNodeController riceve una richiesta PUT sull'endpoint `/api/sessions/{session_id}/devices/{device_id}/assets/{asset_id}/requirements/{req_id}`. Prima di procedere, verifica la presenza del body JSON. Se assente, restituisce immediatamente HTTP 400 senza invocare il service.

2. Costruzione del Command:

Il controller istanzia un EvaluateDecisionNodeCommand (modello Pydantic) con i parametri di path e i campi `node_id` e `answer` estratti dal body. Se la validazione Pydantic fallisce per campi mancanti o tipo non valido, restituisce HTTP 400 senza propagare la richiesta al layer applicativo.

3. Recupero della sessione:

L'EvaluateDecisionNodeService riceve il command e delega il recupero della ses-

sione alla `GetEvaluationSessionPort` tramite `get_evaluation_session(session_id)`. La port restituisce l'`EvaluationSession` completa, incluso il device e i suoi asset.

4. Navigazione al dominio:

Il service naviga l'aggregato di dominio per recuperare l'asset su cui operare tramite `session.device.get_asset(asset_id)`, ottenendo l'oggetto `Asset` corrispondente.

5. Registrazione della scelta:

Il service invoca `asset.set_node_choice(requirement_id, node_id, answer)` sull'oggetto `Asset`. Questa chiamata registra in memoria la risposta dell'utente per il nodo specificato all'interno del requisito.

6. Persistenza della sessione:

Il service passa la `EvaluationSession` aggiornata alla `SaveEvaluationSessionPort` tramite `save_evaluation_session(session)`. La responsabilità di persistere l'intera sessione è delegata alla port: il service non conosce il meccanismo di storage.

7. Risposta di successo:

Il service restituisce `None`. Il controller risponde con HTTP 200 OK e il messaggio `{«message»: «valutazione registrata con successo.»}`.

Flussi alternativi:

- **[Body JSON mancante o non valido]**

Se il body della richiesta è assente o `node_id/answer` non rispettano i tipi attesi da *EvaluateDecisionNodeCommand*, il controller restituisce HTTP 400 Bad Request. Il service non viene mai invocato.

- **[Sessione non trovata]**

Se la *GetEvaluationSessionPort* non trova una sessione con l'identificatore fornito, solleva *EvaluationSessionNotFoundError*. Il service cattura l'eccezione e la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Asset non trovato]**

Se il device nella sessione non contiene un asset con l'identificatore fornito, il dominio solleva *AssetNotFoundError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Risposta non valida per il nodo]**

Se *set_node_choice* riceve un valore non ammesso per il nodo specificato, il dominio solleva *ValueError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Errore di salvataggio]**

Se la *SaveEvaluationSessionPort* non riesce a persistere la sessione, solleva *EvaluationSessionSaveError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

3.5) Schema dati

In questa sezione viene illustrata l'organizzazione logica dei dati all'interno del database MongoDB.

3.5.1) Scelte Architettureali e Formalismo

A differenza dei sistemi relazionali basati su tabelle, MongoDB è un database **NoSQL orientato ai documenti** con struttura *schema-flexible*. Questa scelta architetturale è ideale per gestire strutture dati gerarchiche e dinamiche (come gli alberi decisionali), evitando il sovraccarico computazionale derivante da query di JOIN complesse.

Per la modellazione visiva non si utilizzano diagrammi Entity-Relationship, in quanto non idonei ai database documentali.

Si adotta invece il formalismo di Hackolade (consultabile al link: https://hackolade.com/schemas/Yelp_Challenge_dataset_documentation.html), che descrive chiaramente la gerarchia dei campi, i tipi di dato e le nidificazioni.

Nota sulla validazione dei dati: Benché i documenti vengano illustrati con tipi logici (incluso il tipo `enum` per chiarezza espositiva), la validazione strutturale di base sarà garantita dall'adapter predisposto alle comunicazioni col database.

Le due *collection* principali del database sono:

- **Collection «Compliance Standards»:** Catalogo di sola lettura dei template. Contiene le definizioni degli standard, le versioni e la logica degli alberi decisionali.
- **Collection «Devices»:** Registro dinamico dei device censiti. Memorizza lo stato delle valutazioni e il riferimento puntuale al modello normativo applicato.

3.5.2) Schema Dati Device

Il documento *Device* è l'entità centrale del sistema. La sua struttura gerarchica rispecchia la scomposizione fisica e funzionale dell'oggetto analizzato in tre blocchi logici:

1. **Header Identificativo (Root):** Informazioni anagrafiche, tecniche e puntatore al modello normativo.
2. **Lista Asset (Nodi Figli):** Array di sub-documenti che modellano le singole componenti o funzionalità del dispositivo.
3. **Registro Valutazioni (Nodi Terminali):** Dizionario delle risposte fornite per i requisiti applicabili a ciascun asset.

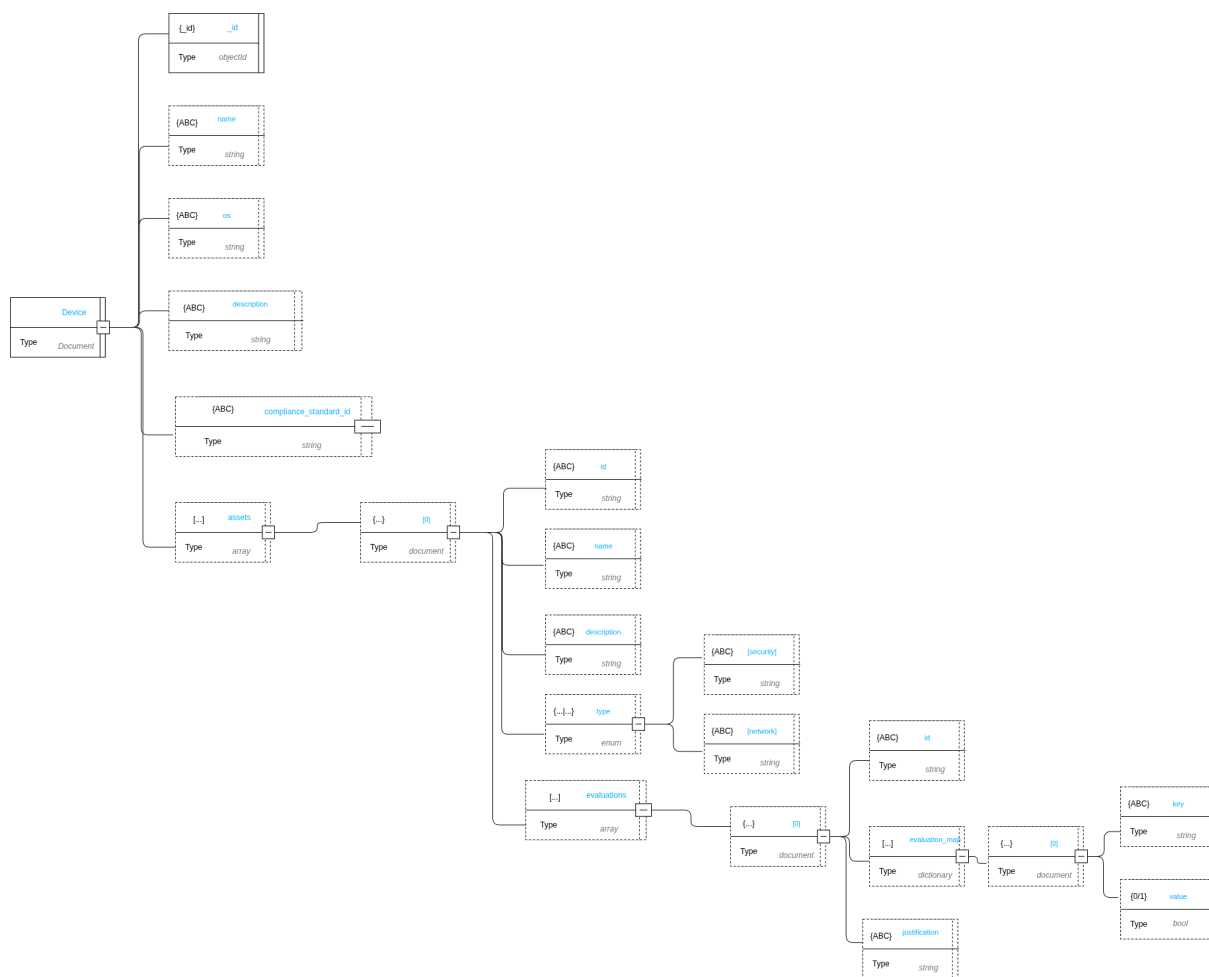


Figura 1: Schema logico: Documento Device

Root --- Device

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco generato dal database.
<code>name</code>	string	Nome assegnato dall'utente al dispositivo. Valore obbligatorio.
<code>os</code>	string	Sistema operativo o firmware installato.
<code>description</code>	string	Testo libero descrittivo del dispositivo.
<code>compliance_standard_id</code>	string	Id del modello di riferimento
<code>assets</code>	array	Elenco degli asset associati al dispositivo. Può essere inizializzato come array vuoto <code>[]</code> .

Tabella 3: Schema dati: Root — Device

Sub-documento --- assets[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	ID applicativo per l'identificazione univoca dell'asset nel dispositivo.
<code>name</code>	string	Nome descrittivo dell'asset.
<code>description</code>	string	Testo libero descrittivo dell'asset.
<code>type</code>	enum	Categoria logica dell'asset. Valori ammessi: <code>security</code> , <code>network</code> .
<code>evaluations</code>	array	Elenco delle valutazioni fornite per questo asset. Può essere vuoto <code>[]</code> .

Tabella 4: Schema dati: Sub-documento — assets[N]

Sub-documento --- evaluations[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Riferimento al codice del requisito presente nel modello.
<code>evaluation_map</code>	dictionary	Mappa chiave-valore delle risposte. La chiave (string) corrisponde al codice del nodo decisionale, il valore <code>bool</code> rappresenta la risposta associata.
<code>justification</code>	string	Testo descrittivo della motivazione. Obbligatorio a livello applicativo in caso di esito <code>FAIL</code> o <code>NA</code> .

Tabella 5: Schema dati: Sub-documento — evaluations[N]

3.5.3) Schema ComplianceStandard

La collection «Modelli» contiene la definizione strutturale degli standard normativi. A differenza del Dispositivo, orientato allo stato, il Modello descrive la logica di valutazione. Si suddivide in:

- **Metadati:** Versionamento e identificazione.
- **Albero dei Requisiti:** Struttura che definisce il testo della norma e la logica decisionale per guidare l'utente alla valutazione finale.

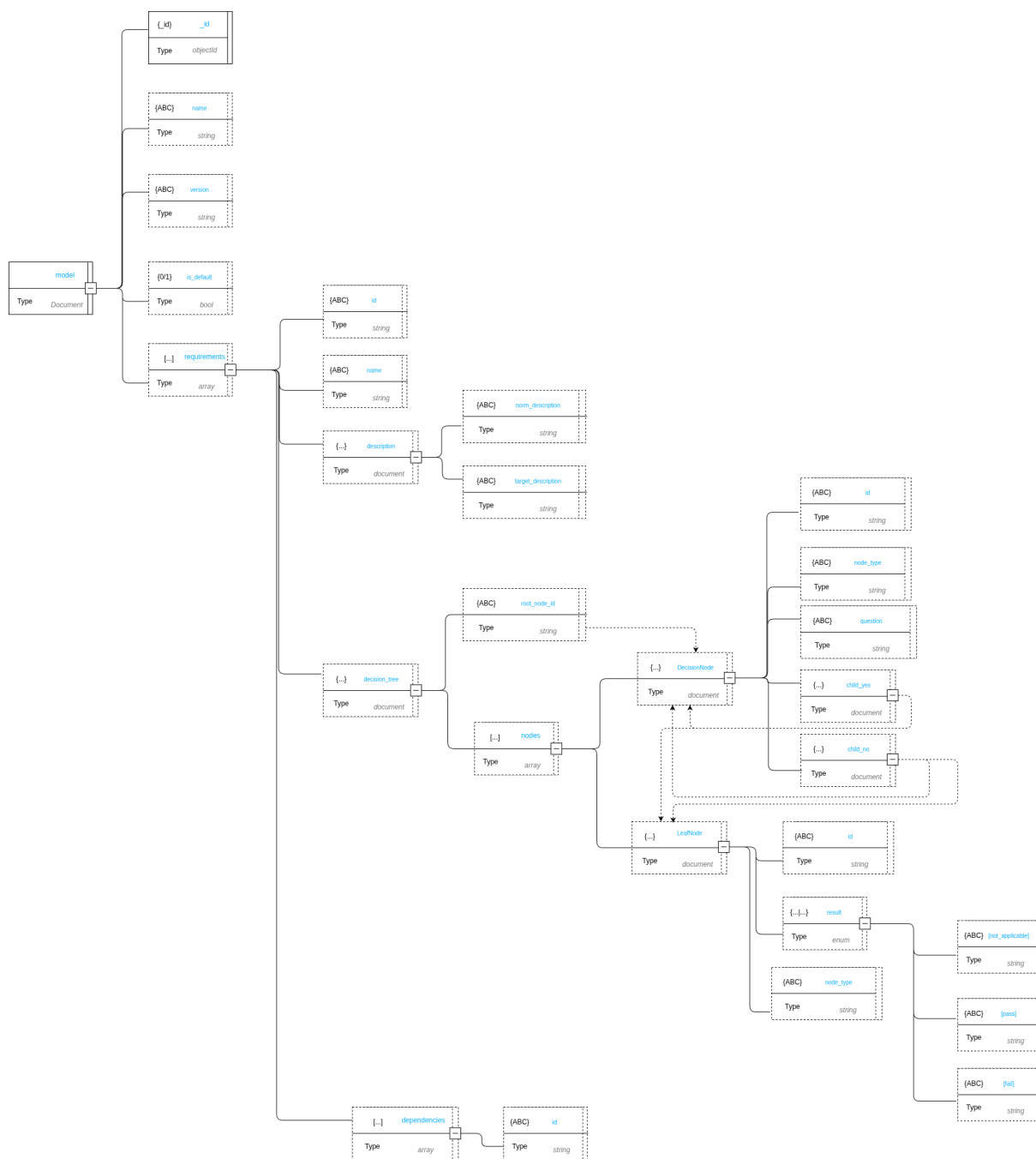


Figura 2: Schema logico: ComplianceStandard

Root --- ComplianceStandard

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco del modello.
<code>name</code>	string	Nome dello standard normativo.
<code>version</code>	string	Identificativo della versione del modello.
<code>is_default</code>	bool	Flag che indica se il modello è quello predefinito per nuove valutazioni.
<code>requirements</code>	array	Elenco dei requisiti che compongono la normativa.

Tabella 6: Schema dati: Root — ComplianceStandard

Sub-documento --- requirements[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Identificativo testuale univoco del requisito.
<code>name</code>	string	Titolo sintetico del requisito.
<code>description</code>	document	Testo descrittivi del contesto normativo (<code>norm_description</code> , <code>target_description</code>).
<code>decision_tree</code>	document	Albero decisionale per questo requisito.
<code>dependencies</code>	array	Elenco di codici di requisiti propedeutici da soddisfare prima della valutazione.

Tabella 7: Schema dati: Sub-documento — requirements[N]

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>root_node</code>	string	Codice identificativo del nodo logico iniziale.
<code>nodes</code>	array	Lista dei nodi che compongono il decision tree. Il <code>node_type</code> funge da discriminante tra nodi di decisione e nodi foglia. Valori ammessi da <code>node_type</code> : <code>decision_node</code> <code>leaf_node</code> .

Tabella 8: Schema dati: Sub-documento — DecisionTree

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo logico.
<code>node_type</code>	enum	<code>decision_node</code> , identifica un nodo di decisione.
<code>question</code>	string	Testo della domanda posta all'utente.
<code>child_yes</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta affermativa.
<code>child_no</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta negativa.

Tabella 9: Schema dati: Sub-documento — DecisionNode

Sub-documento --- LeafNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo foglia.
<code>node_type</code>	enum	<code>leaf_node</code> , identifica un nodo foglia.
<code>result</code>	enum	Stato terminale del percorso logico. Valori ammessi: <code>PASS</code> , <code>FAIL</code> , <code>NA</code> .

Tabella 10: Schema dati: Sub-documento — LeafNode

3.5.4) Gestione degli Esiti e Storicità

La separazione netta tra Modello e Dispositivo ottimizza la memorizzazione e la gestione dei dati portando i seguenti vantaggi:

- **Zero Ridondanza:** Il documento Dispositivo salva unicamente il dizionario delle risposte fornite, mentre il documento Modello conserva esclusivamente la struttura statica dell'albero. Questo evita di duplicare l'intera gerarchia normativa per ogni dispositivo.
- **Disaccoppiamento:** Viene lasciato alla logica applicativa il compito di «fondere» i dati, compilando a runtime il template del Modello con le risposte storicizzate nel Dispositivo.
- **Versionamento:** La collection Modelli conserva tutte le versioni rilasciate di uno standard. Poiché il Dispositivo punta esplicitamente a una singola e specifica versione, lo storico delle certificazioni passate rimane inalterato e coerente anche a fronte di futuri aggiornamenti normativi.

4) Design Pattern

4.1) Principi di Design

4.1.1) Dependency Injection

Descrizione del pattern

La dependency injection prevede che le dipendenze necessarie a una classe o a un modulo non vengano create internamente, ma fornite dall'esterno.

Generalmente questo avviene alla costruzione, tramite un setter o come parametro di un metodo.

Motivazioni dell'utilizzo del pattern

La dependency injection permette di realizzare la Dependency Inversion e garantisce che le classi di dominio non dipendano mai da servizi secondari, framework o tecnologie esterne.

Utilizzo del pattern nel progetto

All'interno del progetto viene usata la Constructor Injection. Il factory di Flask (`create_app()`) funge da **Composition Root**: è l'unico punto centralizzato del sistema a conoscere le implementazioni concrete delle porte e ad occuparsi di iniettarle nei service applicativi.

4.2) Design Pattern Creazionali

4.2.1) Factory

Descrizione del pattern

Il pattern Factory è un pattern creazionale che astrae il processo di creazione degli oggetti.

Delega a un componente specifico la responsabilità di creare oggetti complessi, restituendoli attraverso un'interfaccia comune.

Motivazioni dell'utilizzo del pattern

In un'architettura esagonale, i layer di dominio devono dipendere da interfacce (Porte) e mai da implementazioni concrete.

Se un service dovesse istanziare direttamente le proprie dipendenze, finirebbe per accoppiarsi a librerie o tecnologie esterne.

Il Factory risolve questo problema incapsulando la conoscenza di «come» e «quale» oggetto concreto creare.

Utilizzo del pattern nel progetto

Nel progetto il pattern è applicato a due livelli distinti:

1. **Application Factory:** A livello di framework, la funzione `create_app()` di Flask agisce come macro-factory, assemblando l'applicazione e le sue dipendenze principali all'avvio.

2. **Domain/Service Factory:** A livello applicativo, vengono utilizzate factory specifiche per costruire a runtime le implementazioni corrette di alcune interfacce che richiedono una selezione basata sulle azioni dell'utente (creare il FileExporter corrispondente al formato richiesto dall'utente), restituendo ai service oggetti pronti all'uso che rispettano l'interfaccia attesa.

4.3) Design Pattern strutturali

4.3.1) Adapter

Descrizione del pattern

L'Adapter è un pattern strutturale che traduce l'interfaccia di un componente esterno nell'interfaccia che il sistema si aspetta.

Permette a due componenti incompatibili di collaborare senza che nessuno dei due debba cambiare la propria interfaccia.

Motivazioni dell'utilizzo del pattern

Il dominio definisce le proprie interfacce in termini di concetti di business.

Tramite l'uso sistematico di porte e adapter è possibile ridurre l'accoppiamento permettendo l'aggiornamento parallelo dei sistemi e lasciando all'adapter il compito di tradurre le nuove interfacce

Utilizzo del pattern nel progetto

Al fine di implementare correttamente l'architettura esagonale nel sistema backend, il pattern adapter è stato usato in modo sistematico per gestire le comunicazioni verso servizi esterni.

Gli utilizzi principali sono nella comunicazione col database e con i servizi di interpretazione e generazione di file

4.4) Design Pattern Comportamentali

4.4.1) Template Method

Descrizione del pattern

Il Template Method è un pattern comportamentale che definisce lo scheletro di un algoritmo in una classe astratta, delegando alle sottoclassi l'implementazione dei passi specifici.

La sequenza complessiva è fissa; ciò che cambia è il comportamento dei singoli passi.

Motivazioni dell'utilizzo del pattern

Tutti i formati di export condividono la stessa sequenza operativa: preparare la struttura, scrivere i dati, finalizzare l'output.

Senza Template Method questa sequenza dovrebbe essere replicata in ogni exporter concreto, introducendo duplicazione e rischio di inconsistenze.

Il Template Method consente di scrivere la sequenza una sola volta nella classe astratta, lasciando alle sottoclassi solo i dettagli specifici del formato.

Utilizzo del pattern nel progetto

Viene utilizzato durante l'esportazione di informazioni sotto formato di file in diversi formati.

4.5) Pattern MVVM nel Frontend

Il frontend adotta il pattern Model-View-ViewModel tramite la Composition API di Vue 3. Le responsabilità sono distribuite esplicitamente tra tre categorie differenti con una separazione netta tra logica e presentazione.

4.5.1) Model

Il Model rappresenta i dati dell'applicazione e le regole che li governano, indipendentemente da qualsiasi elemento visivo. Questa responsabilità è affidata a file di schema come `assetFormFields.js` che definiscono i campi di un'entità, i loro valori iniziali e le regole di validazione come funzioni pure. Il Model non dipende da Vue, HTTP o dal DOM.

4.5.2) ViewModel

Il ViewModel è il layer intermedio che espone lo stato reattivo alla View e incapsula la logica applicativa. Nel progetto questa responsabilità è realizzata in due modi, in base alla complessità del modulo. Per i flussi operativi standard (form di creazione e modifica) il ViewModel è implementato tramite `composable`. Il `composable useFormModel` riceve uno schema dal Model e gestisce lo stato reattivo dei campi, la validazione e l'integrazione degli errori provenienti dal server. I widget smart completano questo layer orchestrando la chiamata HTTP, la gestione della risposta e la navigazione. Per i moduli ad alto tasso di interazione (come il widget di valutazione dei requisiti) il ViewModel è centralizzato in uno store Pinia, che coordina lo stato condiviso tra più componenti, la comunicazione con il backend e la logica di dominio frontend.

4.5.3) View

La View è responsabile esclusivamente del rendering. I componenti dumb ricevono dati tramite props ed emettono eventi senza contenere logica applicativa. Il binding bidirezionale con il ViewModel avviene tramite `v-model`.

5) Diagrammi delle classi

5.1) Dominio

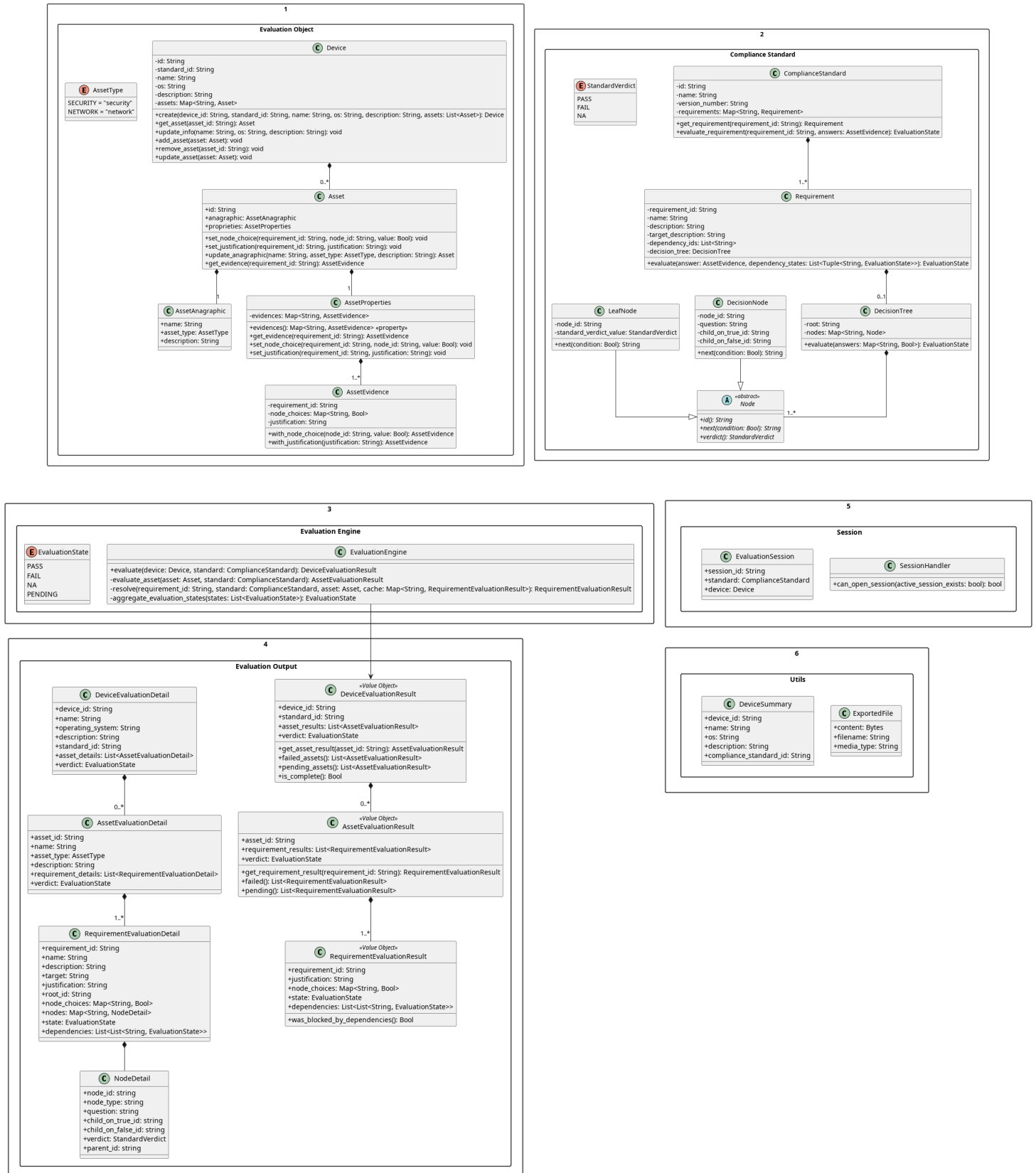


Figura 4: Modello di Dominio

5.1.1) Device

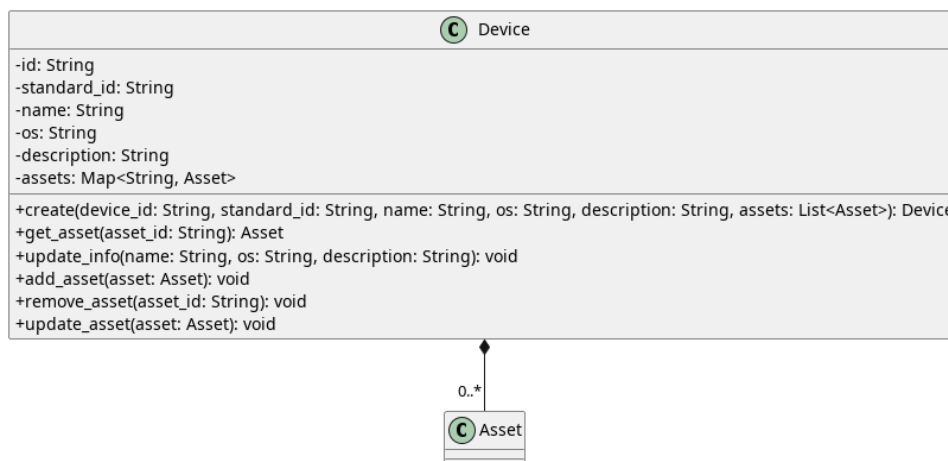


Figura 5: Device

Descrizione

Device è l'entità centrale del dominio che rappresenta il Dispositivo oggetto della valutazione di conformità. È associata alla classe *Asset* con una relazione di composizione avente cardinalità `0..*`.

Attributi

- - `id: String` — identificativo univoco del Dispositivo.
- - `standard_id: String` — identificativo dello standard di conformità associato al dispositivo.
- - `name: String` — nome del Dispositivo.
- - `os: String` — sistema operativo del Dispositivo.
- - `description: String` — descrizione testuale del Dispositivo.
- - `assets: Map<String, Asset>` — mappa degli asset associati al Dispositivo, indicizzati per il loro identificativo.

Metodi

- + `create(device_id: String, standard_id: String, name: String, os: String, description: String, assets: List<Asset>): Device` — metodo per la creazione e istanziazione di un nuovo oggetto Dispositivo.
- + `get_asset(asset_id: String): Asset` — recupera l'*Asset* corrispondente all'identificativo fornito.
- + `update_info(name: String, os: String, description: String): void` — aggiorna le informazioni anagrafiche del Dispositivo (nome, sistema operativo e descrizione).
- + `add_asset(asset: Asset): void` — aggiunge un nuovo *Asset* alla mappa del Dispositivo.
- + `remove_asset(asset_id: String): void` — rimuove l'*Asset* identificato da `asset_id` dalla mappa del Dispositivo.

- + update_asset(asset: Asset): void — aggiorna un *Asset* esistente

5.1.2) Asset

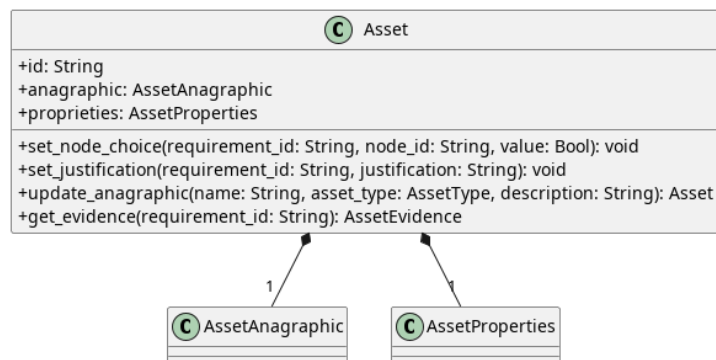


Figura 6: Asset

Descrizione

Asset è l'entità di dominio che rappresenta un asset oggetto di valutazione di conformità all'interno di una sessione.

Attributi

- - asset_id: String — identificativo univoco dell'*Asset*.
- - asset_anagraphic: *AssetAnagraphic* — oggetto che incapsula i dati anagrafici dell'asset.
- - asset_proprieties: *AssetProperties* — oggetto che incapsula le proprietà dell'asset.

Metodi

- + set_node_choice(requirement_id: String, node_id: String, value: Bool): void — imposta o aggiorna la scelta (risposta) effettuata per un determinato nodo decisionale relativo a un requisito.
- + set_justification(requirement_id: String, value: Bool): void — imposta la giustificazione per un determinato requisito.
- + update_anagraphic(name: String, type: AssetType, description: String): void — aggiorna le informazioni anagrafiche dell'asset (nome, tipologia e descrizione), delegando l'aggiornamento all'istanza interna di *AssetAnagraphic*.

5.1.3) AssetAnagraphic

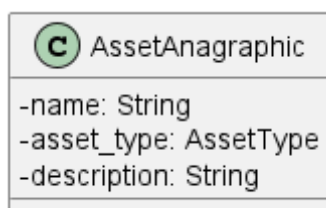


Figura 7: AssetAnagraphic

Descrizione

AssetAnagraphic è l'entità delegata all'incapsulamento delle informazioni anagrafiche e dei metadati di base di un generico asset.

Attributi

- - name: String — stringa di testo che rappresenta il nome identificativo dell'asset.
- - type: AssetType — attributo che definisce la tipologia o la categoria di appartenenza dell'asset.
- - description: String — stringa di testo destinata a contenere una descrizione estesa, note o dettagli aggiuntivi riguardanti le caratteristiche fisiche o logiche dell'asset.

Metodi

AssetAnagraphic definisce dei semplici getter, qui omessi per poca rilevanza.

5.1.4) AssetProperties

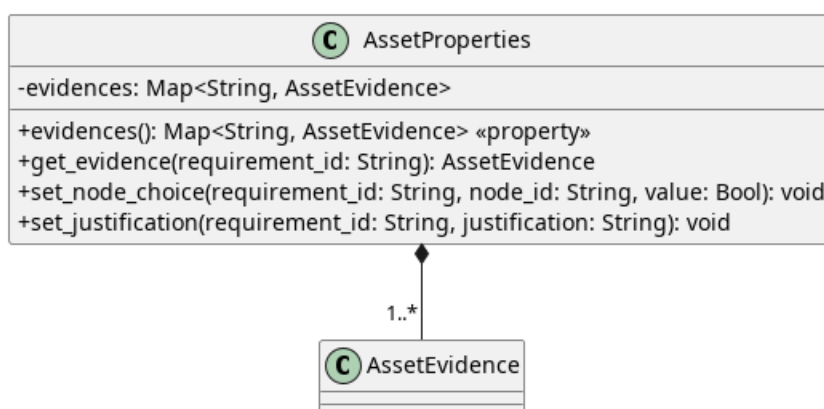


Figura 8: AssetProperties

Descrizione

AssetProperties è l'entità delegata alla gestione dello stato valutativo e delle proprietà specifiche di un asset. Presenta una relazione di composizione con la classe *AssetEvidence* con cardinalità `1..*`, gestendone il ciclo di vita all'interno di una lista.

Attributi

- + asset_evidence_list: List<AssetEvidence> — struttura dati che incapsula e gestisce l'elenco delle evidenze (scelte e giustificazioni) associate all'asset.

Metodi

- + set_node_choice(requirement_id: String, node_id: String, value: Bool): void — imposta o aggiorna la scelta (risposta) effettuata per un determinato nodo decisionale relativo a un requisito.

- + `set_justification(requirement_id: String, value: Bool): void` — imposta o aggiorna la giustificazione testuale per un determinato nodo di un requisito.
- + `get_evidence(requirement_id: String): AssetEvidence | void` — recupera l'oggetto *AssetEvidence* associato a un determinato identificativo di requisito. Restituisce l'evidenza se presente, altrimenti void (nessun valore/null).

5.1.5) AssetType

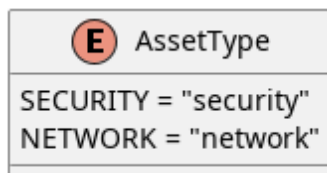


Figura 9: AssetType

Descrizione

AssetType è l'enumerazione che definisce le categorie o le tipologie logiche di appartenenza di un determinato Asset all'interno del dominio valutativo.

Attributi

- - `SECURITY: String` — valore enumerato che identifica un asset di tipo sicurezza («security»).
- - `NETWORK: String` — valore enumerato che identifica un asset di tipo rete («network»).

Metodi

AssetType non definisce metodi.

5.1.6) AssetEvidence

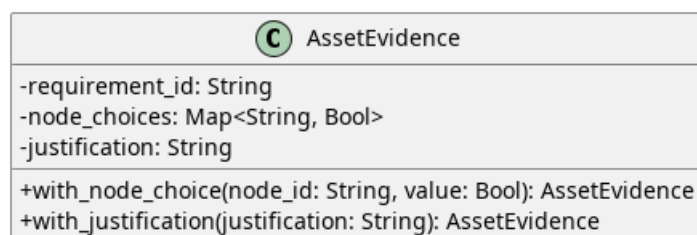


Figura 10: AssetEvidence

Descrizione

AssetEvidence è un Value Object immutabile che incapsula in modo sicuro le evidenze di valutazione associate a un singolo requisito, comprendendo le risposte fornite ai nodi decisionali e l'eventuale giustificazione testuale. Essendo immutabile, ogni modifica genera una nuova istanza.

Attributi

- - `requirement_id`: `String` — identificativo univoco del requisito a cui le evidenze fanno riferimento.
- - `node_choices`: `Map<String, Bool>` — mappa immutabile che associa l'identificativo di un nodo decisionale alla risposta booleana fornita.
- - `justification`: `String` — stringa di testo contenente la giustificazione opzionale per la valutazione del requisito.

Metodi

- + `with_node_choice(node_id: String, value: Bool): AssetEvidence` — crea e restituisce una nuova istanza dell'oggetto contenente la mappa delle risposte aggiornata con il nuovo valore per il nodo specificato.
- + `with_justification(justification: String): AssetEvidence` — crea e restituisce una nuova istanza dell'oggetto aggiornata con il testo della giustificazione fornito.

5.1.7) ComplianceStandard

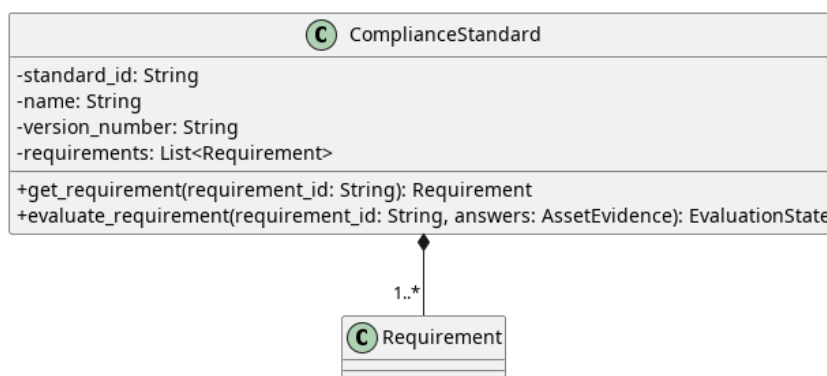


Figura 11: ComplianceStandard

Descrizione

ComplianceStandard è l'entità radice che rappresenta una norma o standard di conformità. È composta da una collezione di requisiti validati univocamente. La classe è progettata per essere immutabile garantendo l'integrità dei dati tramite incapsulamento.

Attributi

- - `standard_id`: `String` — identificativo univoco dello standard.
- - `name`: `String` — nome descrittivo dello standard.
- - `version_number`: `String` — versione specifica dello standard.
- - `requirements`: `List<Requirement>` — tupla immutabile contenente tutti i requisiti associati allo standard.

Metodi

- + `get_requirement(requirement_id: String): Requirement` — recupera uno specifico requisito tramite il suo identificativo, sollevando un'eccezione se non presente.
- + `evaluate_requirement(requirement_id: String, answers: AssetEvidence): EvaluationState` — delega al requisito specificato la valutazione del suo stato basandosi sulle evidenze fornite.

5.1.8) Requirement

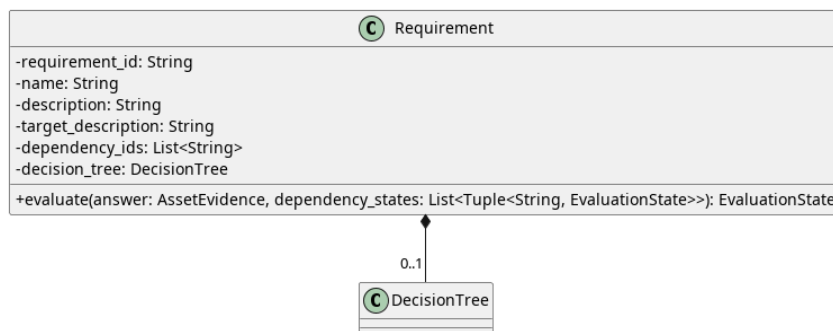


Figura 12: Requirement

Descrizione

Requirement è un'entità immutabile che definisce un singolo requisito di conformità, includendo le sue dipendenze e l'albero decisionale per valutarlo. Applica regole di business stringenti, come il fallimento automatico se il verdetto è «Non Applicabile» (NA) e manca la giustificazione testuale.

Attributi

- - `requirement_id: String` — identificativo univoco del requisito.
- - `name: String` — nome del requisito.
- - `description: String` — descrizione completa del requisito.
- - `target_description: String` — descrizione dell'obiettivo del requisito.
- - `decision_tree: DecisionTree` — albero decisionale contenente la logica per la valutazione.
- - `dependency_ids: List<String>` — tupla contenente gli ID di altri requisiti da cui questo dipende.

Metodi

- + `evaluate(answer: AssetEvidence, dependency_states: List): EvaluationState` — calcola lo stato di valutazione controllando prima lo stato delle dipendenze e poi interrogando l'albero decisionale.
- - `_check_dependencies(dependencies: List): EvaluationState | None` — metodo privato che verifica le dipendenze: se una fallisce, blocca la valutazione in FAIL o PENDING.

5.1.9) DecisionTree

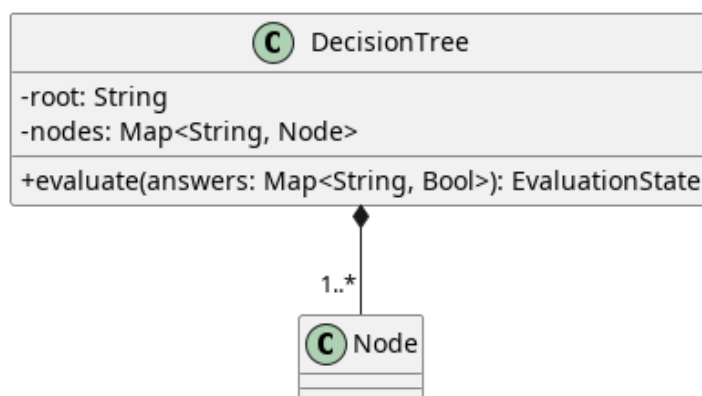


Figura 13: DecisionTree

Descrizione

DecisionTree è il componente che incapsula la logica strutturata di valutazione navigando un grafo di nodi. In fase di inizializzazione convalida l'assenza di nodi duplicati e l'esistenza della radice. Durante la valutazione, traccia i nodi visitati per prevenire cicli infiniti.

Attributi

- - `root_id: String` — identificativo del nodo iniziale (radice) da cui parte la valutazione.
- - `nodes: Map<String, Node>` — mappa immutabile che associa gli ID dei nodi alle rispettive istanze.

Metodi

- + `evaluate(answers: Map<String, Bool>): EvaluationState` — naviga l'albero partendo dalla radice e utilizzando le risposte fornite, restituendo lo stato finale della valutazione.

5.1.10) Node

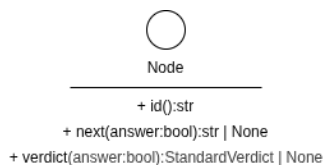


Figura 14: Node

Descrizione

Node è un'interfaccia che definisce il contratto base per tutti gli elementi che compongono un albero decisionale.

Metodi

- - `id(): String` — proprietà astratta che restituisce l'identificativo del nodo.

- - `verdict(): StandardVerdict | None` — proprietà astratta che restituisce il verdetto del nodo, se applicabile.
- + `next(condition: Bool | None): String | None` — metodo astratto che determina l'ID del nodo successivo basandosi sulla condizione fornita.

5.1.11) DecisionNode

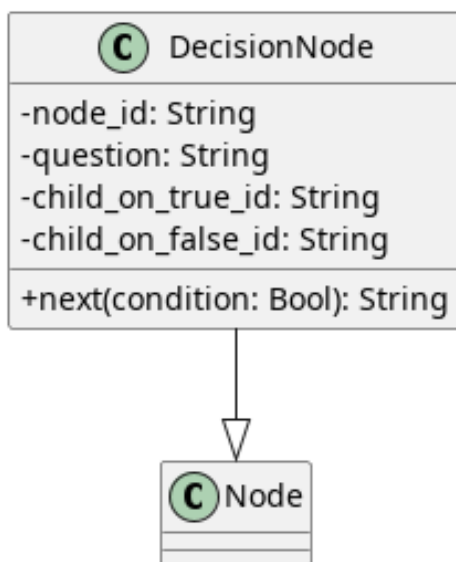


Figura 15: DecisionNode

Descrizione

DecisionNode è un oggetto immutabile che implementa *Node*. Rappresenta uno snodo dell'albero che pone una domanda e biforca il percorso di valutazione.

Attributi

- - `node_id: String` — identificativo del nodo.
- - `question: String` — testo della domanda posta all'utente.
- - `child_on_true_id: String` — ID del nodo successivo se la risposta è affermativa.
- - `child_on_false_id: String` — ID del nodo successivo se la risposta è negativa.

Metodi

- + `id(): str` — restituisce l'id del nodo.
- + `next(condition: Bool | None): String | None` — restituisce l'ID del nodo figlio appropriato a seconda del valore booleano della risposta.
- + `verdict(): None` — restituisce costantemente `None` poiché uno snodo non produce un verdetto finale.

5.1.12) LeafNode

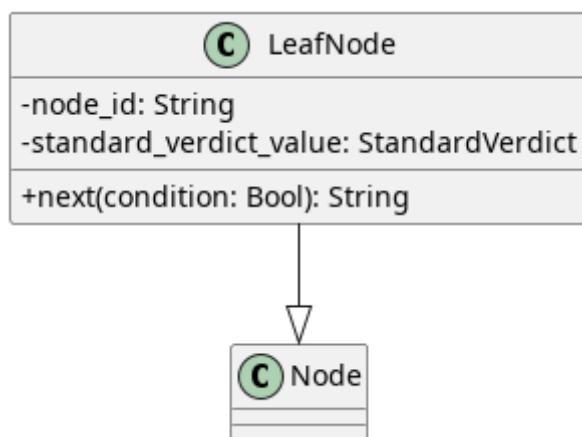


Figura 16: LeafNode

Descrizione

LeafNode è un oggetto immutabile che implementa *Node* e rappresenta la terminazione di un percorso decisionale (foglia), contenendo il risultato finale della valutazione.

Attributi

- - `node_id: String` — identificativo del nodo foglia.
- - `verdict_value: StandardVerdict` — il verdetto definitivo di conformità associato alla foglia.

Metodi

- + `id(): str` — restituisce l'id del nodo.
- + `next(condition: Bool | None): None` — restituisce costantemente `None` poiché una foglia non possiede nodi successivi.
- + `verdict(): StandardVerdict` — restituisce il valore del verdetto di conformità.

5.1.13) StandardVerdict

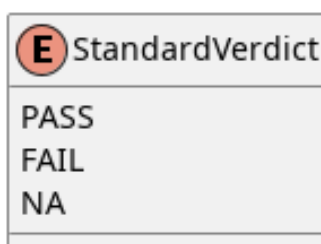


Figura 17: StandardVerdict

Descrizione

StandardVerdict è un'enumerazione di stringhe che definisce formalmente i possibili verdeti terminali generati da un albero decisionale.

Attributi

- - PASS: String — indica che il requisito è soddisfatto («pass»).
- - FAIL: String — indica che il requisito non è soddisfatto («fail»).
- - NA: String — indica che il requisito non è applicabile al contesto («not_applicable»).

Metodi

StandardVerdict non definisce metodi.

5.1.14) EvaluationEngine

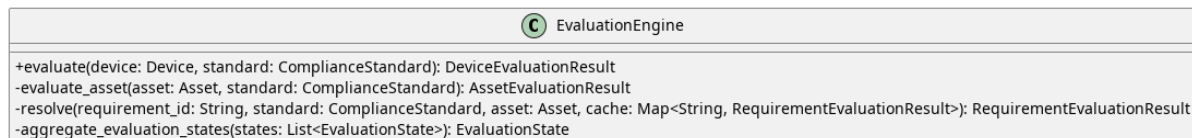


Figura 18: EvaluationEngine

Descrizione

EvaluationEngine è il componente del dominio responsabile di orchestrare il processo di valutazione di un intero Device rispetto a un ComplianceStandard fornito. Il motore elabora iterativamente gli asset, risolvendo in modo ricorsivo le dipendenze tra i requisiti e applicando tecniche di memoizzazione per ottimizzare le performance e prevenire valutazioni ridondanti.

Attributi

La classe non definisce attributi di stato interni, agendo come puro gestore della logica di business.

Metodi

- + evaluate(device: Device, standard: ComplianceStandard): DeviceEvaluationResult — valuta il dispositivo calcolando e aggregando i risultati di tutti i suoi asset, restituendo infine l'esito globale.
- - _evaluate_asset(asset: Asset, standard: ComplianceStandard): AssetEvaluationResult — metodo privato che valuta un singolo asset contro tutti i requisiti dello standard, avvalendosi di una cache per mantenere i risultati e supportare la memoizzazione.
- - _resolve(requirement_id: String, standard: ComplianceStandard, asset: Asset, cache: Map): RequirementEvaluationResult — risolve la valutazione di uno specifico requisito verificando prima ricorsivamente le sue dipendenze. Se l'asset non presenta evidenze per il requisito, imposta forzatamente lo stato su PENDING. Successivamente, salva il risultato nella cache.
- - _aggregate_evaluation_states(states: List<EvaluationState>): EvaluationState — analizza una serie di stati e ne calcola il verdetto aggregato:

restituisce FAIL se rileva almeno un fallimento, PENDING se vi sono valutazioni incomplete, altrimenti restituisce PASS.

5.1.15) EvaluationState

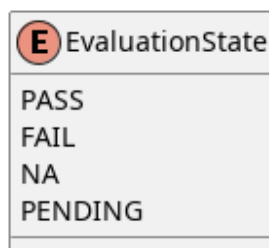


Figura 19: EvaluationState

Descrizione

EvaluationState è un'enumerazione di stringhe (StrEnum) che definisce formalmente i possibili stati di avanzamento o di conclusione di una valutazione.

Attributi

- - PASS: String — indica che la valutazione è stata superata con successo («pass»).
- - FAIL: String — indica che la valutazione ha dato esito negativo («fail»).
- - NA: String — indica che il requisito non è applicabile al contesto valutato («not_applicable»).
- - PENDING: String — indica che la valutazione è attualmente in sospeso per mancanza di evidenze o risposte complete («pending»).

Metodi

- + from_verdict(verdict: StandardVerdict): EvaluationState — metodo di classe che converte un StandardVerdict nel corrispondente EvaluationState, sollevando un ValueError qualora non esista una mappatura definita.

5.1.16) RequirementEvaluationResult

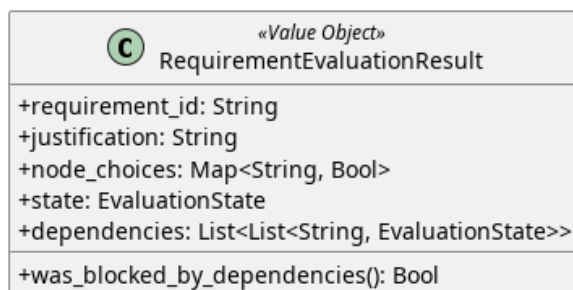


Figura 20: RequirementEvaluationResult

Descrizione

RequirementEvaluationResult è un Value Object immutabile che incapsula l'esito finale della valutazione di un singolo requisito.

Attributi

- + requirement_id: String — identificativo del requisito valutato.
- + justification: String — giustificazione fornita per la valutazione.
- + node_choices: MappingProxyType<String, Bool> — mappa immutabile delle risposte fornite ai nodi decisionali.
- + state: EvaluationState — stato finale calcolato per il requisito.
- + dependencies: List — tupla contenente l'ID e lo stato delle dipendenze del requisito.

Metodi

- + was_blocked_by_dependencies(): Bool — verifica se l'esito è stato bloccato a causa di una o più dipendenze che non hanno raggiunto lo stato di PASS.

5.1.17) AssetEvaluationResult

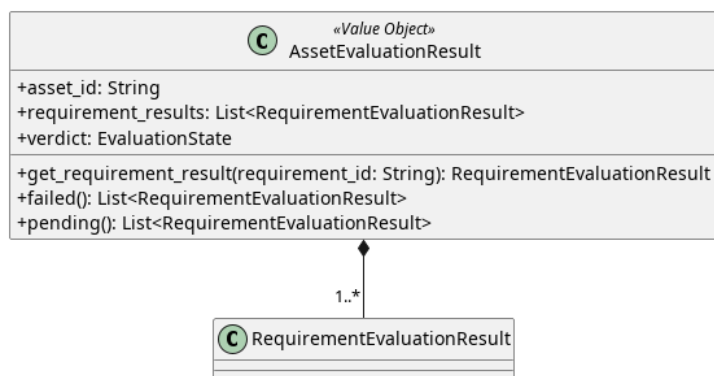


Figura 21: AssetEvaluationResult

Descrizione

AssetEvaluationResult è un Value Object immutabile che raggruppa tutti i risultati dei requisiti calcolati per un singolo asset, determinandone il verdetto complessivo.

Attributi

- + asset_id: String — identificativo dell'asset valutato.
- + requirement_results: List<RequirementEvaluationResult> — tupla contenente i risultati di tutti i requisiti valutati per questo asset.
- + verdict: EvaluationState — stato di conformità globale dell'asset.

Metodi

- + get_requirement_result(requirement_id: String): RequirementEvaluationResult | None — cerca e restituisce il risultato di uno specifico requisito, se presente.

- + `failed(): List<RequirementEvaluationResult>` — filtra e restituisce esclusivamente i requisiti che hanno prodotto uno stato di FAIL.
- + `pending(): List<RequirementEvaluationResult>` — filtra e restituisce esclusivamente i requisiti con valutazione ancora in corso o incompleta (PENDING).

5.1.18) DeviceEvaluationResult

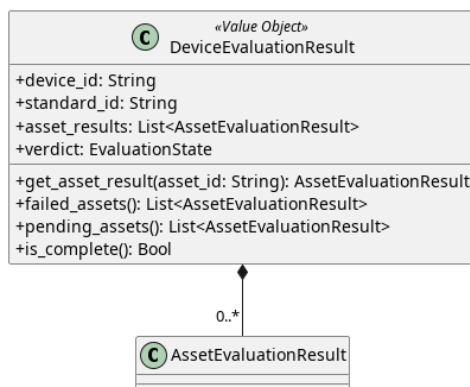


Figura 22: DeviceEvaluationResult

Descrizione

DeviceEvaluationResult rappresenta l'esito globale e immutabile della valutazione di un intero dispositivo rispetto a uno standard. Raggruppa i risultati di tutti i suoi asset.

Attributi

- + `device_id: String` — identificativo del dispositivo.
- + `standard_id: String` — identificativo dello standard di riferimento.
- + `asset_results: List<AssetEvaluationResult>` — tupla con i risultati aggregati per ogni asset.
- + `verdict: EvaluationState` — esito finale della valutazione complessiva del dispositivo.

Metodi

- + `get_asset_result(asset_id: String): AssetEvaluationResult | None` — recupera il risultato di un asset specifico all'interno del dispositivo.
- + `failed_assets(): List<AssetEvaluationResult>` — restituisce gli asset che non hanno superato la valutazione.
- + `pending_assets(): List<AssetEvaluationResult>` — restituisce gli asset la cui valutazione è ancora in sospeso.
- + `is_complete(): Bool` — restituisce True se l'intera valutazione del dispositivo è conclusa (non ci sono stati PENDING).

5.1.19) NodeDetail

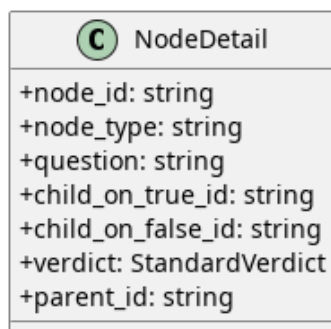


Figura 23: NodeDetail

Descrizione

NodeDetail è un oggetto di dominio, immutabile, utilizzato per esporre i dettagli strutturali e il contenuto informativo di un nodo dell'albero decisionale.

Attributi

- + `node_id`: `String` — identificativo del nodo.
- + `node_type`: `String` — indica la natura del nodo («decision» o «leaf»).
- + `question`: `String` | `None` — testo della domanda, presente solo nei nodi decisionali.
- + `child_on_true_id`: `String` | `None` — riferimento al nodo figlio per risposta affermativa.
- + `child_on_false_id`: `String` | `None` — riferimento al nodo figlio per risposta negativa.
- + `verdict`: `StandardVerdict` | `None` — esito associato al nodo, valorizzato solo per i nodi foglia.
- + `parent_id`: `String` | `None` — identificativo del nodo genitore, utile per navigare l'albero a ritroso.

Metodi

NodeDetail non definisce metodi.

5.1.20) RequirementEvaluationDetail

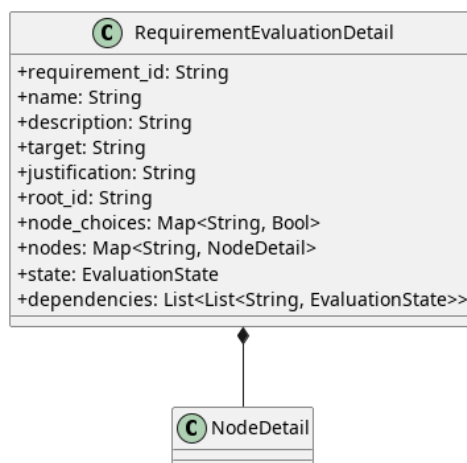


Figura 24: RequirementEvaluationDetail

Descrizione

RequirementEvaluationDetail è un oggetto immutabile che consolida tutte le informazioni di un requisito (anagrafica, albero, nodi) e il relativo stato di valutazione nel contesto di un asset. È concepito per arricchire il risultato grezzo con i testi completi.

Attributi

- + `requirement_id: String` — identificativo univoco.
- + `name: String` — nome del requisito.
- + `description: String` — testo descrittivo del requisito.
- + `target: String` — obiettivo prefissato.
- + `justification: String` — motivazione associata alla risposta.
- + `root_id: String` — identificativo del nodo radice dell'albero.
- + `node_choices: MappingProxyType<String, Bool>` — mappa delle risposte effettuate.
- + `nodes: Map<String, NodeDetail>` — dizionario di tutti i nodi appartenenti all'albero di questo requisito.
- + `state: EvaluationState` — esito attuale.
- + `dependencies: List` — lista delle dipendenze correlate con il relativo stato.

Metodi

RequirementEvaluationDetail non definisce metodi.

5.1.21) AssetEvaluationDetail

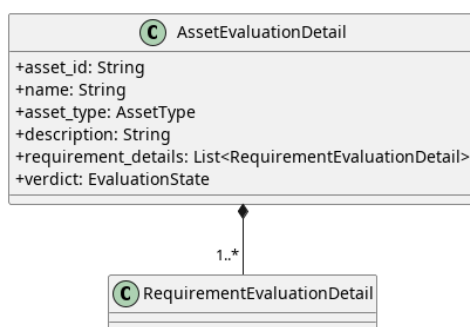


Figura 25: AssetEvaluationDetail

Descrizione

AssetEvaluationDetail è un oggetto immutabile che aggrega tutti i dettagli descrittivi e di valutazione dei requisiti di uno specifico asset.

Attributi

- + `asset_id: String` — identificativo dell'asset.
- + `name: String` — nome dell'asset.
- + `asset_type: AssetType` — categoria di appartenenza.
- + `description: String` — descrizione aggiuntiva.
- + `requirement_details: List<RequirementEvaluationDetail>` — insieme arricchito di dettagli per ogni requisito.
- + `verdict: EvaluationState` — stato globale dell'asset.

Metodi

AssetEvaluationDetail non definisce metodi.

5.1.22) DeviceEvaluationDetail

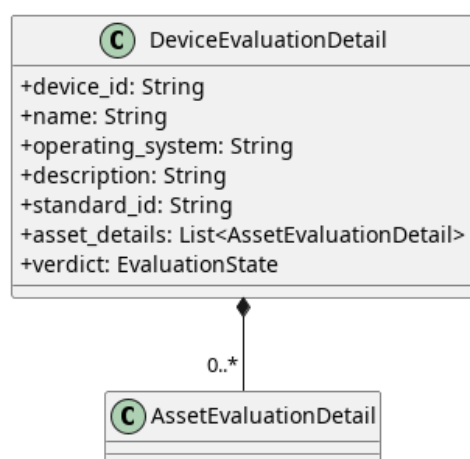


Figura 26: DeviceEvaluationDetail

Descrizione

DeviceEvaluationDetail è il livello radice della struttura di dettaglio: consolida e struttura in modo gerarchico tutte le informazioni testuali e di valutazione per l'intero dispositivo.

Attributi

- + device_id: String — identificativo del dispositivo.
- + name: String — nome del dispositivo.
- + operating_system: String — sistema operativo in uso.
- + description: String — breve descrizione del dispositivo.
- + standard_id: String — identificativo dello standard applicato.
- + asset_details: List<AssetEvaluationDetail> — dettaglio di ogni asset.
- + verdict: EvaluationState — stato complessivo.

Metodi

DeviceEvaluationDetail non definisce metodi.

5.1.23) EvaluationSession

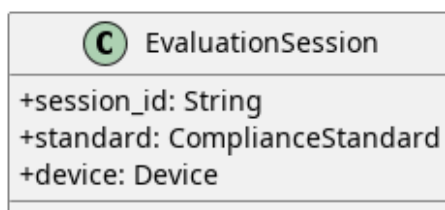


Figura 27: EvaluationSession

Descrizione

EvaluationSession è la classe che modella il contesto centrale di un'operazione di valutazione. Raggruppa e associa un identificativo univoco di sessione a uno specifico dispositivo e allo standard di conformità di riferimento scelto per l'analisi.

Attributi

- + session_id: String — identificativo univoco della sessione di valutazione.
- + standard: ComplianceStandard — istanza dello standard di conformità applicato per la valutazione corrente.
- + device: Device — istanza del dispositivo che viene sottoposto a valutazione.

Metodi

EvaluationSession non definisce metodi.

5.1.24) SessionHandler

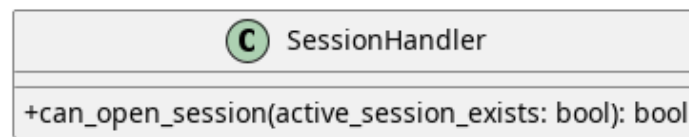


Figura 28: SessionHandler

Descrizione

SessionHandler è il componente di dominio che incapsula le regole di business fondamentali relative all'apertura di una nuova sessione. Valuta la fattibilità dell'operazione controllando le precondizioni del sistema.

Tale contesto viene passato tramite parametro di funzione. Sebbene al momento la logica sia molto semplice, questa struttura risulterà utile per estensioni future.

Attributi

La classe non definisce attributi di stato interni, agendo come puro gestore di logica.

Metodi

- `+ can_open_session(active_session_exists: Bool): Bool` — verifica se è possibile avviare una nuova sessione, restituendo `False` nel caso in cui ne esista già una attiva (impedendo sovrapposizioni), altrimenti restituisce `True`.

5.1.25) ExportedFile

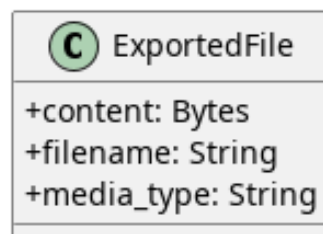


Figura 29: ExportedFile

Descrizione

ExportedFile è un oggetto immutabile utilizzato per incapsulare in un'unica struttura il contenuto binario e i metadati di un file pronto per l'esportazione o il download.

Attributi

- `+ content: IO<bytes>` — stream binario contenente i dati grezzi del file da esportare.
- `+ filename: String` — nome del file, comprensivo della relativa estensione.
- `+ media_type: String` — tipo di media o formato MIME (ad esempio, «application/pdf» o «application/json») associato al file.

Metodi

ExportedFile non definisce metodi.

5.1.26) DeviceSummary

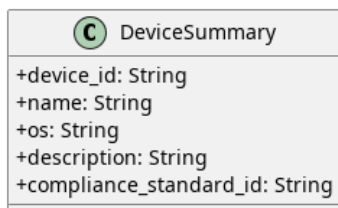


Figura 30: DeviceSummary

Descrizione

DeviceSummary è un oggetto immutabile utilizzato per incapsulare e trasportare una vista sintetica e leggera delle informazioni anagrafiche di un dispositivo. È progettato per fornire i dati essenziali senza esporre l'intera struttura complessa o gli asset associati, risultando ideale per le operazioni di visualizzazione o per la generazione di elenchi.

Attributi

- + `device_id: String` — identificativo univoco del dispositivo.
- + `name: String` — nome assegnato al dispositivo.
- + `os: String` — sistema operativo in uso sul dispositivo.
- + `description: String` — breve descrizione testuale del dispositivo.
- + `compliance_standard_id: String` — identificativo dello standard di conformità a cui il dispositivo fa riferimento.

Metodi

DeviceSummary non definisce metodi.

5.2) Device

5.2.1) CreateDevice



Figura 31: Caso d'uso CreateDevice

Il diagramma illustra l'architettura del modulo di creazione dei Dispositivi secondo i principi dell'architettura esagonale.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.

5.2.1.1) FlaskWriteDeviceController



Figura 32: FlaskWriteDeviceController

Descrizione

FlaskWriteDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione dei Dispositivi e le inoltra al livello applicativo.

Attributi

- - `create_device_use_case`: `CreateDeviceUseCase` — inbound port usata per la creazione di un device.
- - `update_device_use_case`: `UpdateDeviceUseCase` — inbound port usata per la modifica di un device.
- - `delete_device_use_case`: `DeleteDeviceUseCase` — inbound port usata per l'eliminazione di un device.

Metodi

- + `create_device(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Dispositivo, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `update_device(req: Request): Response` — riceve la richiesta HTTP di modifica di un Dispositivo esistente, estrae i dati aggiornati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_device(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Dispositivo, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.2.1.2) CreateDeviceUseCase

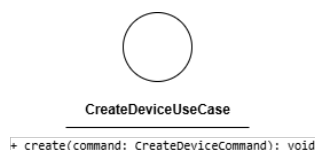


Figura 33: CreateDeviceUseCase

Descrizione

CreateDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Dispositivo. Viene implementata da *CreateDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

CreateDeviceUseCase non definisce attributi.

Metodi

- + `create(command: CreateDeviceCommand): void` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.2.1.3) CreateDeviceCommand

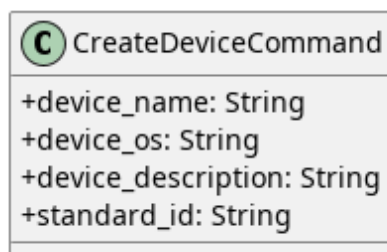


Figura 34: CreateDeviceCommand

Descrizione

CreateDeviceCommand è il Command Object che veicola i dati necessari alla creazione di un Dispositivo dal controller al service. L'uso di questo pattern separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + device_name: String — nome del nuovo Dispositivo.
- + device_os: String — sistema operativo del Dispositivo.
- + device_description: String — descrizione testuale del Dispositivo.
- + standard_id: String — identificativo dello standard di conformità associato.

Metodi

CreateDeviceCommand non definisce metodi.

5.2.1.4) CreateDeviceService



Figura 35: CreateDeviceService

Descrizione

CreateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Dispositivo. Implementa l'interfaccia *CreateDeviceUseCase*, riceve il comando in ingresso, costruisce l'entità di dominio e ne coordina la persistenza tramite *RegisterDevicePort*.

Attributi

- - register_device_port: RegisterDevicePort — porta outbound utilizzata per la persistenza del nuovo Dispositivo.

Metodi

- + create(command: CreateDeviceCommand): void — concretizza il contratto definito da *CreateDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede la registrazione tramite *RegisterDevicePort*.

5.2.1.5) RegisterDevicePort

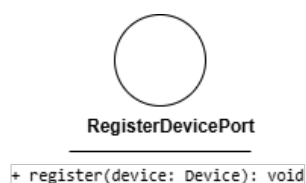


Figura 36: RegisterDevicePort

Descrizione

RegisterDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la registrazione di un nuovo Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *CreateDeviceService*.

Attributi

RegisterDevicePort non definisce attributi.

Metodi

- + `register(device: Device): void` — firma del metodo che esegue l’inserimento fisico del Dispositivo nel sistema di persistenza.

5.2.1.6) MongoDeviceAdapter



Figura 37: MongoDeviceAdapter

Descrizione

MongoDeviceAdapter è la classe dell’Outbound Adapter che implementa le porte outbound del sistema, traducendo le operazioni del dominio in interazioni concrete con MongoDB tramite `pymongo.Collection`.

Attributi

- - `collection: pymongo.Collection` — riferimento alla collezione MongoDB su cui vengono eseguite le operazioni di persistenza.

Metodi

- + `save(device: Device): void` — salva le modifiche a un Dispositivo esistente nella collezione.
- + `register(device: Device): void` — inserisce un nuovo Dispositivo nella collezione.
- + `delete(device_id: String): void` — rimuove il Dispositivo identificato da `device_id` dalla collezione.
- + `find_by_id(device_id: String): Device` — recupera il Dispositivo corrispondente all’identificativo fornito.
- + `find_all(): List<DeviceSummary>` — recupera la lista sintetica di tutti i Dispositivi presenti nella collezione.

5.2.2) DeleteDevice



Figura 38: Caso d'uso DeleteDevice

Il diagramma illustra l'architettura del modulo dedicato all'eliminazione di un Dispositivo.

- Per la definizione di *FlaskWriteDeviceController*, vedere la sezione **Sezione 5.2.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.2.1) DeleteDeviceUseCase

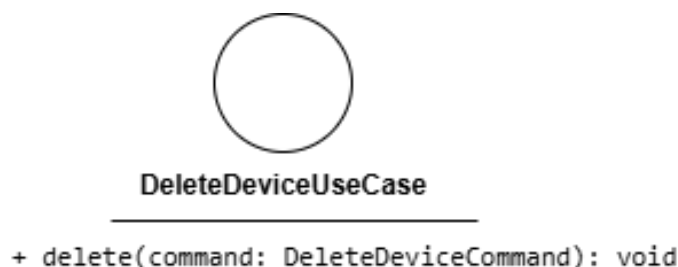


Figura 39: DeleteDeviceUseCase

Descrizione

DeleteDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Dispositivo. Viene implementata da *DeleteDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

DeleteDeviceUseCase non definisce attributi.

Metodi

- + delete(device_id: String): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dall'identificativo del Dispositivo.

5.2.2.2) DeleteDeviceCommand

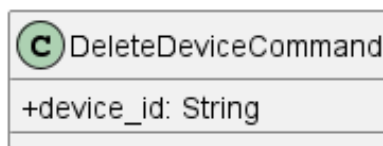


Figura 40: DeleteDeviceCommand

Descrizione

DeleteDeviceCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un Dispositivo.

Attributi

- + device_id: String — identificativo univoco del Dispositivo da eliminare.

Metodi

DeleteDeviceCommand non definisce metodi propri.

5.2.2.3) DeleteDeviceService

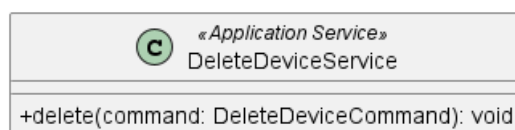


Figura 41: DeleteDeviceService

Descrizione

DeleteDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Dispositivo. Implementa l'interfaccia *DeleteDeviceUseCase* e riceve un *DeleteDeviceCommand* contenente l'identificativo del Dispositivo, coordinandone la rimozione dal sistema.

Attributi

- - delete_device_port: DeleteDevicePort — porta outbound utilizzata per eliminare il device

Metodi

- + delete(command: DeleteDeviceCommand): void — riceve il Command Object contenente l'identificativo univoco del Dispositivo e ne coordina la rimozione tramite *DeleteDevicePort*.

5.2.2.4) DeleteDevicePort

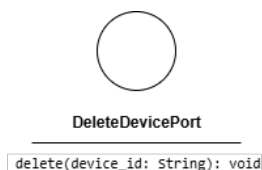


Figura 42: DeleteDevicePort

Descrizione

DeleteDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la rimozione fisica di un Dispositivo dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *DeleteDeviceService*.

Attributi

DeleteDevicePort non definisce attributi.

Metodi

- + delete(device_id: String): void — firma del metodo che esegue la rimozione fisica del Dispositivo identificato da device_id dal sistema di persistenza.

5.2.3) UpdateDevice

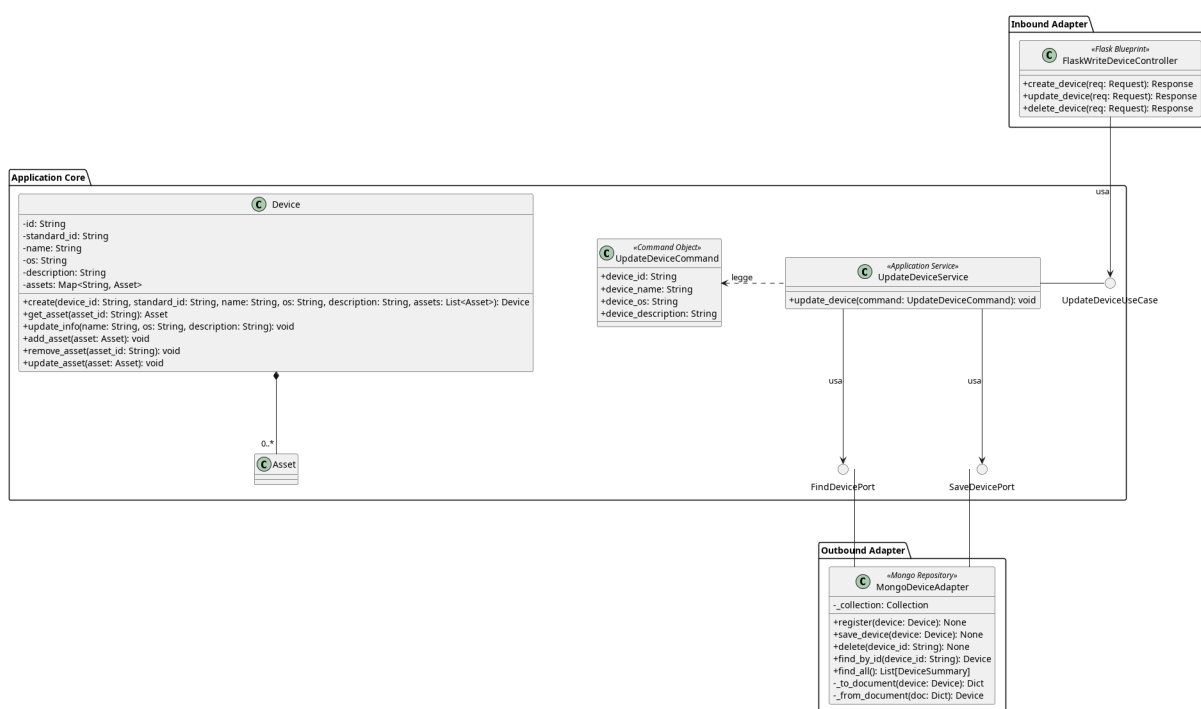


Figura 43: Caso d'uso UpdateDevice

Il diagramma illustra l'architettura del modulo dedicato alla modifica e al salvataggio dello stato di un Dispositivo esistente.

- Per la definizione di *FlaskWriteDeviceController*, vedere la sezione **Sezione 5.2.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.3.1) UpdateDeviceUseCase

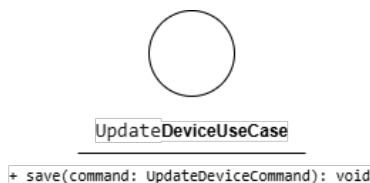


Figura 44: UpdateDeviceUseCase

Descrizione

UpdateDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Dispositivo esistente. Viene implementata da *UpdateDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

UpdateDeviceUseCase non definisce attributi.

Metodi

- `update_device(command: UpdateDeviceCommand)` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.2.3.2) UpdateDeviceCommand



Figura 45: UpdateDeviceCommand

Descrizione

UpdateDeviceCommand è il Command Object che veicola i dati necessari alla modifica di un Dispositivo dal controller al service. Analogamente a *CreateDeviceCommand*, separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- `+ device_id: String` — identificativo univoco del Dispositivo da aggiornare.
- `+ device_name: String` — nome aggiornato del Dispositivo.

- + device_os: String — sistema operativo aggiornato.
- + device_description: String — descrizione testuale aggiornata.

Metodi

UpdateDeviceCommand non definisce metodi.

5.2.3.3) UpdateDeviceService

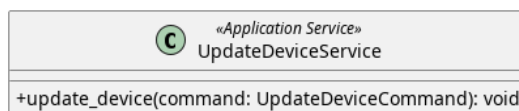


Figura 46: UpdateDeviceService

Descrizione

UpdateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Dispositivo esistente. Implementa l'interfaccia *UpdateDeviceUseCase*, riceve il comando in ingresso e ne coordina la persistenza tramite *SaveDevicePort*.

Attributi

- - find_device_port: FindDevicePort — utilizza la porta di outbound per prelevare il device
- - save_device_port: SaveDevicePort — utilizza la porta di outbound per salvare le modifiche

Metodi

- + update_device(command: UpdateDeviceCommand): void — concretizza il contratto definito da *SaveDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede l'aggiornamento tramite *SaveDevicePort*.

5.2.3.4) SaveDevicePort

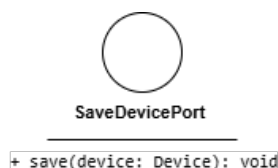


Figura 47: SaveDevicePort

Descrizione

SaveDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per l'aggiornamento fisico di un Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *UpdateDeviceService*.

Attributi

UpdateDevicePort non definisce attributi.

Metodi

- `+ save(device: Device): void` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo nel sistema di persistenza.

5.2.3.5) FindDevicePort

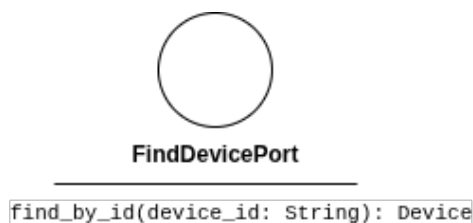


Figura 48: FindDevicePort

Descrizione

FindDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per prelevare un Device tramite id. Viene implementata da *MongoDeviceAdapter* e utilizzata da *UpdateDeviceService*.

Attributi

FindDevicePort non definisce attributi.

Metodi

- `+ find_by_id(device_id: String): Device` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo nel sistema di persistenza.

5.2.4) GetDeviceDetail

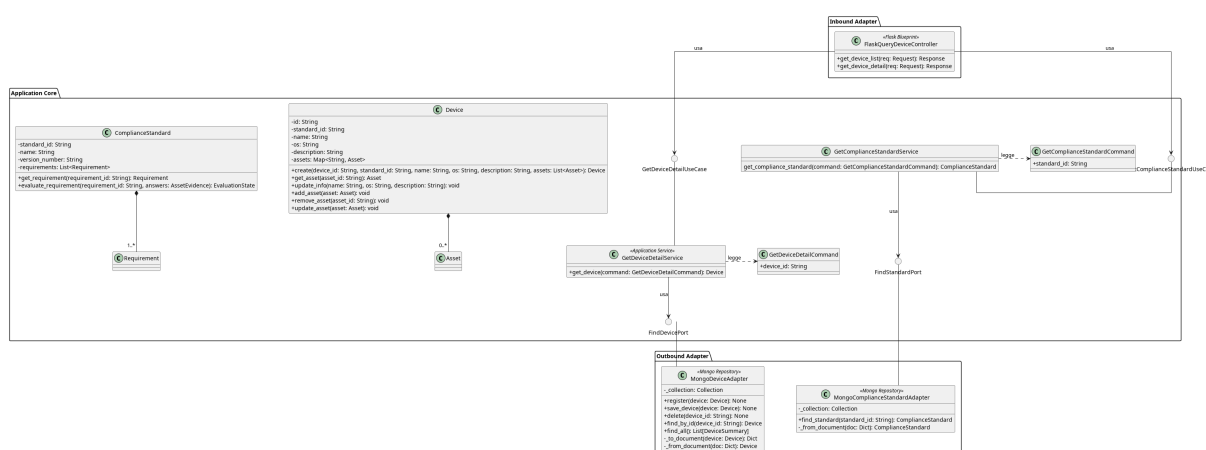


Figura 49: GetDeviceDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Dispositivo.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.3.5**.
- Per la definizione di *ComplianceStandard*, vedere la sezione **Sezione 5.1.7**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.4.1) FlaskQueryDeviceController



Figura 50: FlaskQueryDeviceController

Descrizione

FlaskQueryDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura relative ai Dispositivi e le inoltra al livello applicativo.

Attributi

- - `get_device_list_use_case`: *GetDeviceListUseCase* — inbound port usata per prendere la lista dei dispositivi
- - `get_device_detail_use_case`: *GetDeviceDetailUseCase* — inbound port usata per prendere il dettaglio di un dispositivo
- - `get_compliance_standard_use_case`: *GetComplianceStandardUseCase* — inbound port usata per prendere lo Standard

Metodi

- + `get_device_list(req: Request): Response` — riceve la richiesta HTTP di recupero della lista dei Dispositivi e restituisce una risposta HTTP con l'elenco sintetico.
- + `get_device_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Dispositivo specifico e restituisce una risposta HTTP con i dati completi, per recuperare questi dati utilizza le porte *GetDeviceDetailUseCase* per le informazioni del dispositivo e *GetComplianceStandardUseCase* per recuperare lo standard associato.

5.2.4.2) GetDeviceDetailUseCase

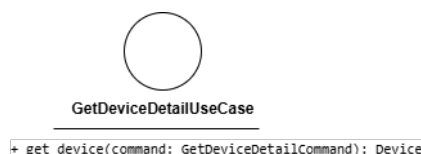


Figura 51: GetDeviceDetailUseCase

Descrizione

GetDeviceDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Dispositivo. Viene implementata da *GetDeviceDetailService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetDeviceDetailUseCase non definisce attributi.

Metodi

- + `get_device(command: GetDeviceDetailCommand): Device` — firma del metodo delegato al recupero del Dispositivo corrispondente al Command fornito.

5.2.4.3) GetDeviceDetailService

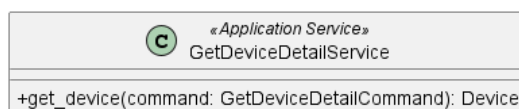


Figura 52: GetDeviceDetailService

Descrizione

GetDeviceDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Dispositivo. Implementa l'interfaccia *GetDeviceDetailUseCase* e coordina il recupero dei dati tramite la porta outbound *FindDevicePort*.

Attributi

- `find_device_port: FindDevicePort` — outbound port usata per prelevare un dispositivo.

Metodi

- + `get_device(command: GetDeviceDetailCommand): Device` — concretizza il contratto definito da *GetDeviceDetailUseCase*. Recupera il Dispositivo corrispondente all'identificativo contenuto nel Command tramite *FindDevicePort*.

5.2.4.4) GetDeviceDetailCommand

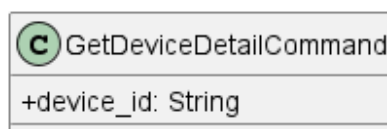


Figura 53: GetDeviceDetailCommand

Descrizione

GetDeviceDetailCommand è utilizzato per trasportare i dati necessari al recupero del dettaglio di un Dispositivo. Incapsula i parametri di input del metodo esposto da *GetDeviceDetailUseCase*.

Attributi

- + `device_id`: String — identificativo univoco del Dispositivo di cui recuperare il dettaglio.

Metodi

GetDeviceDetailCommand non definisce metodi propri.

5.2.4.5) GetComplianceStandardUseCase

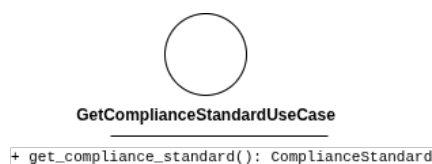


Figura 54: GetComplianceStandardUseCase

Descrizione

GetComplianceStandardUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero dello standard. Viene implementata da *GetComplianceStandardService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetComplianceStandardUseCase non definisce attributi.

Metodi

- + `get_compliance_standard(command: GetComplianceStandardCommand): ComplianceStandard` — firma del metodo delegato al recupero dello Standard corrispondente al Command fornito.

5.2.4.6) GetComplianceStandardService

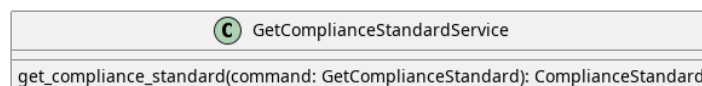


Figura 55: GetComplianceStandardService

Descrizione

GetComplianceStandardService è il service applicativo appartenente all'Application Core responsabile del recupero di un Compliance Standard. Implementa l'interfaccia *GetComplianceStandardUseCase* e coordina il recupero dei dati tramite la porta out-bound *FindStandardPort*.

Attributi

- `find_standard_port`: `FindStandardPort` — outbound port usata per prelevare lo Standard.

Metodi

- + `get_compliance_standard(command: GetComplianceStandardCommand): ComplianceStandard` — concretizza il contratto definito da *GetComplianceStandardUseCase*. Recupera il Compliance Standard corrispondente all'identificativo contenuto nel Command tramite *FindStandardPort*.

5.2.4.7) GetComplianceStandardCommand

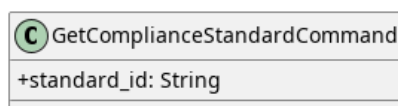


Figura 56: GetComplianceStandardCommand

Descrizione

GetComplianceStandardCommand è utilizzato per trasportare i dati necessari al recupero del dettaglio di un Compliance Standard. Incapsula i parametri di input del metodo esposto da *GetComplianceStandardUseCase*.

Attributi

- + `standard_id`: `String` — identificativo univoco dello Standard di cui recuperare il dettaglio.

Metodi

GetComplianceStandardCommand non definisce metodi propri.

5.2.4.8) FindStandardPort

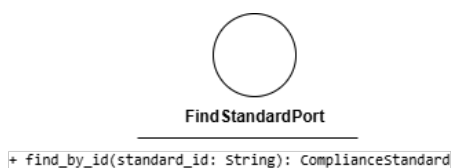


Figura 57: FindStandardPort

Descrizione

FindStandardPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero di uno standard di conformità dal sistema di persistenza. Viene implementata da *MongoStandardAdapter* e utilizzata da *OpenEvaluationSessionService*.

Attributi

FindStandardPort non definisce attributi.

Metodi

- + find_by_id(standard_id: String): ComplianceStandard — firma del metodo che recupera lo standard di conformità corrispondente all'identificativo fornito.

5.2.4.9) MongoStandardAdapter



Figura 58: MongoStandardAdapter

Descrizione

MongoStandardAdapter è la classe dell'Outbound Adapter annotata come *Mongo Repository* che implementa *FindStandardPort*, traducendo le operazioni di recupero degli standard di conformità in interazioni concrete con MongoDB.

Attributi

MongoStandardAdapter non definisce attributi propri nel diagramma.

Metodi

- + save(standard: ComplianceStandard): void — persiste uno standard di conformità nel database.
- + find_by_id(standard_id: String): ComplianceStandard — recupera lo standard di conformità corrispondente all'identificativo fornito.

5.2.5) GetDeviceList

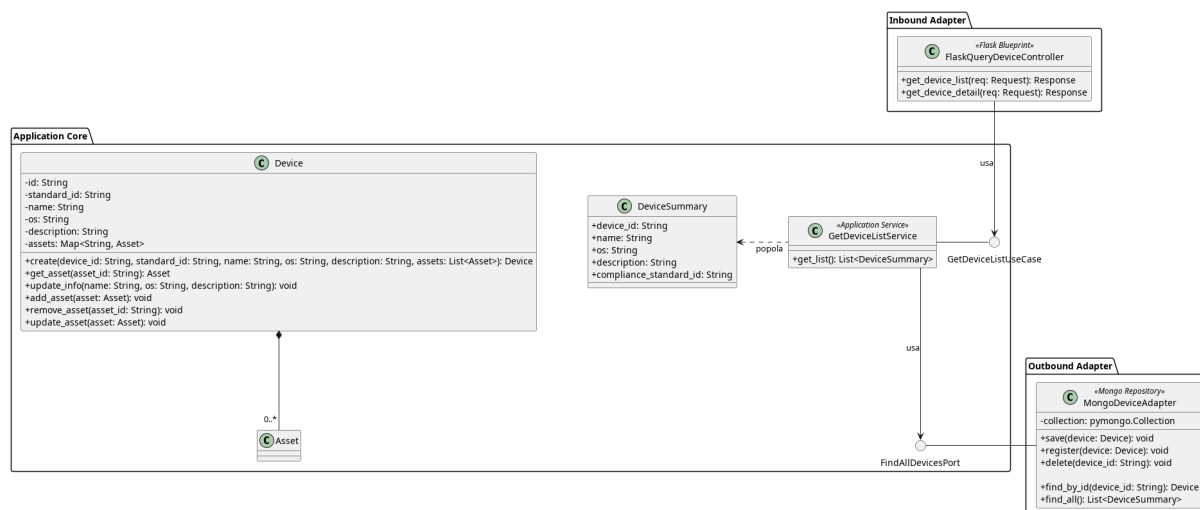


Figura 59: Caso d'uso GetDeviceList

Il diagramma illustra l'architettura del modulo dedicato al recupero della lista sintetica dei Dispositivi.

- Per la definizione di *FlaskQueryDeviceController*, vedere la sezione **Sezione 5.2.4.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *DeviceSummary*, vedere la sezione **Sezione 5.1.26**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.5.1) GetDeviceListUseCase

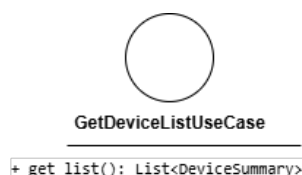


Figura 60: GetDeviceListUseCase

Descrizione

GetDeviceListUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero della lista sintetica dei Dispositivi. Viene implementata da *GetDeviceListService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetDeviceListUseCase non definisce attributi.

Metodi

- + get_list(): List<DeviceSummary> — firma del metodo delegato al recupero della lista sintetica di tutti i Dispositivi presenti nel sistema.

5.2.5.2) GetDeviceListService



Figura 61: GetDeviceListService

Descrizione

GetDeviceListService è il service applicativo appartenente all'Application Core responsabile del recupero della lista sintetica dei Dispositivi. Implementa l'interfaccia *GetDeviceListUseCase* e coordina il recupero tramite la porta outbound *FindAllDevicesPort*.

Attributi

- - `find_port: FindAllDevicesPort` — porta outbound utilizzata per il recupero della lista dei Dispositivi dal sistema di persistenza.

Metodi

- + `get_list(): List<DeviceSummary>` — concretizza il contratto definito da *GetDeviceListUseCase*. Recupera la lista sintetica di tutti i Dispositivi tramite *FindAllDevicesPort*.

5.2.5.3) DeviceSummary

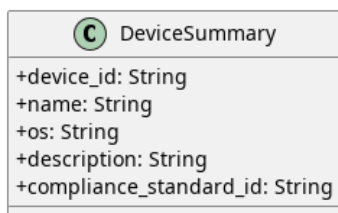


Figura 62: DeviceSummary

Descrizione

DeviceSummary è il Data Transfer Object che veicola la rappresentazione sintetica di un Dispositivo verso il livello di presentazione. Espone esclusivamente le informazioni necessarie alla visualizzazione in lista, evitando di esporre l'intera entità di dominio.

Attributi

- + `device_id: String` — identificativo univoco del dispositivo
- + `name: String` — nome del dispositivo
- + `os: String` — sistema operativo del dispositivo
- + `description: String` — descrizione del dispositivo
- + `compliance_standard_id: String` — identificativo univoco dello Standard

Metodi

DeviceSummary non definisce metodi.

5.2.5.4) FindAllDevicesPort

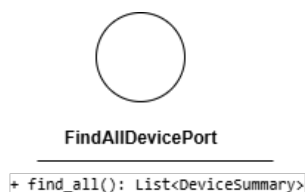


Figura 63: FindAllDevicesPort

Descrizione

FindAllDevicesPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della lista sintetica di tutti i Dispositivi dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceListService*.

Attributi

FindAllDevicesPort non definisce attributi.

Metodi

- `+ find_all(): List<DeviceSummary>` — firma del metodo che recupera la lista sintetica di tutti i Dispositivi presenti nel sistema di persistenza.

5.2.6) GetDeviceEvaluationDetail

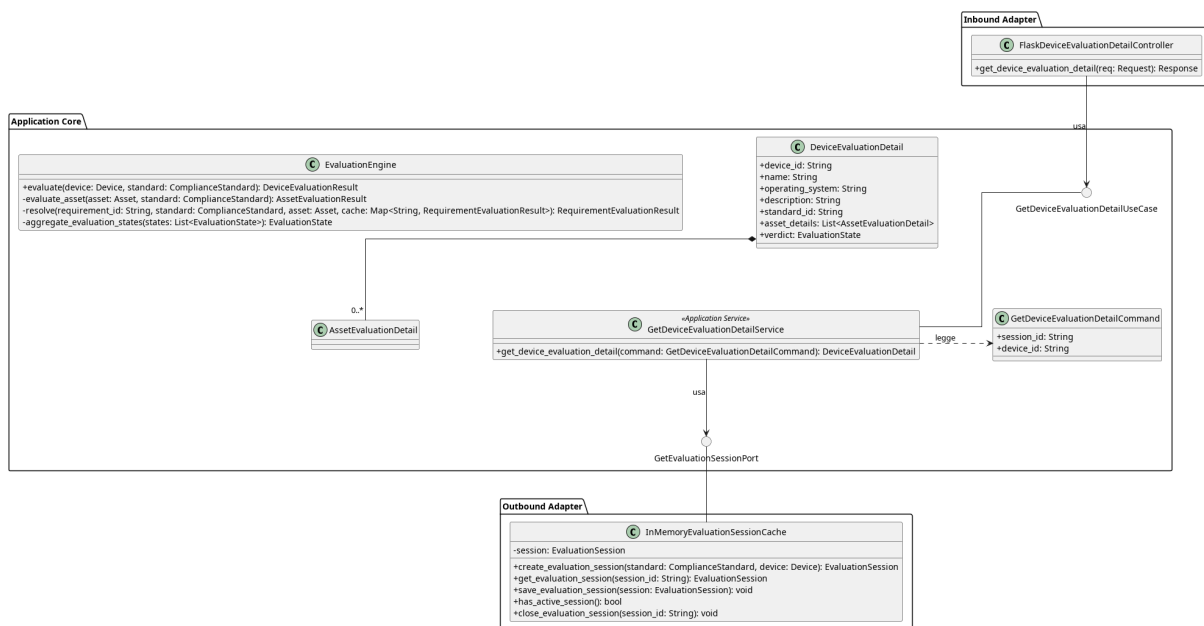


Figura 64: GetDeviceEvaluationDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero della dashboard di un Dispositivo, che aggrega le informazioni della sessione di valutazione attiva in una vista sintetica.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *EvaluationEngine*, vedere la sezione **Sezione 5.1.14**
- per la definizione di *DeviceEvaluationDetail*, vedere la sezione di **Sezione 5.1.22**

5.2.6.1) FlaskDeviceEvaluationDetailController

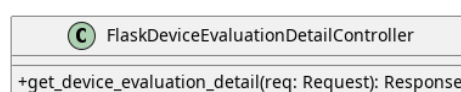


Figura 65: FlaskDeviceEvaluationDetailController

Descrizione

FlaskDeviceEvaluationDetailController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di lettura di un device per la dashboard.

Attributi

- -
get_device_evaluation_detail_use_case: GetDeviceEvaluationDetailUseCase
— inbound port usata per prelevare un *DeviceEvaluationDetail*.

Metodi

- + get_device_evaluation_detail(req: Request): Response — riceve la richiesta HTTP di recupero di un *DeviceEvaluationDetail* per la dashboard.

5.2.6.2) GetDeviceEvaluationDetailCommand

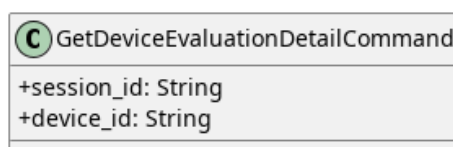


Figura 66: GetDeviceEvaluationDetailCommand

Descrizione

GetDeviceEvaluationDetailCommand è il Command Object utilizzato per trasportare i dati necessari al recupero della dashboard di un Dispositivo. Incapsula i parametri di input del metodo esposto da *GetDeviceEvaluationDetailUseCase*.

Attributi

- + session_id: String — identificativo univoco della sessione
- + device_id: String — identificativo univoco del Dispositivo di cui recuperare le informazioni.

Metodi

GetDeviceEvaluationDetailCommand non definisce metodi propri.

5.2.6.3) GetDeviceEvaluationDetailUseCase



Figura 67: GetDeviceEvaluationDetailUseCase

Descrizione

GetDeviceEvaluationDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero della dashboard di un Dispositivo. Viene implementata da *GetDeviceEvaluationDetailService* e utilizzata da *FlaskQueryDashboardController*.

Attributi

GetDeviceEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_device_evaluation_detail(command: GetDeviceEvaluationDetailCommand): DeviceEvaluationDetail` — firma del metodo delegato al recupero e all'aggregazione delle informazioni della sessione di valutazione.

5.2.6.4) GetDeviceEvaluationDetailService

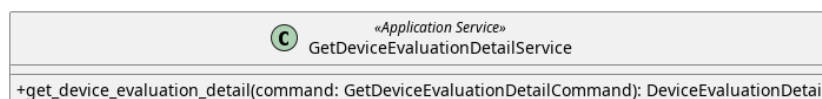


Figura 68: GetDeviceEvaluationDetailService

Descrizione

GetDeviceEvaluationDetailService è il service applicativo appartenente all'Application Core responsabile del recupero della dashboard di un Dispositivo. Implementa l'interfaccia *GetDeviceEvaluationDetailUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort* e costruisce l' *EvaluationDetailCommand* con le informazioni aggregate.

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port per prelevare la sessione di valutazione.

Metodi

- + `get_device_evaluation_detail(command: GetDeviceEvaluationDetailCommand): DeviceEvaluationDetail` —

concretizza il contratto definito da *GetDeviceEvaluationDetailUseCase*. Recupera la sessione attiva, aggrega le informazioni del Dispositivo e dei suoi Asset e restituisce la rappresentazione *DeviceEvaluationDetail*.

5.2.6.5) DTO

Qui vengono elencati i DTO usati dal controller per gestire ed esporre i dettagli della valutazione di un dispositivo.

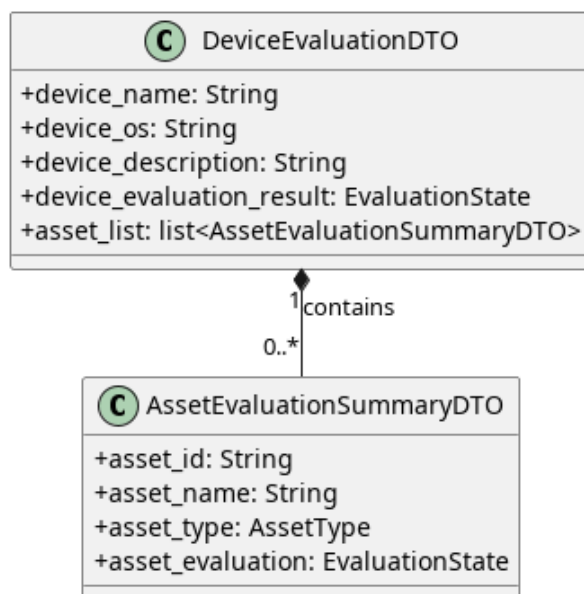


Figura 69: DeviceEvaluationDTO

5.2.6.5.1) DeviceEvaluationDTO

Descrizione

DeviceEvaluationDTO è il Data Transfer Object principale utilizzato per consolidare e trasportare le informazioni anagrafiche e l'esito complessivo della valutazione di un intero dispositivo. Funge da aggregatore principale per la vista dashboard, includendo al suo interno una lista sintetica dello stato di tutti gli asset ad esso associati.

Attributi

- + `device_name`: String — nome assegnato al dispositivo.
- + `device_os`: String — sistema operativo in uso sul dispositivo.
- + `device_description`: String — breve descrizione testuale del dispositivo.
- + `device_evaluation_result`: `EvaluationState` — stato globale e finale della valutazione di conformità per l'intero dispositivo.
- + `asset_list`: `List<AssetEvaluationSummaryDTO>` — tupla contenente i DTO di riepilogo per ciascun asset analizzato all'interno del dispositivo.

Metodi

DeviceEvaluationDTO non definisce metodi.

5.2.6.5.2) AssetEvaluationSummaryDTO

Descrizione

AssetEvaluationSummaryDTO è un Data Transfer Object leggero e di supporto, istanziato esclusivamente per popolare la lista degli asset all'interno di un *DeviceEvaluationDTO*. Fornisce una vista sintetica contenente le informazioni

essenziali e il verdetto di un singolo asset, risultando ottimale per le visualizzazioni a elenco o per le tabelle riassuntive nella dashboard.

Attributi

- + asset_id: String — identificativo univoco dell'asset valutato.
- + asset_name: String — nome dell'asset.
- + asset_type: AssetType — categoria o tipologia a cui appartiene l'asset.
- + asset_evaluation: EvaluationState — stato corrente e finale della valutazione specifica per questo asset.

Metodi

AssetEvaluationSummaryDTO non definisce metodi.

5.3) Asset

5.3.1) CreateAsset

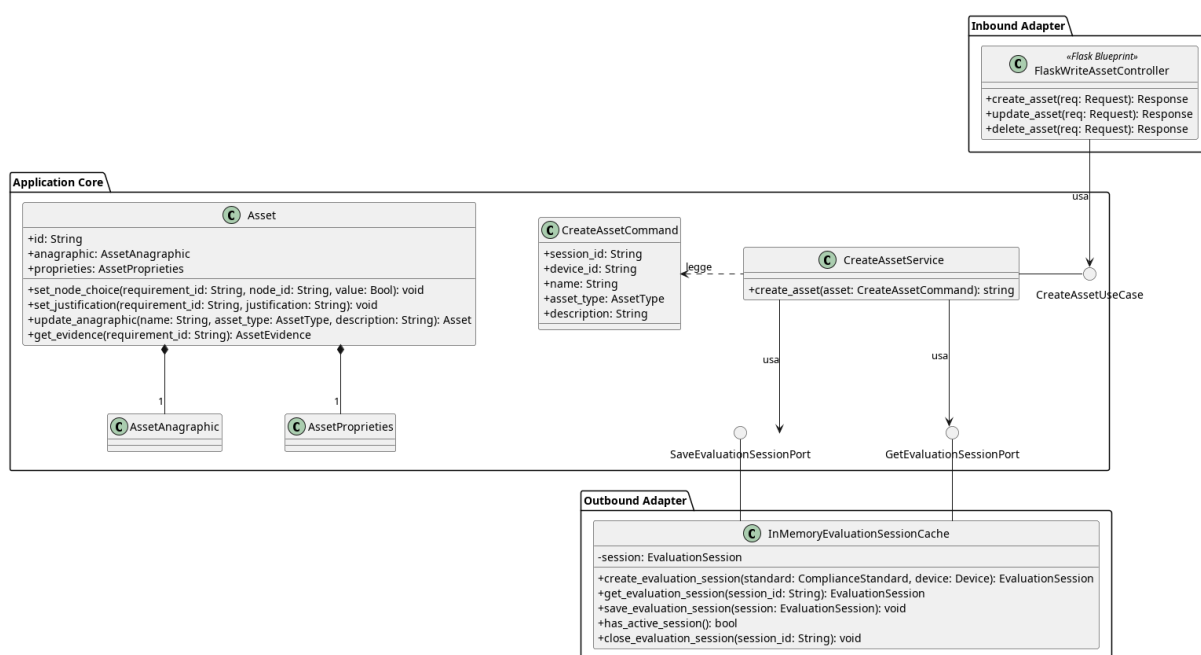


Figura 70: Caso d'uso CreateAsset

Il diagramma illustra l'architettura del modulo di creazione di un Asset all'interno di una sessione di valutazione attiva.

- per la definizione di *Asset*, vedere la sezione di **Sezione 5.1.2**

5.3.1.1) FlaskWriteAssetController



Figura 71: FlaskWriteAssetController

Descrizione

FlaskWriteAssetController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione (scrittura) degli Asset e le inoltra al livello applicativo.

Attributi

- - `create_asset_use_case`: `CreateAssetUseCase` — inbound port usata per la creazione di un asset
- - `delete_asset_use_case`: `DeleteAssetUseCase` — inbound port usata per la rimozione di un asset
- - `update_asset_use_case`: `UpdateAssetUseCase` — inbound port usata per l'aggiornamento di un asset

Metodi

- + `create_asset(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Asset, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `update_asset(req: Request): Response` — riceve la richiesta HTTP di aggiornamento di un Asset esistente, estrae i dati modificati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_asset(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Asset, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.3.1.2) CreateAssetUseCase

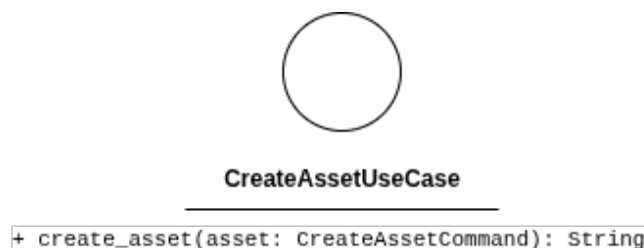


Figura 72: CreateAssetUseCase

Descrizione

CreateAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Asset all'interno della sessione di valutazione. Viene implementata da *CreateAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

CreateAssetUseCase non definisce attributi.

Metodi

- + `create_asset(asset: CreateAssetCommand): String` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.3.1.3) CreateAssetCommand

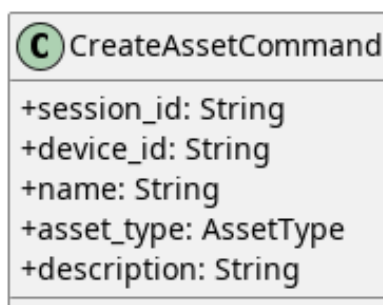


Figura 73: CreateAssetCommand

Descrizione

CreateAssetCommand è il Command Object che veicola i dati necessari alla creazione di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + `session_id: String` — identificativo della sessione di valutazione attiva a cui l'Asset viene associato.
- + `device_id: String` — identificativo del dispositivo che lo contiene.
- + `name: String` — nome del nuovo Asset.
- + `type: AssetType` — tipo dell'Asset.
- + `description: String` — descrizione testuale dell'Asset.

Metodi

CreateAssetCommand non definisce metodi.

5.3.1.4) CreateAssetService

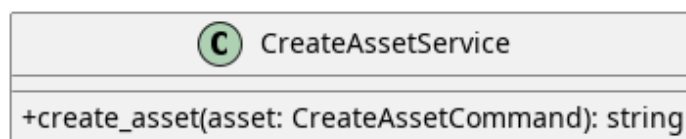


Figura 74: CreateAssetService

Descrizione

CreateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Asset. Implementa l'interfaccia *CreateAssetUseCase*, recupera la sessione attiva tramite *GetEvaluationSession*, aggiunge il nuovo Asset e persiste la sessione aggiornata tramite *SaveEvaluationSession*.

Attributi

- - `save_evaluation_session_port`: `SaveSessionPort` — outbound port usata per il salvataggio delle modifiche nella sessione
- - `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per prelevare la sessione di valutazione

Metodi

- + `create_asset(asset: CreateAssetCommand): string` — concretizza il contratto definito da *CreateAssetUseCase*. Recupera la sessione attiva, vi aggiunge il nuovo Asset e ne persiste lo stato aggiornato. Ritorna l'id dell'asset creato.

5.3.1.5) InMemoryEvaluationSessionCache

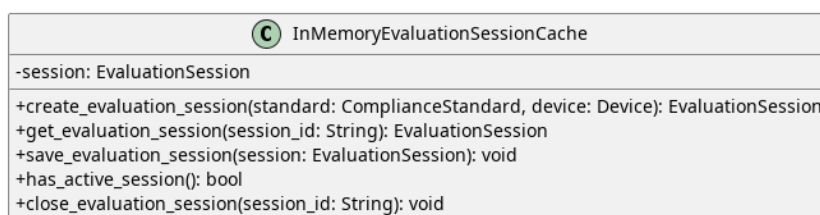


Figura 75: InMemoryEvaluationSessionCache

Descrizione

InMemoryEvaluationSessionCache è l'Outbound Adapter che funge da Session Cache. Poiché il sistema prevede un utilizzo mono-utente, la classe gestisce in memoria una singola sessione di valutazione attiva per volta.

Attributi

- - `session`: `EvaluationSession` — contiene l'oggetto `EvaluationSession`

Metodi

- + `create_evaluation_session(standard: ComplianceStandard, device: Device): EvaluationSession` — crea e registra una nuova sessione di valutazione.
- + `get_evaluation_session(session_id: String): EvaluationSession` — recupera la sessione corrispondente all'identificativo fornito.
- + `save_evaluation_session(session: EvaluationSession): void` — aggiorna la sessione in memoria.
- + `has_active_session(): bool` — verifica se esiste una sessione attiva.
- + `close_evaluation_session(session_id: String): void` — chiude la sessione identificata da `session_id`.

5.3.1.6) SaveEvaluationSessionPort



Figura 76: SaveEvaluationSessionPort

Descrizione

SaveEvaluationSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il salvataggio della sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che modificano lo stato della sessione, tra cui *CreateAssetService*, *SaveAssetService* e *DeleteAssetService*.

Attributi

SaveEvaluationSessionPort non definisce attributi.

Metodi

- + `save_evaluation_session(session: EvaluationSession): void` — firma del metodo che persiste la sessione aggiornata nel sistema in memoria.

5.3.1.7) GetEvaluationSessionPort



Figura 77: GetEvaluationSessionPort

Descrizione

GetEvaluationSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della sessione di valutazione dal sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del

modulo che necessitano di accedere alla sessione corrente, tra cui *CreateAssetService*, *SaveAssetService*, *DeleteAssetService* e *GetAssetAnagraphicService*.

Attributi

GetEvaluationSessionPort non definisce attributi.

Metodi

- + `get_evaluation_session(session_id: String): EvaluationSession` — firma del metodo che recupera la sessione di valutazione attiva corrispondente all'identificativo fornito.

5.3.2) UpdateAsset

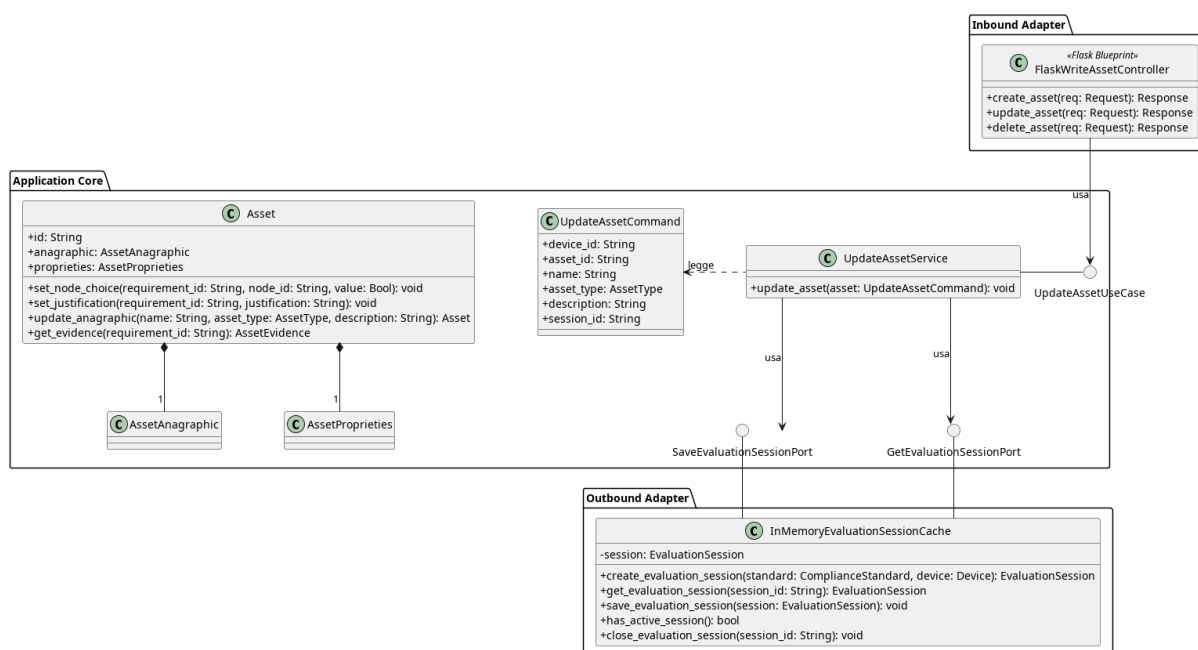


Figura 78: Caso d'uso UpdateAsset

Il diagramma illustra l'architettura del modulo di modifica di un Asset esistente all'interno di una sessione di valutazione attiva.

- Per la definizione di *FlaskWriteAssetController*, vedere la sezione **Sezione 5.3.1.1.**
- Per la definizione di *Asset*, vedere la sezione **Sezione 5.1.2.**
- Per la definizione di *SaveEvaluationSession*, vedere la sezione **Sezione 5.3.1.6.**
- Per la definizione di *GetEvaluationSession*, vedere la sezione **Sezione 5.3.1.7.**
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5.**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.2.1) UpdateAssetUseCase



Figura 79: UpdateAssetUseCase

Descrizione

UpdateAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Asset esistente all'interno della sessione di valutazione. Viene implementata da *UpdateAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

UpdateAssetUseCase non definisce attributi.

Metodi

- + `update_asset(asset: UpdateAssetCommand): void` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.3.2.2) UpdateAssetCommand

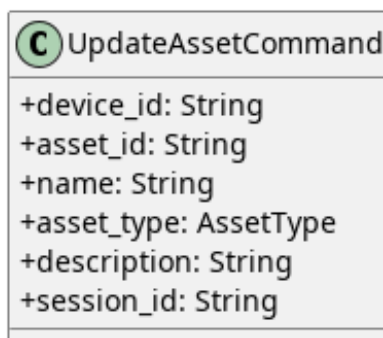


Figura 80: UpdateAssetCommand

Descrizione

UpdateAssetCommand è il Command Object che veicola i dati necessari alla modifica di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- + `device_id: String` — identificativo del device a cui appartiene.
- + `asset_id: String` — identificativo univoco dell'Asset da modificare.
- + `name: String` — nome aggiornato dell'Asset.
- + `asset_type: AssetType` — tipo aggiornato dell'Asset.
- + `description: String` — descrizione testuale aggiornata dell'Asset.

- + session_id: String — identificativo della sessione di valutazione attiva.

Metodi

UpdateAssetCommand non definisce metodi.

5.3.2.3) UpdateAssetService

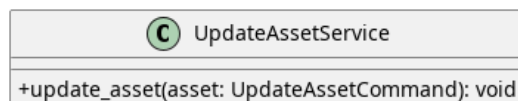


Figura 81: UpdateAssetService

Descrizione

UpdateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Asset esistente. Implementa l'interfaccia *UpdateAssetUseCase*, recupera la sessione attiva tramite *GetEvaluationSession*, aggiorna l'Asset corrispondente e persiste la sessione modificata tramite *SaveEvaluationSession*.

Attributi

- - save_evaluation_session_port: *SaveSessionPort* — outbound port usata per il salvataggio delle modifiche nella sessione
- - get_evaluation_session_port: *GetEvaluationSessionPort* — outbound port usata per prelevare la sessione di valutazione

Metodi

- + update_asset(asset: *UpdateAssetCommand*): void — concretizza il contratto definito da *UpdateAssetUseCase*. Recupera la sessione attiva, individua l'Asset da aggiornare tramite `asset_id` e ne persiste lo stato modificato.

5.3.3) DeleteAsset

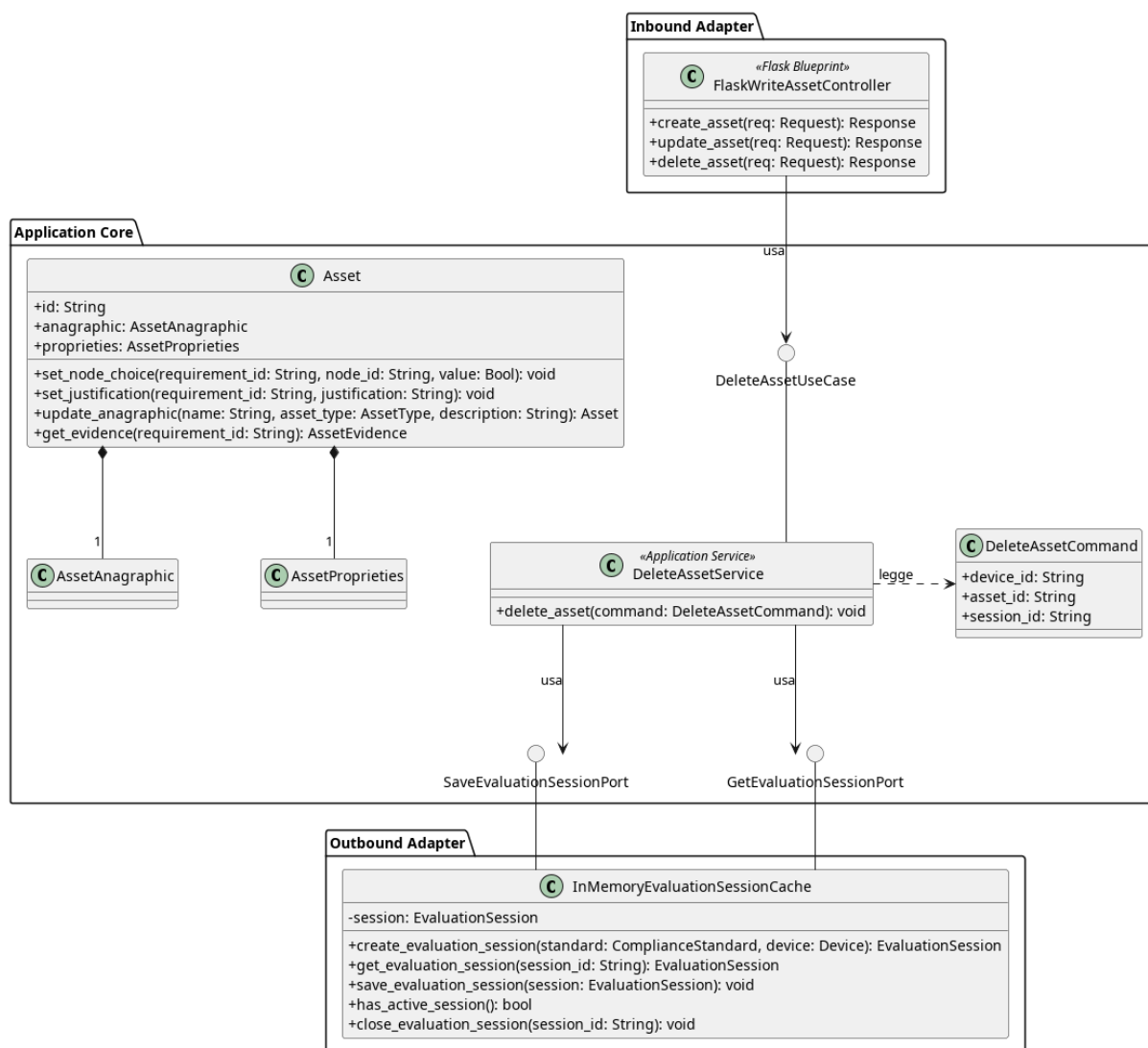


Figura 82: Caso d'uso DeleteAsset

Il diagramma illustra l'architettura del modulo di eliminazione di un Asset esistente all'interno di una sessione di valutazione attiva.

- Per la definizione di *FlaskWriteAssetController*, vedere la sezione **Sezione 5.3.1.1**.
- Per la definizione di *Asset*, vedere la sezione **Sezione 5.1.2**.
- Per la definizione di *SaveEvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *GetEvaluationSession*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.3.1) DeleteAssetCommand



Figura 83: DeleteAssetCommand

Descrizione

DeleteAssetCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un Asset dalla sessione di valutazione. Incapsula i parametri di input del metodo esposto da *DeleteAssetUseCase*.

Attributi

- `+ device_id: String` — identificativo univoco del dispositivo che contiene l'asset.
- `+ asset_id: String` — identificativo univoco del Asset da eliminare.
- `+ session_id: String` — identificativo univoco della sessione.

Metodi

DeleteAssetCommand non definisce metodi propri.

5.3.3.2) DeleteAssetUseCase



Figura 84: DeleteAssetUseCase

Descrizione

DeleteAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Asset dalla sessione di valutazione. Viene implementata da *DeleteAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

DeleteAssetUseCase non definisce attributi.

Metodi

- + delete_asset(command: DeleteAssetCommand): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dai dati contenuti nel Command.

5.3.3.3) DeleteAssetService

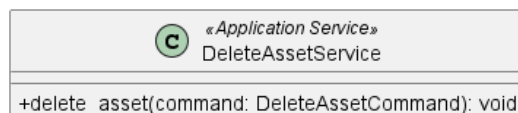


Figura 85: DeleteAssetService

Descrizione

DeleteAssetService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Asset. Implementa l'interfaccia *DeleteAssetUseCase*, riceve un *DeleteAssetCommand* contenente gli identificativi necessari, recupera la sessione tramite *GetEvaluationSession*, rimuove l'Asset corrispondente e persiste la sessione aggiornata tramite *SaveEvaluationSession*.

Attributi

- - save_evaluation_session_port: SaveSessionPort — outbound port usata per il salvataggio delle modifiche nella sessione
- - get_evaluation_session_port: GetEvaluationSessionPort — outbound port usata per prelevare la sessione di valutazione

Metodi

- + delete_asset(command: DeleteAssetCommand): void — concretizza il contratto definito da *DeleteAssetUseCase*. Recupera la sessione attiva, individua e rimuove l'Asset corrispondente e ne persiste lo stato aggiornato.

5.3.4) GetAssetAnagraphic

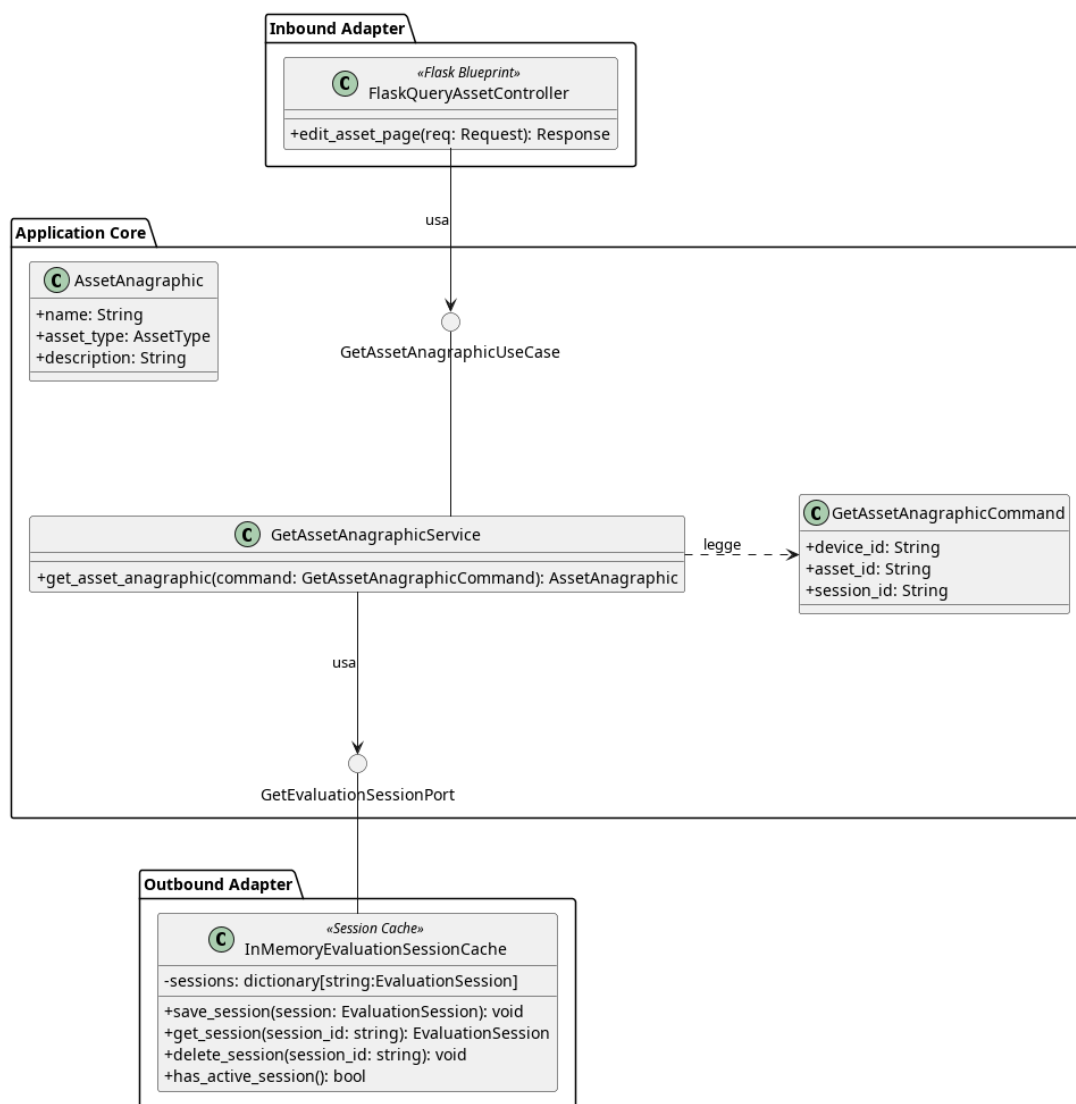


Figura 86: Caso d'uso GetAssetAnagraphic

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Asset all'interno di una sessione di valutazione attiva.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**
- Per la definizione di *AssetAnagraphic*, vedere la sezione **Sezione 5.1.3**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.4.1) FlaskQueryAssetController



Figura 87: FlaskQueryAssetController

Descrizione

FlaskQueryAssetController è un adapter di input (controller) basato su Flask. Ha la responsabilità di intercettare le richieste web in lettura relative agli Asset, interagire con i casi d'uso applicativi e restituire le viste HTML (template) popolate con i dati di dominio mappati tramite Data Transfer Object (DTO).

Attributi

- - `_get_asset_anagraphic_use_case`: `GetAssetAnagraphicUseCase` — istanza del caso d'uso iniettata a runtime, necessaria per recuperare le informazioni anagrafiche di un asset specifico.

Metodi

- + `edit_asset_page(session_id: String, device_id: String, asset_id: String): Response` — delega al caso d'uso il recupero dell'anagrafica dell'asset, mappa le informazioni di dominio in un `AssetAnagraphicDTO` e restituisce il template HTML precompilato per la visualizzazione/modifica.

5.3.4.2) GetAssetAnagraphicCommand

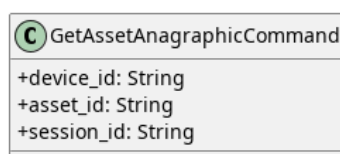


Figura 88: GetAssetAnagraphicCommand

Descrizione

GetAssetAnagraphicCommand è il Command Object utilizzato per trasportare i dati necessari al recupero del dettaglio di un Asset. Incapsula i parametri di input del metodo esposto da *GetAssetAnagraphicUseCase*.

Attributi

- + `device_id: String` — identificativo univoco del device che contiene l'asset
- + `asset_id: String` — identificativo univoco dell'Asset di cui recuperare il dettaglio.
- + `session_id: String` — identificativo univoco della sessione di valutazione corrente.

Metodi

GetAssetAnagraphicCommand non definisce metodi propri.

5.3.4.3) GetAssetAnagraphicUseCase

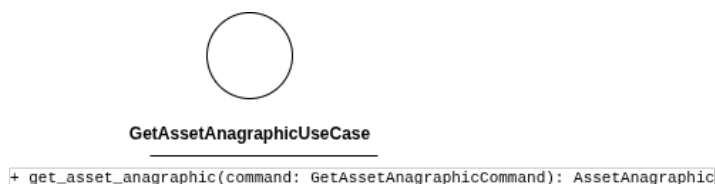


Figura 89: GetAssetAnagraphicUseCase

Descrizione

GetAssetAnagraphicUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Asset. Viene implementata da *GetAssetAnagraphicService*.

Attributi

GetAssetAnagraphicUseCase non definisce attributi.

Metodi

- + `get_asset_anagraphic(command: GetAssetAnagraphicCommand): AssetAnagraphic` — firma del metodo delegato al recupero della anagrafica dell'asset ricercato.

5.3.4.4) GetAssetAnagraphicService

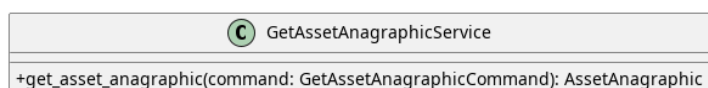


Figura 90: GetAssetAnagraphicService

Descrizione

GetAssetAnagraphicService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Asset. Implementa l'interfaccia *GetAssetAnagraphicUseCase*, recupera la sessione attiva tramite *GetEvaluationSession* e ritorna l'*AssetAnagraphic* tramite *Asset*.

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port usata per prelevare la sessione di valutazione

Metodi

- + `get_asset_anagraphic(command: GetAssetAnagraphicCommand): AssetAnagraphic` — concretizza il contratto definito da

GetAssetAnagraphicUseCase. Recupera la sessione attiva, individua l'Asset richiesto e ne estrae l'*AssetAnagraphic*.

5.3.5) GetAssetEvaluationDetail

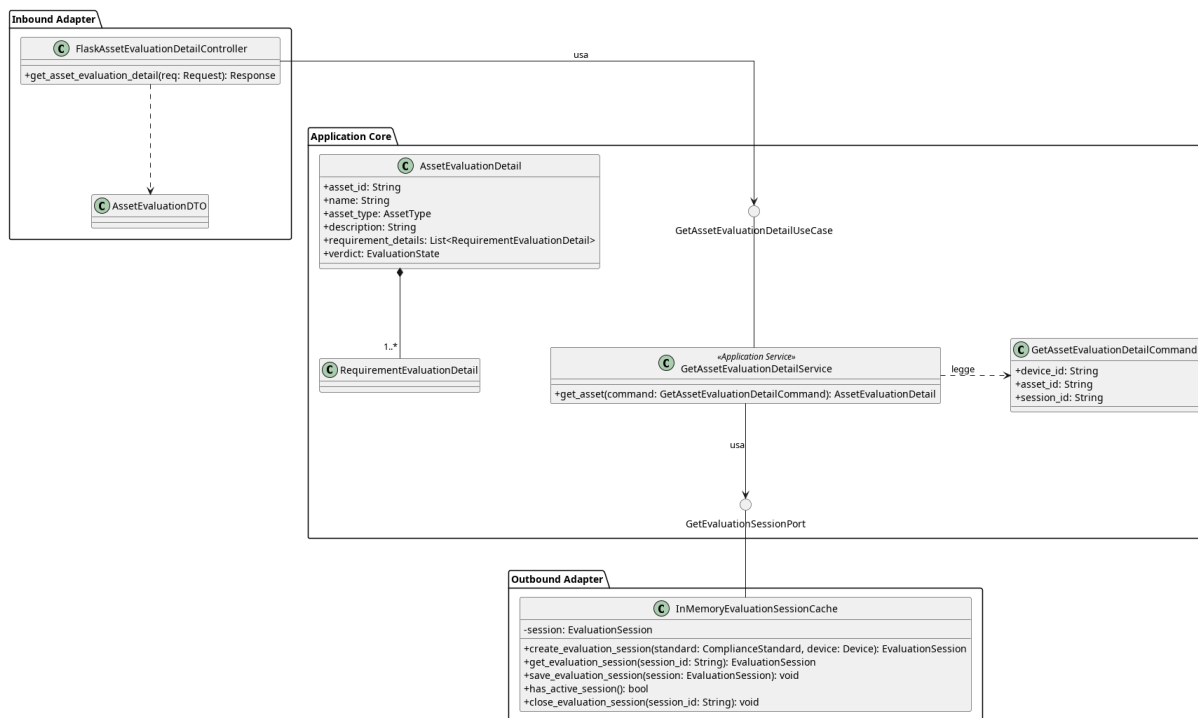


Figura 91: GetAssetEvaluationDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero di un *AssetEvaluationDetail* contenente informazioni anagrafiche e stato di valutazione del dispositivo.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *AssetEvaluationDetail*, vedere la sezione **Sezione 5.1.21**

5.3.5.1) FlaskAssetEvaluationDetailController

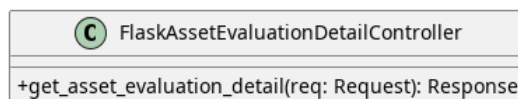


Figura 92: GetAssetEvaluationDetail

Descrizione

FlaskAssetEvaluationDetailController è un adapter di input (controller) che ha la responsabilità di intercettare le richieste per la visualizzazione del dettaglio di valutazione di un asset, interrogare il core applicativo (attraverso le porte di inbound) e presentare i risultati o gli eventuali errori di dominio alla vista.

Attributi

- - `_get_asset_ev_detail_use_case`: `GetAssetDetailUseCase` — istanza del caso d'uso necessaria per recuperare i dettagli della valutazione dell'asset richiesto.

Metodi

- + `get_asset_evaluation_detail(session_id: String, device_id: String, asset_id: String)`: `Response` — preleva tramite la porta di inbound l'oggetto di dominio `AssetEvaluationDetail`.

5.3.5.2) GetAssetEvaluationDetailCommand

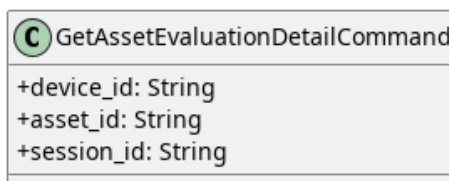


Figura 93: GetAssetEvaluationDetailCommand

Descrizione

GetAssetEvaluationDetailCommand è il Command Object utilizzato per trasportare i dati necessari al recupero di un *AssetEvaluationDetail*. Incapsula i parametri di input del metodo esposto da *GetAssetEvaluationDetailUseCase*.

Attributi

- + `session_id`: `String` — identificativo univoco della sessione
- + `device_id`: `String` — identificativo univoco del Dispositivo di cui recuperare le informazioni.

Metodi

GetAssetEvaluationDetailCommand non definisce metodi propri.

5.3.5.3) GetAssetEvaluationDetailUseCase



Figura 94: GetAssetEvaluationDetailUseCase

Descrizione

GetAssetEvaluationDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero di un *AssetEvaluationDetail*. Viene implementata da *GetAssetEvaluationDetailService* e utilizzata da *FlaskAssetEvaluationDetailController*.

Attributi

GetAssetEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_asset(command: GetAssetEvaluationDetailCommand): AssetEvaluationDetail` — firma del metodo delegato al recupero e all'aggregazione delle informazioni della sessione di valutazione.

5.3.5.4) GetAssetEvaluationDetailsService

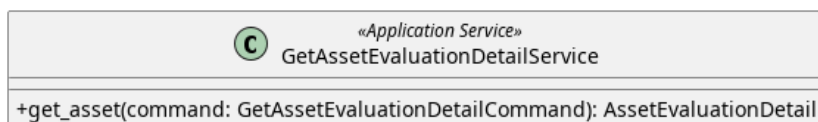


Figura 95: GetAssetEvaluationDetailsService

Descrizione

Concretizza il contratto definito da *GetAssetEvaluationDetailUseCase*. Recupera la sessione di valutazione specificata, esegue la valutazione del dispositivo tramite l'engine e isola i risultati dell'Asset richiesto. Infine, aggrega i dati anagrafici di tale Asset con i risultati della valutazione dei suoi requisiti, restituendo la rappresentazione *AssetEvaluationDetail*

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port usata per recuperare la sessione.

Metodi

- + `get_asset(command: GetAssetEvaluationDetailCommand): AssetEvaluationDetail` —

concretizza il contratto definito da *GetAssetEvaluationDetailUseCase*. Recupera la sessione attiva, aggrega le informazioni del Dispositivo e dei suoi Asset e restituisce la rappresentazione *AssetEvaluationDetail*.

5.3.5.5) DTO

Qui vengono elencati i dto usati dal controller per gestire ed esporre i dettagli della valutazione di un asset verso l'interfaccia frontend o le API.

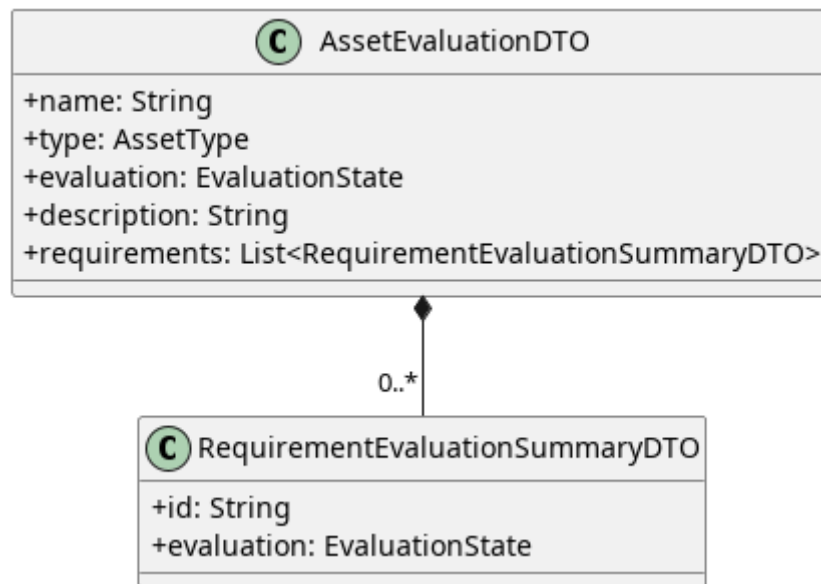


Figura 96: AssetEvaluationDTO

5.3.5.5.1) AssetEvaluationDTO

Descrizione

AssetEvaluationDTO è il Data Transfer Object principale utilizzato per esporre all'interfaccia utente i dettagli anagrafici e l'esito complessivo della valutazione di uno specifico asset. Agisce come aggregatore, includendo al suo interno una lista sintetica dello stato di tutti i requisiti ad esso associati.

Attributi

- + name: String — nome dell'asset valutato.
- + type: AssetType — categoria o tipologia a cui appartiene l'asset.
- + evaluation: EvaluationState — stato globale e finale della valutazione di conformità per l'intero asset.
- + description: String — descrizione testuale aggiuntiva dell'asset.
- + requirements: List<RequirementEvaluationSummaryDTO> — tupla contenente i DTO di riepilogo per ciascun requisito associato all'asset.

Metodi

AssetEvaluationDTO non definisce metodi.

5.3.5.5.2) RequirementEvaluationSummaryDTO

Descrizione

RequirementEvaluationSummaryDTO è un Data Transfer Object estremamente leggero e di supporto, istanziato esclusivamente per popolare la lista dei requisiti all'interno di un *AssetEvaluationDTO*. Fornisce una vista sintetica limitata all'identificativo e al verdetto, ideale per le visualizzazioni a elenco o in forma di tabella.

Attributi

- + id: String — identificativo univoco del requisito valutato.
- + evaluation: EvaluationState — stato corrente della valutazione specifica per questo requisito.

Metodi

RequirementEvaluationSummaryDTO non definisce metodi.

5.3.6) GetRequirement

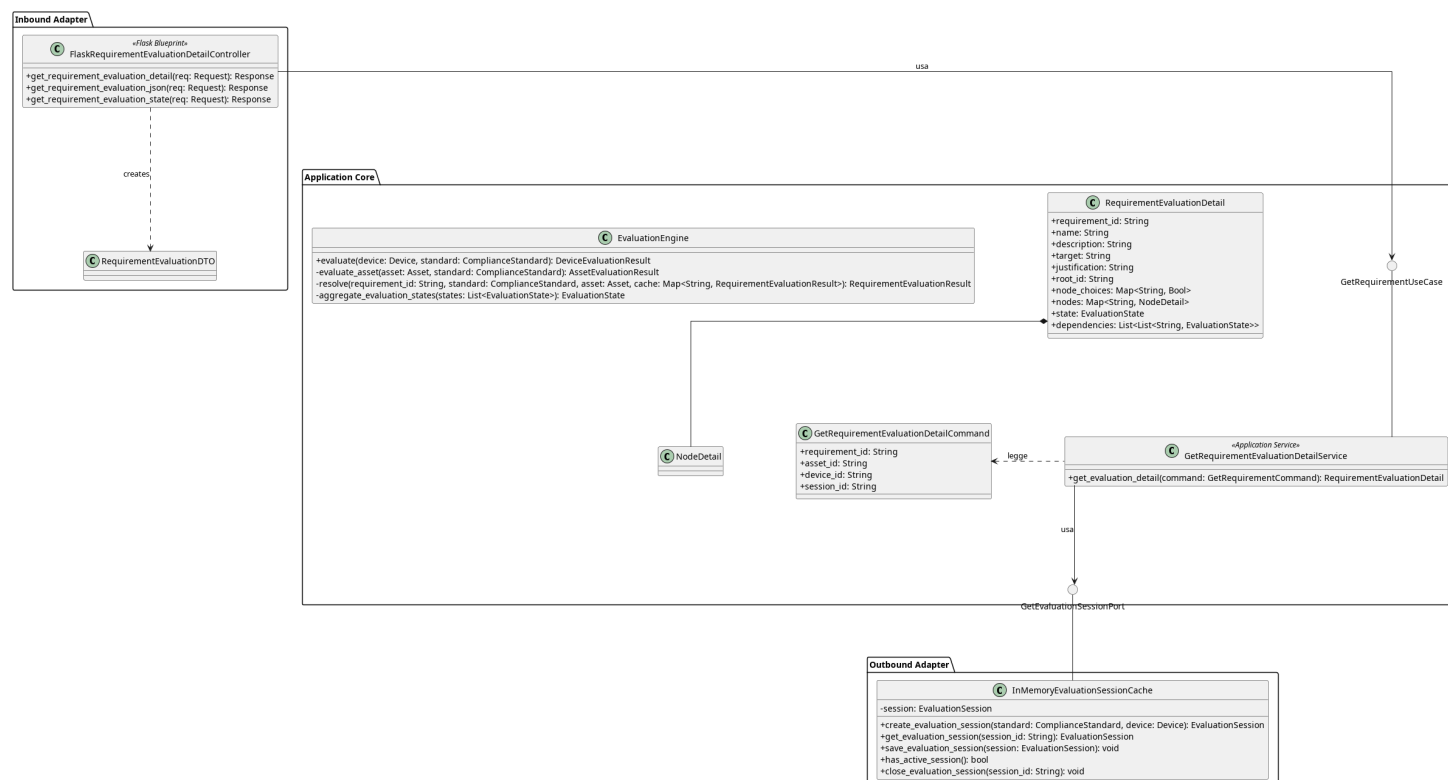


Figura 97: Caso d'uso GetRequirement

Il diagramma illustra l'architettura del modulo dedicato al recupero di un requisito di conformità con il relativo albero decisionale e le dipendenze associate.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *EvaluationEngine*, vedere la sezione **Sezione 5.1.14**
- Per la definizione di *RequirementEvaluationDetail*, vedere la sezione **Sezione 5.1.20**

5.3.6.1) FlaskRequirementEvaluationDetailController

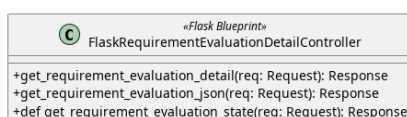


Figura 98: FlaskRequirementEvaluationDetailController

Descrizione

FlaskRequirementEvaluationDetailController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di recupero di un requisito di conformità e le inoltra al livello applicativo.

Attributi

- - `get_requirement_ev_detail_use_case:`
GetRequirementEvaluationDetailUseCase — inbound port usata per recuperare l'oggetto *RequirementEvaluationDetail*

Metodi

- + `get_requirement_evaluation_detail(req: Request): Response` — endpoint GET che restituisce l'intera pagina HTML con il dettaglio completo del requisito valutato.
- + `get_requirement_evaluation_json(req: Request): Response` — endpoint GET che restituisce il dettaglio completo della valutazione del requisito in formato JSON.
- + `get_requirement_evaluation_state(req: Request): Response` — endpoint GET leggero che restituisce esclusivamente lo stato attuale di valutazione in formato JSON.

5.3.6.2) GetRequirementEvaluationDetailUseCase

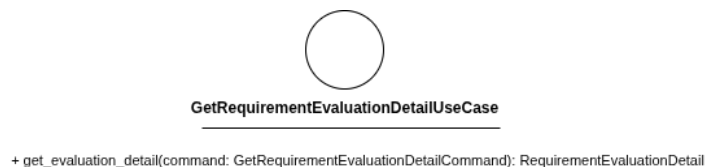


Figura 99: GetRequirementEvaluationDetailUseCase

Descrizione

GetRequirementEvaluationDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero di un requisito di conformità. Viene implementata da *GetRequirementEvaluationService* e utilizzata da *FlaskRequirementEvaluationDetailController*.

Attributi

GetRequirementEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_evaluation_detail(command: GetRequirementEvaluationDetailCommand): RequirementResponse` — firma del metodo delegato al recupero del requisito corrispondente ai parametri incapsulati nel comando fornito in input.

5.3.6.3) GetRequirementEvaluationDetailCommand

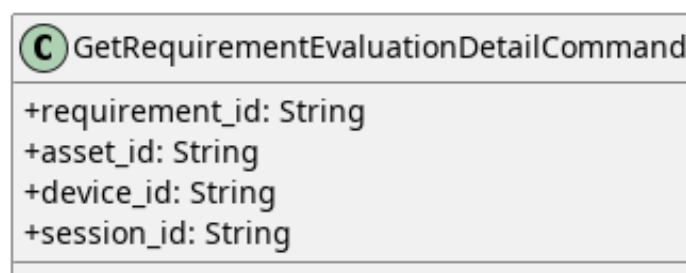


Figura 100: GetRequirementEvaluationDetailCommand

Descrizione

GetRequirementEvaluationDetailCommand è l'oggetto che veicola i parametri necessari al recupero di un requisito dal controller al service.

Attributi

- + `requirement_id: String` — identificativo del requisito da recuperare.
- + `asset_id: String` — identificativo dell'asset oggetto di valutazione.
- + `device_id: String` — identificativo del dispositivo contenente l'asset.
- + `session_id: String` — identificativo della sessione di valutazione attiva.

Metodi

GetRequirementEvaluationDetailCommand non definisce metodi.

5.3.6.4) GetRequirementEvaluationDetailService

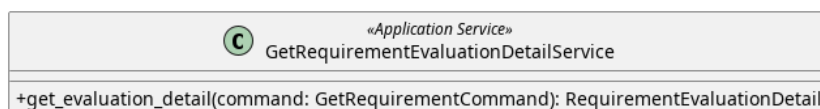


Figura 101: GetRequirementEvaluationDetailService

Descrizione

GetRequirementEvaluationDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di valutazione di un singolo requisito. Implementa l'interfaccia *GetRequirementEvaluationDetailUseCase*, legge i parametri dal *GetRequirementEvaluationDetailCommand*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, esegue la valutazione tramite l'*EvaluationEngine* e costruisce un *RequirementEvaluationDetail*.

Attributi

- `get_evaluation_session_port: GetEvaluationSessionPort`
- `evaluation_engine: EvaluationEngine`

Metodi

- + `get_evaluation_detail(command: GetRequirementEvaluationDetailCommand): RequirementEvaluationDetail` — concretizza il contratto definito da *GetRequirementEvaluationDetailUseCase*. Recupera la sessione attiva, individua il requisito richiesto utilizzando i parametri incapsulati nel comando e ne costruisce la relativa rappresentazione sotto forma di *RequirementEvaluationDetail*.

5.3.6.5) DTO

Qui vengono elencati i dto usati dal controller per gestire ed esporre i dettagli della valutazione di un requisito.

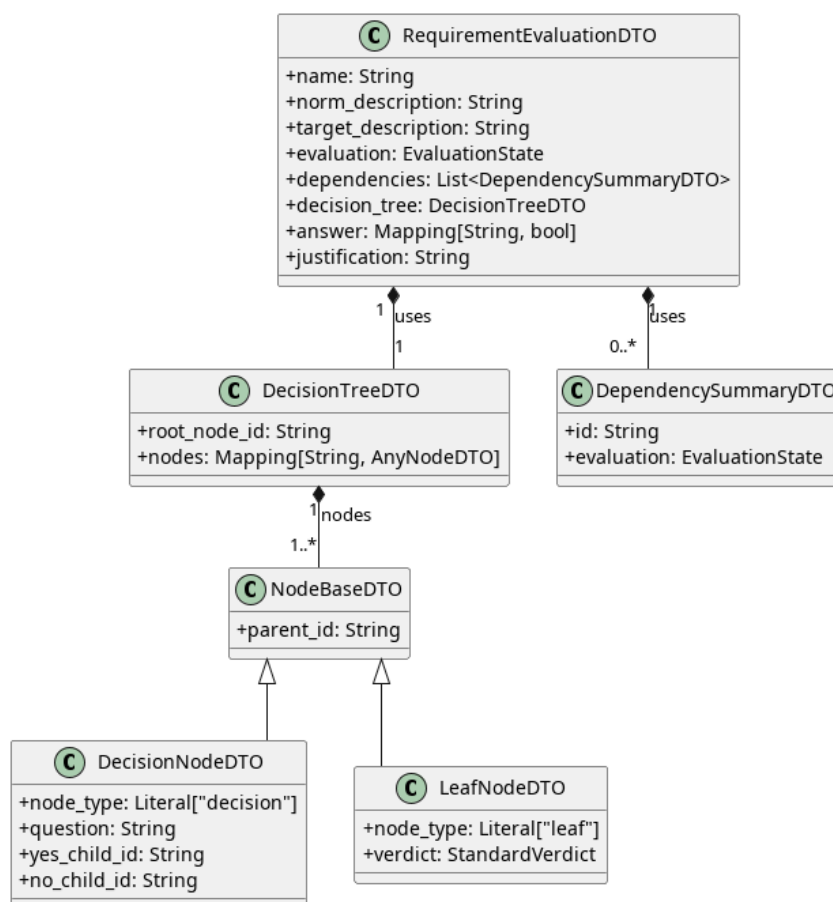


Figura 102: RequirementEvaluationDTO

5.3.6.5.1) RequirementEvaluationDTO

Descrizione

RequirementEvaluationDTO è il Data Transfer Object principale che consolida tutte le informazioni necessarie per la visualizzazione del dettaglio di un singolo requisito valutato. Raccoglie i dati anagrafici, l'esito della valutazione, lo stato delle dipendenze, la struttura dell'albero decisionale e le risposte fornite dall'utente.

Attributi

- + `name: String` — nome del requisito.

- + norm_description: String — descrizione normativa estesa del requisito.
- + target_description: String — descrizione dell'obiettivo del requisito.
- + evaluation: EvaluationState — stato corrente e complessivo della valutazione.
- + dependencies: List<DependencySummaryDTO> — collezione di DTO che riepilogano lo stato dei requisiti da cui questo dipende.
- + decision_tree: DecisionTreeDTO — DTO che mappa la struttura dell'albero decisionale associato al requisito.
- + answer: Map<String, Bool> — mappa che associa gli ID dei nodi decisionali alle risposte fornite.
- + justification: String | None — eventuale giustificazione testuale fornita durante la valutazione.

Metodi

RequirementEvaluationDTO non definisce metodi.

5.3.6.5.2) DependencySummaryDTO

Descrizione

DependencySummaryDTO è un Data Transfer Object leggero utilizzato per esporre esclusivamente l'identificativo e lo stato di valutazione di un requisito che funge da dipendenza.

Attributi

- + id: String — identificativo del requisito dipendente.
- + evaluation: EvaluationState — stato attuale della valutazione della dipendenza.

Metodi

DependencySummaryDTO non definisce metodi.

5.3.6.5.3) DecisionTreeDTO

Descrizione

DecisionTreeDTO è il DTO responsabile di incapsulare l'intera struttura dell'albero decisionale. Espone il riferimento al nodo di partenza e l'insieme di tutti i nodi che compongono l'albero, utilizzando il polimorfismo per rappresentare sia i nodi decisionali che quelli terminali.

Attributi

- + root_node_id: String — identificativo del nodo radice da cui inizia la navigazione dell'albero.

- + nodes: Map<String, AnyNodeDTO> — dizionario che associa gli identificativi univoci dei nodi ai rispettivi oggetti DTO (che possono essere istanze di *DecisionNodeDTO* o *LeafNodeDTO*).

Metodi

DecisionTreeDTO non definisce metodi.

5.3.6.5.4) LeafNodeDTO

Descrizione

LeafNodeDTO estende *NodeBaseDTO* e rappresenta un nodo terminale (foglia) dell'albero decisionale. Definisce l'esito finale della valutazione per quel particolare percorso.

Attributi

- + parent_id: String | None — identificativo del nodo genitore (ereditato).
- + node_type: String — costante valorizzata a «leaf», utilizzata come discriminatore dal frontend e dai validatori per distinguere il tipo di nodo.
- + verdict: StandardVerdict — verdetto di conformità finale associato alla foglia.

Metodi

LeafNodeDTO non definisce metodi.

5.3.6.5.5) DecisionNodeDTO

Descrizione

DecisionNodeDTO estende *NodeBaseDTO* e rappresenta uno snodo intermedio dell'albero decisionale. Contiene la domanda da porre all'utente e i riferimenti per la navigazione in base alla risposta data.

Attributi

- + parent_id: String | None — identificativo del nodo genitore (ereditato).
- + node_type: String — costante valorizzata a «decision», utilizzata come discriminatore strutturale.
- + question: String — testo della domanda posta dal nodo decisionale.
- + yes_child_id: String | None — identificativo del nodo figlio da visitare nel caso la risposta sia affermativa.
- + no_child_id: String | None — identificativo del nodo figlio da visitare nel caso la risposta sia negativa.

Metodi

DecisionNodeDTO non definisce metodi.

5.3.6.5.6) NodeBaseDTO

Descrizione

NodeBaseDTO è la classe base per i DTO che rappresentano i nodi dell'albero decisionale. Fattorizza le proprietà condivise in modo trasparente da tutti i nodi (sia foglie che decisionali).

Attributi

- + parent_id: String | None — identificativo del nodo genitore, necessario per permettere la navigazione a ritroso dell'albero da parte dell'interfaccia utente.

Metodi

NodeBaseDTO non definisce metodi.

5.4) Import

5.4.1) ImportDevice

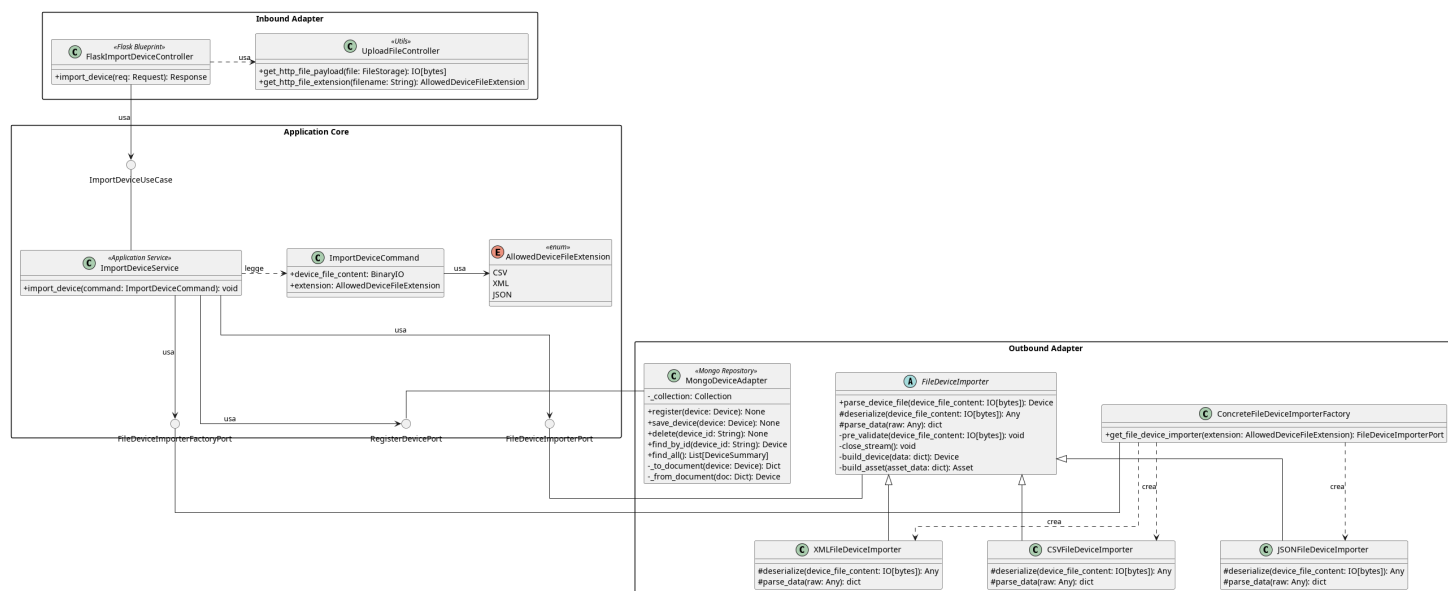


Figura 103: Caso d'uso ImportDevice

Il diagramma illustra l'architettura del modulo dedicato all'importazione di Dispositivi tramite file. Il modulo supporta tre formati — CSV, XML e JSON — gestiti tramite il pattern *Template Method* e una factory dedicata. Il componente *MongoDeviceAdapter* è già descritto nella sezione *CreateDevice*.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *RegisterDevicePort*, vedere la sezione **Sezione 5.2.1.5**

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.4.1.1) UploadFileController



Figura 104: UploadFileController

Descrizione

UploadFileController è una classe di utilità appartenente all’Inbound Adapter che fornisce i metodi comuni per l’estrazione del contenuto e dell’estensione di un file dalla richiesta HTTP. Viene utilizzata da *FlaskImportDeviceController*.

Attributi

UploadFileController non definisce attributi propri.

Metodi

- + `get_http_file_payload(file: FileStorage): IO[bytes]` — estrae il contenuto binario del file dalla richiesta HTTP.
- + `get_http_file_extension(filename: String): AllowedDeviceFileExtension` — estrae l’estensione del file dalla richiesta HTTP.

5.4.1.2) FlaskImportDeviceController



Figura 105: FlaskImportDeviceController

Descrizione

FlaskImportDeviceController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di importazione di Dispositivi tramite file. Utilizza *UploadFileController* per estrarre il contenuto binario e l’estensione del file dalla request.

Attributi

- - `_service: ImportDeviceUseCase` — riferimento alla porta di Inbound (caso d’uso) responsabile della logica applicativa di importazione.

Metodi

- + `import_device(req: Request): Response` — riceve la richiesta HTTP di importazione, estrae il file e la sua estensione tramite *UploadFileController* e inoltra il Command al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.4.1.3) ImportDeviceUseCase

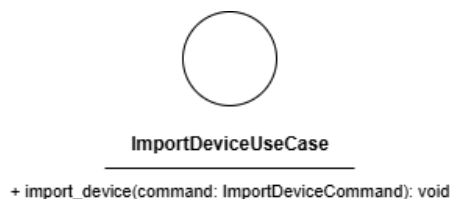


Figura 106: ImportDeviceUseCase

Descrizione

ImportDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'importazione di Dispositivi da file. Viene implementata da *ImportDeviceService* e utilizzata da *FlaskImportDeviceController*.

Attributi

ImportDeviceUseCase non definisce attributi.

Metodi

- `+ import_device(command: ImportDeviceCommand): void` — firma del metodo delegato all'esecuzione della logica di importazione a partire dai dati contenuti nel `Command`.

5.4.1.4) ImportDeviceCommand

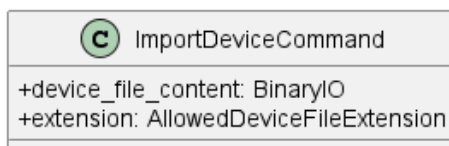


Figura 107: ImportDeviceCommand

Descrizione

ImportDeviceCommand è il Command Object che veicola i dati necessari all'importazione di Dispositivi dal controller al service.

Attributi

- `+ device_file_content: BinaryIO` — contenuto binario del file da importare.
- `+ extension: AllowedDeviceFileExtension` — estensione del file, vincolata ai valori dell'enumerazione *AllowedDeviceFileExtension*.

Metodi

ImportDeviceCommand non definisce metodi propri.

5.4.1.5) AllowedDeviceFileExtension

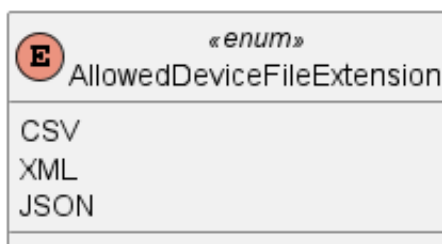


Figura 108: AllowedDeviceFileExtension

Descrizione

AllowedDeviceFileExtension è un'enumerazione che definisce i formati di file supportati per l'importazione di Dispositivi.

Valori

- CSV — formato CSV.
- XML — formato XML.
- JSON — formato JSON.

5.4.1.6) ImportDeviceService



Figura 109: ImportDeviceService

Descrizione

ImportDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di importazione di Dispositivi da file. Implementa l'interfaccia *ImportDeviceUseCase* e coordina il parsing del file tramite *FileDeviceImporterFactoryPort*, la lettura del contenuto tramite *FileDeviceImporterPort* e la persistenza tramite *RegisterDevicePort*.

Attributi

- `device_importer_factory`: *FileDeviceImporterFactoryPort*
- `register_device_port` = *RegisterDevicePort*

Metodi

- `+ import_device(command: ImportDeviceCommand): void` — concretizza il contratto definito da *ImportDeviceUseCase*. Ottiene l'importer appropriato tramite la factory, effettua il parsing del file e persiste i Dispositivi estratti tramite *DeviceRepositoryPort*.

5.4.1.7) FileDeviceImporterPort



Figura 110: FileDeviceImporterPort

Descrizione

FileDeviceImporterPort è l'interfaccia (Outbound Port) che definisce il contratto per il parsing di un file contenente dati di Dispositivi. Viene implementata dalle classi concrete *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* tramite *FileDeviceImporter*.

Attributi

FileDeviceImporterPort non definisce attributi.

Metodi

- + `parse_device_file(device_file_content: BinaryIO): Device` — firma del metodo che effettua il parsing del contenuto binario del file e restituisce l'entità *Device* estratta.

5.4.1.8) FileDeviceImporter

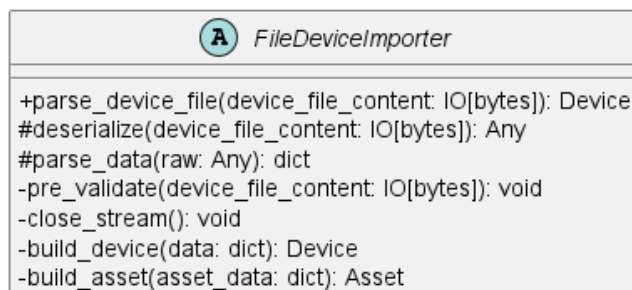


Figura 111: FileDeviceImporter

Descrizione

FileDeviceImporter è la classe astratta appartenente all'Outbound Adapter che implementa il pattern *Template Method* per il parsing dei file di Dispositivi. Definisce lo scheletro dell'algoritmo di importazione, delegando alle sottoclassi concrete l'implementazione dei passi specifici per ciascun formato.

Attributi

FileDeviceImporter non definisce attributi propri.

Metodi

- + `parse_device_file(device_file_content: IO[bytes]): Device` — metodo pubblico che orchestra l'algoritmo di parsing invocando in sequenza i metodi del template; restituisce l'entità *Device* estratta dal file.
- # `deserialize(device_file_content: IO[bytes]): Any` — metodo protetto astratto che deserializza il contenuto binario del file nella struttura dati grezza specifica del formato; deve essere implementato dalle sottoclassi.
- # `parse_data(raw: Any): dict` — metodo protetto astratto che estrae e normalizza i campi necessari dalla struttura dati grezza; deve essere implementato dalle sottoclassi.
- - `pre_validate(device_file_content: IO[bytes]): void` — metodo privato che verifica che il file non superi la dimensione massima consentita di 10 MB prima di procedere al parsing.
- - `close_stream(): void` — metodo privato che chiude lo stream di lettura del file.
- - `build_device(data: dict): Device` — metodo privato che costruisce l'entità *Device* a partire dal dizionario normalizzato, verificando la presenza dei campi obbligatori.
- - `build_asset(asset_data: dict): Asset` — metodo privato che costruisce un'entità *Asset* a partire dai dati del singolo asset estratti dal file.

5.4.1.9) XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

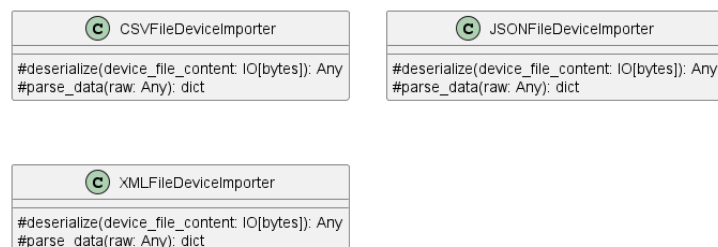


Figura 112: XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

Descrizione

XMLFileDeviceImporter, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* sono le implementazioni concrete di *FileDeviceImporter*, ciascuna specializzata nel parsing del rispettivo formato di file. Implementano i metodi protetti del template per la gestione dello stream e del parsing specifico del formato.

Attributi

Le tre classi non definiscono attributi propri.

Metodi

Ciascuna classe implementa i metodi ereditati da *FileDeviceImporter*.

- `# deserialize(device_file_content: IO[bytes]): Any` — metodo protetto che deserializza il contenuto binario del file nel formato specifico della sottoclasse.
- `# parse_data(raw: Any): dict` — metodo protetto che estrae e normalizza i campi necessari dalla struttura dati grezza prodotta da `deserialize`.

5.4.1.10) FileDeviceImporterFactoryPort



Figura 113: FileDeviceImporterFactoryPort

Descrizione

FileDeviceImporterFactoryPort è l'interfaccia (Outbound Port) che definisce il contratto per l'ottenimento dell'importer appropriato in base all'estensione del file. Viene implementata da *ConcreteFileDeviceImporterFactory*.

Attributi

FileDeviceImporterFactoryPort non definisce attributi.

Metodi

- `+ get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer appropriato per il formato specificato.

5.4.1.11) ConcreteFileDeviceImporterFactory



Figura 114: ConcreteFileDeviceImporterFactory

Descrizione

ConcreteFileDeviceImporterFactory è la classe dell'Outbound Adapter che implementa *FileDeviceImporterFactoryPort*. Istanza e restituisce l'importer appropriato in base all'estensione del file fornita, selezionando tra *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter*.

Attributi

ConcreteFileDeviceImporterFactory non definisce attributi propri.

Metodi

- + `get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer corrispondente al formato specificato.

5.5) Export

5.5.1) ExportDevice

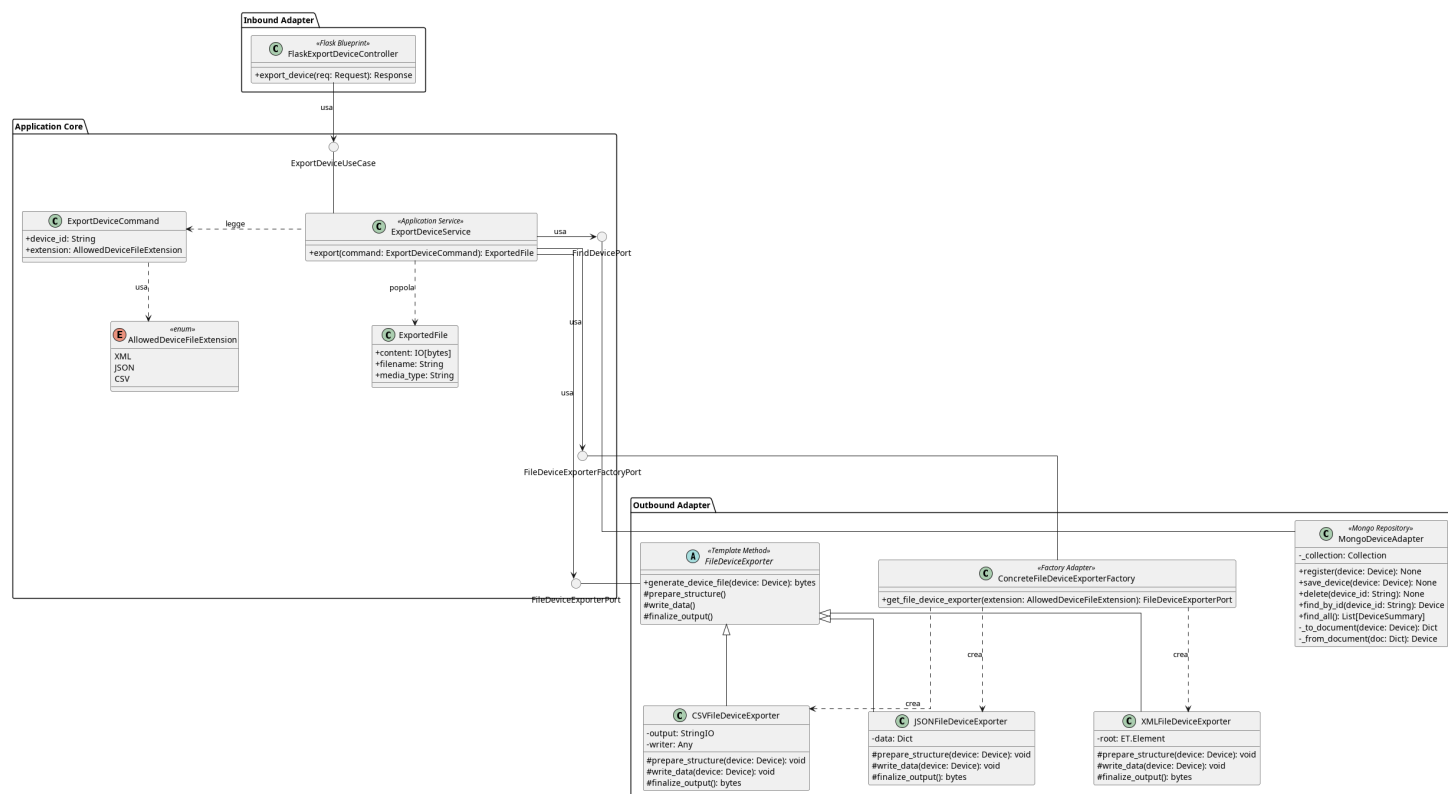


Figura 115: Caso d'uso ExportDevice

Il diagramma illustra l'architettura del modulo dedicato all'esportazione dei dati di un dispositivo (inclusi i suoi asset e le relative valutazioni di conformità) in un file scaricabile. Il sistema supporta tre formati — CSV, XML e JSON — gestiti tramite il pattern *Factory* per la selezione dell'esportatore corretto e il pattern *Template Method* per standardizzare l'algoritmo di generazione del file.

- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.3.5**.
- Per la definizione di *AllowedDeviceFileExtension*, vedere la sezione **Sezione 5.4.1.5**.
- Per la definizione di *ExportedFile*, vedere la sezione **Sezione 5.1.25**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.5.1.1) FlaskExportDeviceController



Figura 116: FlaskExportDeviceController

Descrizione

FlaskExportDeviceController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di esportazione di un dispositivo, estrae l’identificativo del dispositivo e il formato desiderato dalla request e inoltra il Command al livello applicativo.

Attributi

- - `export_device_use_case`: `ExportDeviceUseCase` — inbound port usata per esportare un dispositivo.

Metodi

- + `export_device(req: Request): Response` — riceve la richiesta HTTP, estrae l’identificativo del dispositivo e il formato desiderato come query parameter, invoca il caso d’uso di esportazione e restituisce il file generato come risposta HTTP scaricabile.

5.5.1.2) ExportDeviceUseCase



Figura 117: ExportDeviceUseCase

Descrizione

ExportDeviceUseCase è l’interfaccia (Inbound Port) che definisce il contratto per l’esportazione dei dati di un dispositivo in un file. Viene implementata da *ExportDeviceService* e utilizzata da *FlaskExportDeviceController*.

Attributi

ExportDeviceUseCase non definisce attributi.

Metodi

- + `export_device(command: ExportDeviceCommand): ExportedFile` — firma del metodo delegato all’esecuzione della logica di esportazione a partire dai dati contenuti nel Command.

5.5.1.3) ExportDeviceCommand

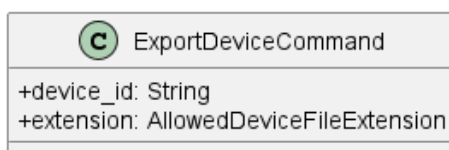


Figura 118: ExportDeviceCommand

Descrizione

ExportDeviceCommand è il Command Object che veicola i parametri della richiesta di esportazione dal controller al service.

Attributi

- + `device_id`: String — identificativo univoco del dispositivo da esportare.
- + `extension`: AllowedDeviceFileExtension — formato del file richiesto per l'esportazione, vincolato ai valori dell'enumerazione *AllowedDeviceFileExtension*.

Metodi

ExportDeviceCommand non definisce metodi propri.

5.5.1.4) ExportDeviceService



Figura 119: ExportDeviceService

Descrizione

ExportDeviceService è il service applicativo appartenente all'Application Core responsabile di orchestrare il processo di esportazione. Implementa l'interfaccia *ExportDeviceUseCase*, recupera il dispositivo tramite *FindDevicePort*, ottiene l'esportatore appropriato tramite *FileDeviceExporterFactoryPort* e avvia la generazione del file tramite *FileDeviceExporterPort*; restituisce il risultato incapsulato in un oggetto *ExportedFile*.

Attributi

- - `find_device`: FindDevicePort — outbound port usata per prelevare il dispositivo.
- - `exporter_factory`: FileDeviceExporterFactoryPort — outbound port usata per ottenere l'exporter desiderato.

Metodi

- + `export_device(command: ExportDeviceCommand): ExportedFile` — concretizza il contratto definito da *ExportDeviceUseCase*. Recupera il dispositivo, ottiene l'esportatore corretto tramite la factory e restituisce il file generato incapsulato in *ExportedFile*.

5.5.1.5) FileDeviceExporterFactoryPort



Figura 120: FileDeviceExporterFactoryPort

Descrizione

FileDeviceExporterFactoryPort è l'interfaccia (Outbound Port) che definisce il contratto per l'ottenimento dell'esportatore appropriato in base al formato richiesto. Viene implementata da *ConcreteFileDeviceExporterFactory* e utilizzata da *ExportDeviceService*.

Attributi

FileDeviceExporterFactoryPort non definisce attributi.

Metodi

- `+ get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort` — restituisce l'istanza dell'esportatore corrispondente al formato specificato.

5.5.1.6) FileDeviceExporterPort

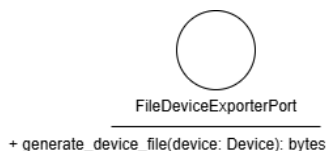


Figura 121: FileDeviceExporterPort

Descrizione

FileDeviceExporterPort è l'interfaccia (Outbound Port) che definisce il contratto per la generazione del file di esportazione. Viene implementata da *FileDeviceExporter* e utilizzata da *ExportDeviceService*.

Attributi

FileDeviceExporterPort non definisce attributi.

Metodi

- `+ generate_device_file(device: Device): bytes` — firma del metodo delegato alla generazione del file; riceve l'entità *Device* e restituisce il file generato come bytes.

5.5.1.7) FileDeviceExporter

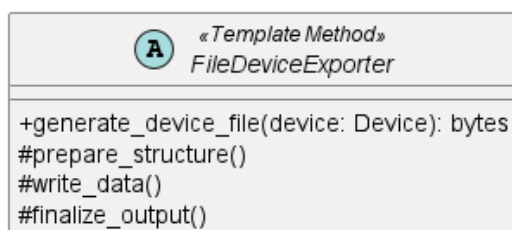


Figura 122: FileDeviceExporter

Descrizione

FileDeviceExporter è la classe astratta appartenente all'Outbound Adapter che implementa *FileDeviceExporterPort* definendo lo scheletro dell'algoritmo di esportazione tramite il pattern *Template Method*. Le sottoclassi concrete implementano i passi specifici per ciascun formato.

Attributi

FileDeviceExporter non definisce attributi propri.

Metodi

- + `generate_device_file(device: Device): bytes` — metodo pubblico che orchestra l'algoritmo di esportazione invocando in sequenza i metodi protetti del template.
- # `prepare_structure(device: Device): void` — metodo protetto astratto che inizializza la struttura del documento nel formato specifico.
- # `write_data(device: Device): void` — metodo protetto astratto che scrive i dati del dispositivo nella struttura inizializzata.
- # `finalize_output(): bytes` — metodo protetto astratto che chiude il documento e restituisce il contenuto serializzato come array di byte.

5.5.1.8) ConcreteFileDeviceExporterFactory



Figura 123: ConcreteFileDeviceExporterFactory

Descrizione

ConcreteFileDeviceExporterFactory è la classe dell'Outbound Adapter che implementa *FileDeviceExporterFactoryPort*. Istanza e restituisce l'esportatore appropriato in base al formato richiesto, selezionando tra *CSVFileDeviceExporter*, *JSONFileDeviceExporter* e *XMLFileDeviceExporter*.

Attributi

- - exporters: Dict[AllowedDeviceFileExtension, FileDeviceExporterPort]
— dizionario che mappa ogni estensione supportata alla corrispondente istanza dell'esportatore.

Metodi

- + get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort — restituisce l'istanza dell'esportatore corrispondente al formato specificato.

5.5.1.9) CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter

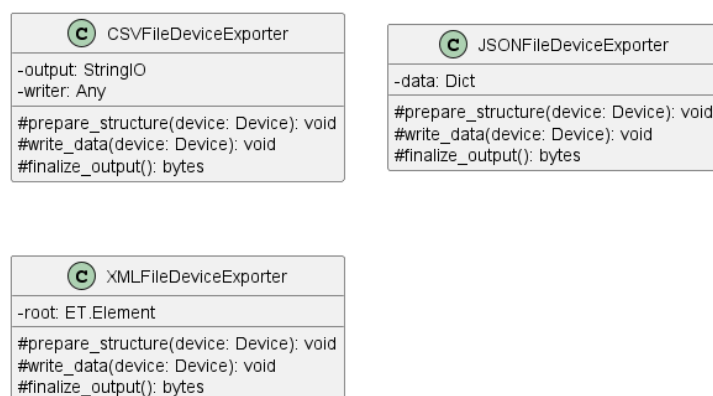


Figura 124: CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter

Descrizione

CSVFileDeviceExporter, *JSONFileDeviceExporter* e *XMLFileDeviceExporter* sono le implementazioni concrete di *FileDeviceExporter*, ciascuna specializzata nella generazione del file nel rispettivo formato. Implementano i metodi protetti del template per la gestione della struttura e della serializzazione specifica del formato.

Attributi

- CSVFileDeviceExporter: - output: StringIO, - writer: Any — output è lo stream in memoria su cui viene scritto il contenuto CSV; writer è il csv.writer che scrive le righe su output.
- JSONFileDeviceExporter: - data: Dict — dizionario che accumula incrementalmente la struttura JSON del dispositivo durante write_data e viene serializzato in finalize_output.
- XMLFileDeviceExporter: - root: ET.Element — nodo radice dell'albero XML costruito durante write_data e serializzato in finalize_output.

Metodi

- # prepare_structure(device: Device): void — inizializza la struttura del documento nel formato specifico.

- `# write_data(device: Device): void` — scrive i dati del dispositivo nella struttura inizializzata.
- `# finalize_output(): bytes` — serializza il documento e restituisce il contenuto come array di byte.

5.5.2) ExportReport

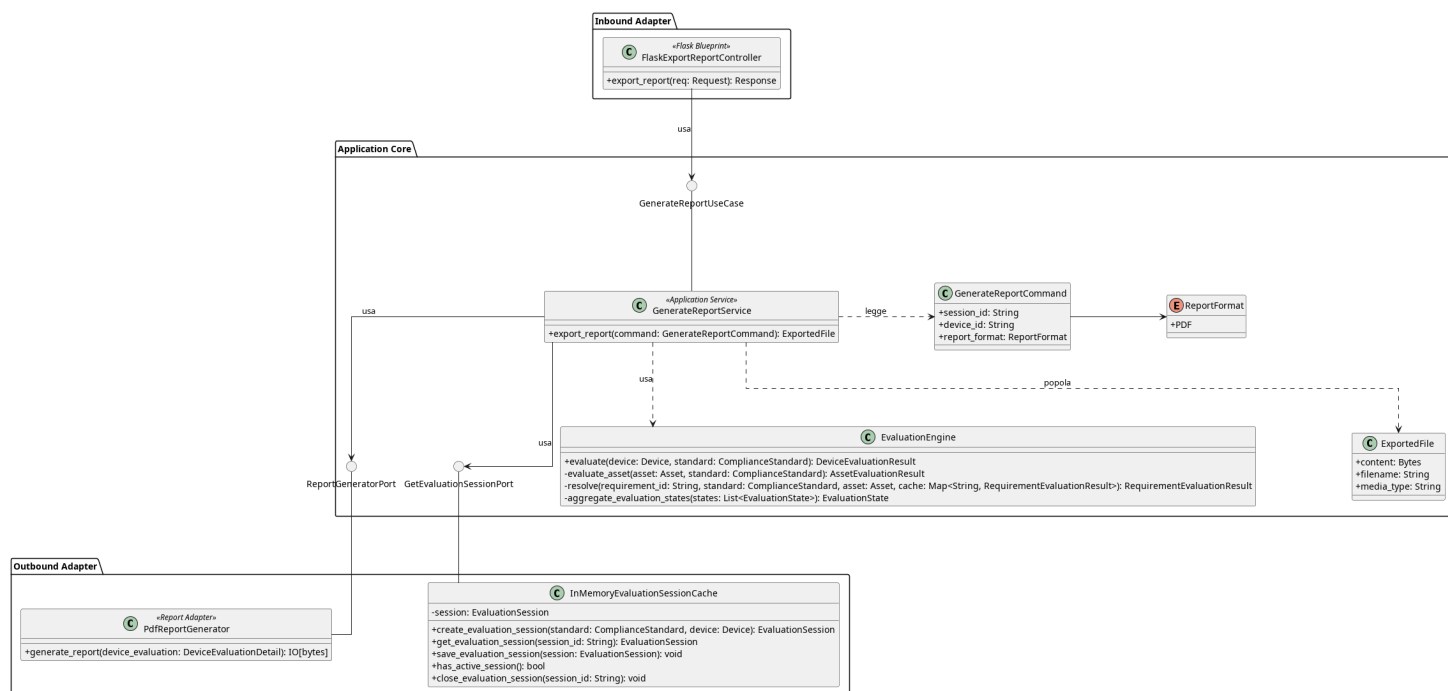


Figura 125: ExportReport

Il diagramma illustra l'architettura del modulo dedicato alla generazione e all'esportazione dei report riassuntivi di una valutazione. Il flusso permette di recuperare i dati di una sessione, valutare il dispositivo tramite *EvaluationEngine* e compilare dinamicamente un documento PDF da restituire all'utente tramite download.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *ExportedFile*, vedere la sezione **Sezione 5.1.25**.
- Per la definizione di *EvaluationEngine*, vedere la sezione **Sezione 5.1.14**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.5.2.1) FlaskExportReportController



Figura 126: ExportReportController

Descrizione

ExportReportController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP per la generazione e l'esportazione del report di una sessione di valutazione. Estrae dalla request i parametri necessari, costruisce il Command e inoltra la richiesta al livello applicativo.

Attributi

- `generate_report_use_case`: `GenerateReportUseCase` — porta di inbound usata per generare un Report.

Metodi

- `+ export_report(session_id: String, device_id: String, fmt: String): Response` — riceve la richiesta HTTP, valida il formato richiesto, costruisce il *GenerateReportCommand* e invoca il caso d'uso; restituisce il file generato come risposta HTTP scaricabile.

5.5.2.2) GenerateReportUseCase



Figura 127: GenerateReportUseCase

Descrizione

GenerateReportUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la generazione e l'esportazione di un report. Viene implementata da *GenerateReportService* e utilizzata da *ExportReportController*.

Attributi

GenerateReportUseCase non definisce attributi.

Metodi

- `+ export_report(command: GenerateReportCommand): ExportedFile` — firma del metodo delegato all'esecuzione della logica di generazione del report a partire dai dati contenuti nel Command; restituisce un oggetto *ExportedFile* contenente il file generato, il nome del file e il tipo MIME necessari alla costruzione della risposta HTTP.

5.5.2.3) GenerateReportCommand

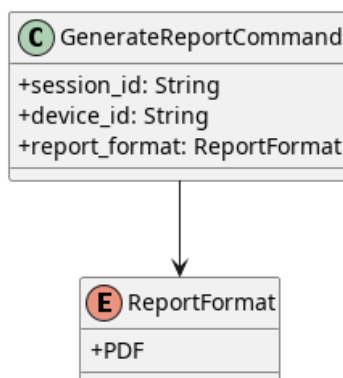


Figura 128: GenerateReportCommand

Descrizione

GenerateReportCommand è il Command Object che veicola i parametri necessari alla generazione del report dal controller al service.

Attributi

- `+ session_id: String` — identificativo univoco della sessione di valutazione da cui generare il report.
- `+ device_id: String` — identificativo univoco del dispositivo oggetto del report.
- `+ report_format: ReportFormat` — formato desiderato per il report in uscita, vincolato ai valori dell'enumerazione *ReportFormat*.

Metodi

GenerateReportCommand non definisce metodi propri.

5.5.2.4) ReportFormat

Descrizione

ReportFormat è un'enumerazione che definisce i formati di report supportati dal sistema.

Valori

- PDF — formato PDF.

5.5.2.5) GenerateReportService



Figura 129: GenerateReportService

Descrizione

GenerateReportService è il service applicativo appartenente all'Application Core responsabile dell'orchestrazione del processo di generazione del report. Implementa l'interfaccia *GenerateReportUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, valuta il dispositivo tramite *EvaluationEngine*, costruisce il dettaglio della valutazione e delega la generazione materiale del documento a *ReportGeneratorPort*; restituisce il risultato incapsulato in un oggetto *ExportedFile*. Internamente utilizza i metodi privati *_build_device_detail*, *_build_asset_detail* e *_build_requirement_detail* per costruire la struttura gerarchica dei dettagli di valutazione.

Attributi

- *get_evaluation_session_port*: *GetEvaluationSessionPort* — outbound port usata per prelevare la sessione di valutazione.
- *report_generator_port*: *ReportGeneratorPort* — outbound port usata per generare il Report.
- *evaluation_engine*: *EvaluationEngine* — oggetto di dominio usato per calcolare il *DeviceResult* i cui dati saranno immessi nel Report.

Metodi

- + *export_report*(command: *GenerateReportCommand*): *ExportedFile* — concretizza il contratto definito da *GenerateReportUseCase*. Recupera la sessione, valuta il dispositivo tramite *EvaluationEngine*, costruisce il dettaglio della valutazione e restituisce il file generato incapsulato in *ExportedFile*.

5.5.2.6) ReportGeneratorPort

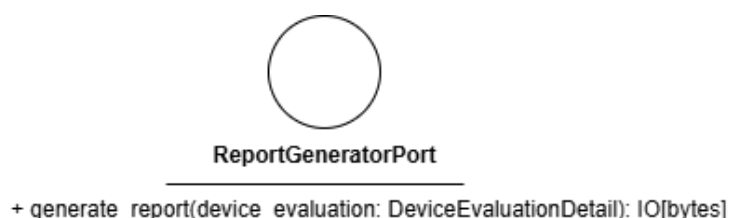


Figura 130: ReportGeneratorPort

Descrizione

ReportGeneratorPort è l'interfaccia (Outbound Port) che definisce il contratto per la generazione materiale del report a partire dal dettaglio della valutazione. Viene implementata da *PdfReportGenerator* e utilizzata da *GenerateReportService*.

Attributi

ReportGeneratorPort non definisce attributi.

Metodi

- + generate_report(device_evaluation: DeviceEvaluationDetail): IO[bytes]
— firma del metodo delegato alla generazione del file del report a partire dal dettaglio della valutazione del dispositivo.

5.5.2.7) PdfReportGenerator



Figura 131: PdfReportGenerator

Descrizione

PdfReportGenerator è l'Outbound Adapter che implementa *ReportGeneratorPort*. Si occupa della formattazione e della generazione materiale del file del report in formato PDF a partire dal dettaglio della valutazione. Internamente utilizza metodi privati per la formattazione dell'intestazione, del corpo e del piè di pagina del documento.

Attributi

PdfReportGenerator non definisce attributi propri.

Metodi

- + generate_report(device_evaluation: DeviceEvaluationDetail): IO[bytes]
— implementa il metodo dell'interfaccia, avviando il processo di creazione del PDF e restituendo lo stream di byte generato

5.6) Session

5.6.1) OpenEvaluationSession

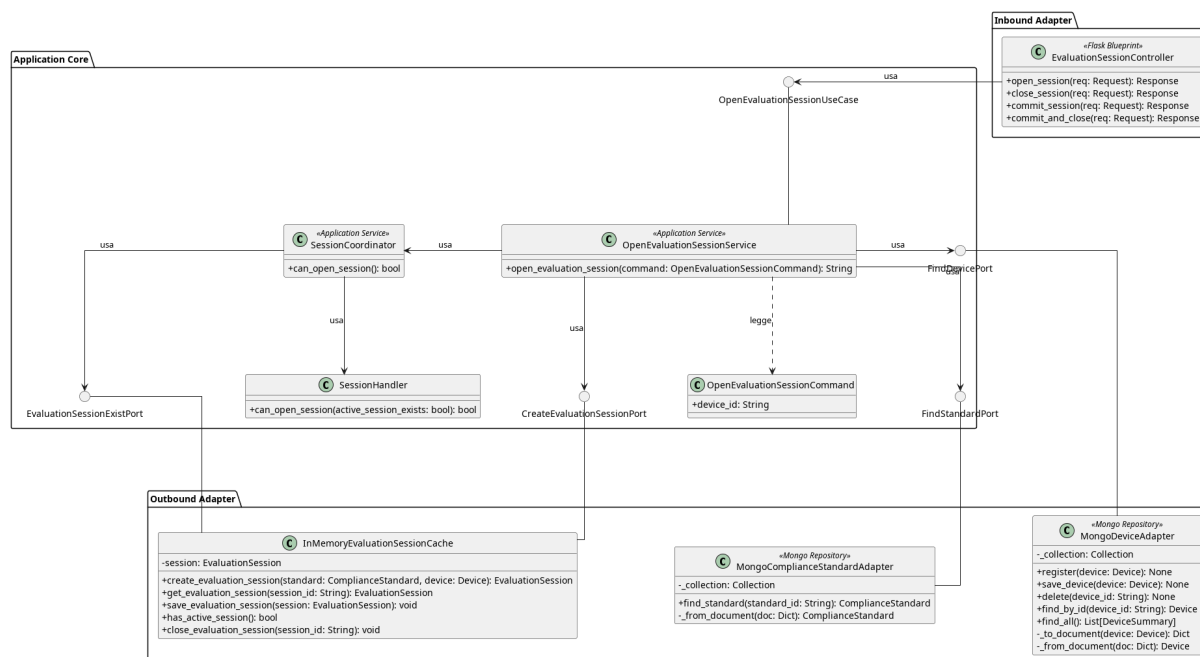


Figura 132: Caso d'uso OpenEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato all'apertura di una sessione di valutazione.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.3.5**.
- Per la definizione di *MongoStandardAdapter*, vedere la sezione **Sezione 5.2.4.9**.
- Per la definizione di *FindStandardPort*, vedere la sezione **Sezione 5.2.4.8**.
- Per la definizione di *SessionHandler*, vedere la sezione **Sezione 5.1.24**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.6.1.1) EvaluationSessionController

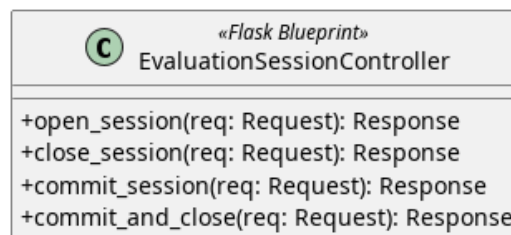


Figura 133: EvaluationSessionController

Descrizione

EvaluationSessionController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione del ciclo di vita della sessione di valutazione e le inoltra al livello applicativo.

Attributi

- - `open_use_case`: `OpenEvaluationSessionUseCase` — inbound port usata per aprire la sessione
- - `close_use_case`: `CloseEvaluationSessionUseCase` — inbound port usata per chiudere la sessione
- - `commit_use_case`: `CommitEvaluationSessionUseCase` — inbound port usata per salvare la sessione in memoria

Metodi

- + `open_session(req: Request): Response` — riceve la richiesta HTTP di apertura di una nuova sessione di valutazione e restituisce una risposta HTTP con l'identificativo della sessione creata.
- + `close_session(req: Request): Response` — riceve la richiesta HTTP di chiusura della sessione corrente e restituisce una risposta HTTP con l'esito dell'operazione.
- + `commit_session(req: Request): Response` — riceve la richiesta HTTP di commit della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- + `commit_and_close(req: Request): Response` — riceve la richiesta HTTP di commit e chiusura contestuale della sessione e restituisce una risposta HTTP con l'esito dell'operazione.

5.6.1.2) OpenEvaluationSessionCommand

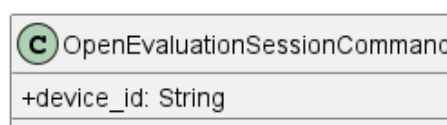


Figura 134: OpenEvaluationSessionCommand

Descrizione

OpenEvaluationSessionCommand è il Command Object utilizzato per trasportare i dati necessari all'apertura di una nuova sessione di valutazione. Incapsula i parametri di input del metodo esposto da *OpenEvaluationSessionUseCase*.

Attributi

- + device_id: String — identificativo univoco del Dispositivo per cui aprire la sessione.

Metodi

OpenEvaluationSessionCommand non definisce metodi propri.

5.6.1.3) OpenEvaluationSessionUseCase



Figura 135: OpenEvaluationSessionUseCase

Descrizione

OpenEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'apertura di una nuova sessione di valutazione. Viene implementata da *OpenEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

OpenEvaluationSessionUseCase non definisce attributi.

Metodi

- + open_evaluation_session(command: OpenEvaluationSessionCommand): String — firma del metodo delegato all'esecuzione della logica di apertura della sessione a partire dai dati contenuti nel Command.

5.6.1.4) OpenEvaluationSessionService

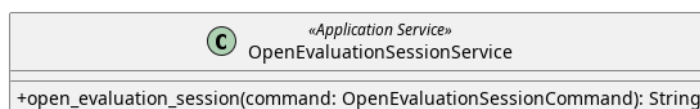


Figura 136: OpenEvaluationSessionService

Descrizione

OpenEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di apertura di una sessione di valutazione. Implementa l'interfaccia *OpenEvaluationSessionUseCase* e coordina il recupero del Dispositivo tramite *FindDevicePort*, il recupero dello standard di conformità tramite *FindStandardPort* e la creazione della sessione tramite *CreateEvaluationSessionPort*.

Attributi

- - `session_coordinator`: `SessionCoordinator` — classe allo stato applicativo che centralizza le regole di business per autorizzare l'apertura di una nuova sessione
- - `create_session_port`: `CreateSessionPort` — outbound port usata per creare una sessione
- - `find_device_port`: `FindDevicePort` — outbound port usata per trovare il dispositivo
- - `find_standard_port`: `FindStandardPort` — outbound port usata per trovare lo Standard

Metodi

- + `open_evaluation_session(command: OpenEvaluationSessionCommand)`: `String` — concretizza il contratto definito da *OpenEvaluationSessionUseCase*. Recupera il Dispositivo e lo standard associato, inizializza una nuova *EvaluationSession* e ne richiede la creazione tramite *CreateEvaluationSessionPort*; restituisce l'identificativo della sessione creata.

5.6.1.5) SessionCoordinator



Figura 137: SessionCoordinator

Descrizione

SessionCoordinator è il Service che coordina la logica di dominio relativa alla gestione della sessione di valutazione. Verifica le precondizioni necessarie all'apertura di una sessione, come l'assenza di sessioni attive per il Dispositivo.

Attributi

- - `exist_port`: `EvaluationSessionExistPort` — Outbound port per verificare l'esistenza di sessioni attive nel sistema.
- - `session_handler`: `SessionHandler` — Componente di dominio che incapsula le regole di business per l'apertura delle sessioni.

Metodi

- + `can_open_session(session_type: SessionType)`: `bool` — verifica se è possibile aprire una nuova sessione del tipo specificato, restituendo `true` se le precondizioni sono soddisfatte.

Metodi

- + `can_open_session(active_session_exists: bool): bool` — verifica se è possibile avviare una nuova sessione, restituendo `false` nel caso in cui esista già una sessione attiva, altrimenti `true`.

5.6.1.6) EvaluationSessionExistPort

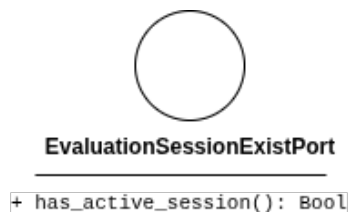


Figura 138: EvaluationSessionExistPort

Descrizione

EvaluationSessionExistPort è l'interfaccia (Outbound Port) che definisce il contratto per verificare la presenza di eventuali sessioni di valutazione attualmente attive all'interno del sistema.

Attributi

La classe *EvaluationSessionExistPort* non definisce attributi.

Metodi

- + `has_active_session(): bool` — interroga il sistema per determinare se esiste già una sessione attiva, restituendo il risultato come valore booleano.

5.6.1.7) CreateEvaluationSessionPort

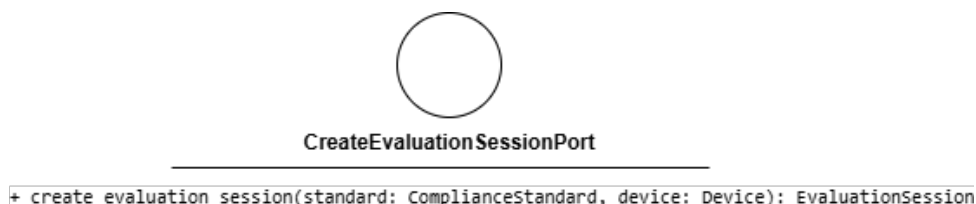


Figura 139: CreateEvaluationSessionPort

Descrizione

CreateSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per la creazione di una nuova sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *OpenEvaluationSessionService*.

Attributi

CreateSessionPort non definisce attributi.

Metodi

- + create_session(): EvaluationSession — firma del metodo che inizializza e registra una nuova sessione di valutazione nel sistema in memoria.

5.6.2) CommitEvaluationSession

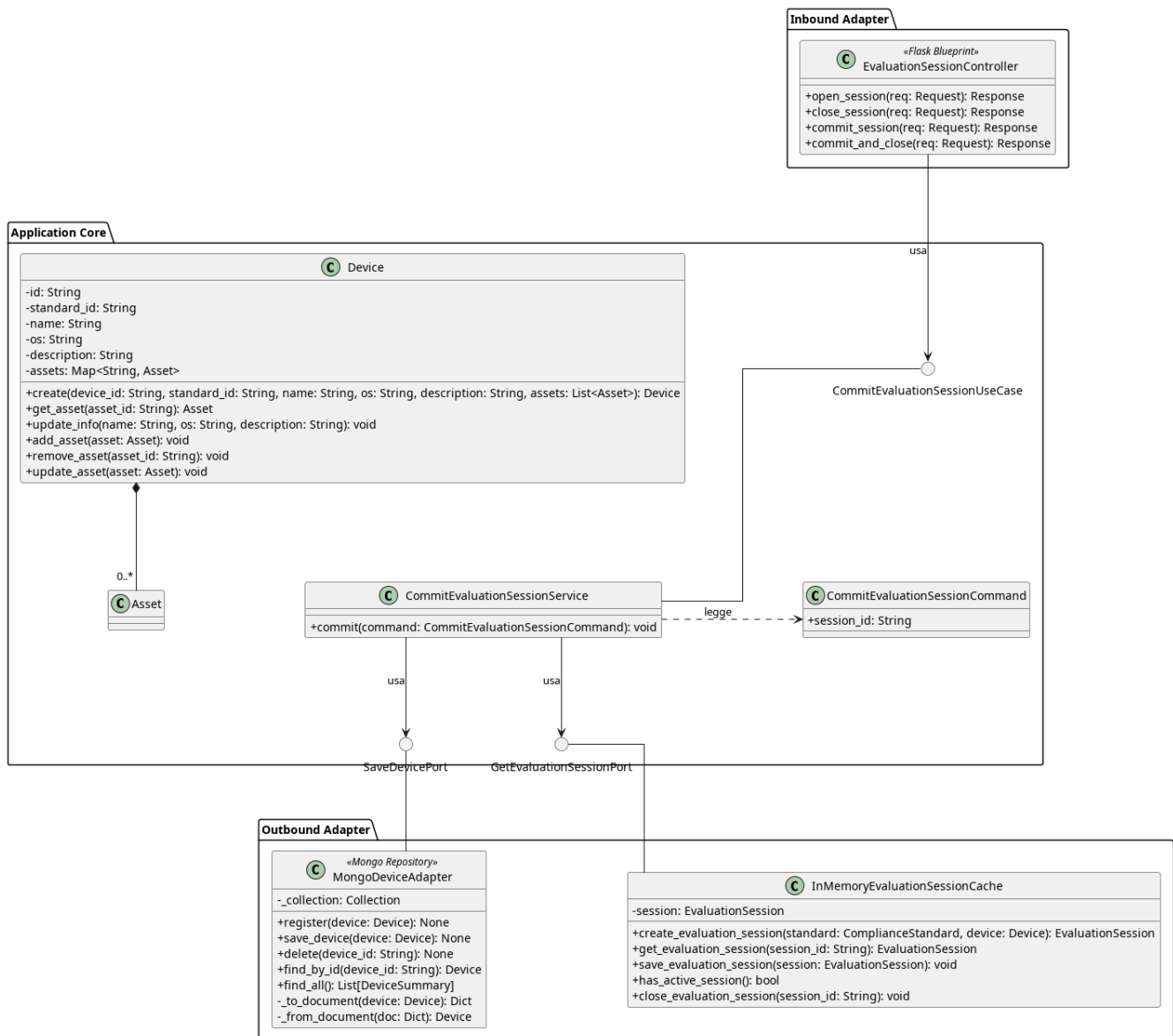


Figura 140: Caso d'uso CommitEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato esclusivamente al consolidamento (commit) dei dati di una sessione di valutazione verso il dispositivo, senza richiederne la chiusura o l'eliminazione dalla memoria temporanea.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.1.6**.
- Per la definizione di *SaveDevicePort*, vedere la sezione **Sezione 5.2.3.4**.

- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti specifici introdotti per questo flusso operativo.

5.6.2.1) CommitEvaluationSessionUseCase



Figura 141: CommitEvaluationSessionUseCase

Descrizione

CommitEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per eseguire il consolidamento (commit) dei dati di una sessione di valutazione attiva, applicando definitivamente le modifiche all'entità dispositivo associata. Viene implementata a livello applicativo dal *CommitEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CommitEvaluationSessionUseCase non definisce attributi.

Metodi

- + commit(command: CommitEvaluationSessionCommand): void — firma del metodo delegato all'esecuzione della logica di consolidamento dei dati della sessione di valutazione a partire dal comando ricevuto in input.

5.6.2.2) CommitEvaluationSessionService

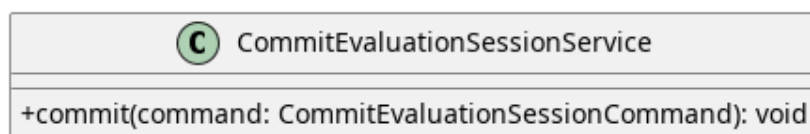


Figura 142: CommitEvaluationSessionService

CommitEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare l'operazione di commit. Implementa l'interfaccia *CommitEvaluationSessionUseCase*. Coordina il recupero della sessione attualmente in corso (tramite la porta *GetEvaluationSessionPort*), legge i dati necessari dal *CommitEvaluationSessionCommand* e applica le modifiche definitive delegando il salvataggio all'entità dispositivo (tramite *SaveDevicePort*).

Attributi

- `save_device_port`: `SaveDevicePort` — outbound port usata per salvare il dispositivo in memoria
- `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per prelevare la *EvaluationSession*

Metodi

- `+ commit(command: CommitEvaluationSessionCommand): void` — concretizza la logica di business relativa al consolidamento dei dati. Utilizza i parametri incapsulati nel comando per applicare le modifiche allo stato persistente del dispositivo.

5.6.2.3) CommitEvaluationSessionCommand

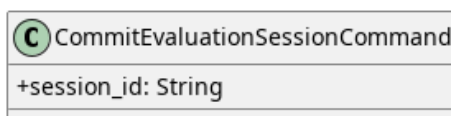


Figura 143: CommitEvaluationSessionCommand

Descrizione

CommitEvaluationSessionCommand incapsula i parametri necessari per richiedere il consolidamento (commit) di una sessione di valutazione. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- `+ session_id: String` — l'identificativo univoco della sessione di valutazione di cui si richiede il consolidamento dei dati.

Metodi

CommitEvaluationSessionCommand non definisce metodi.

5.6.3) CloseEvaluationSession

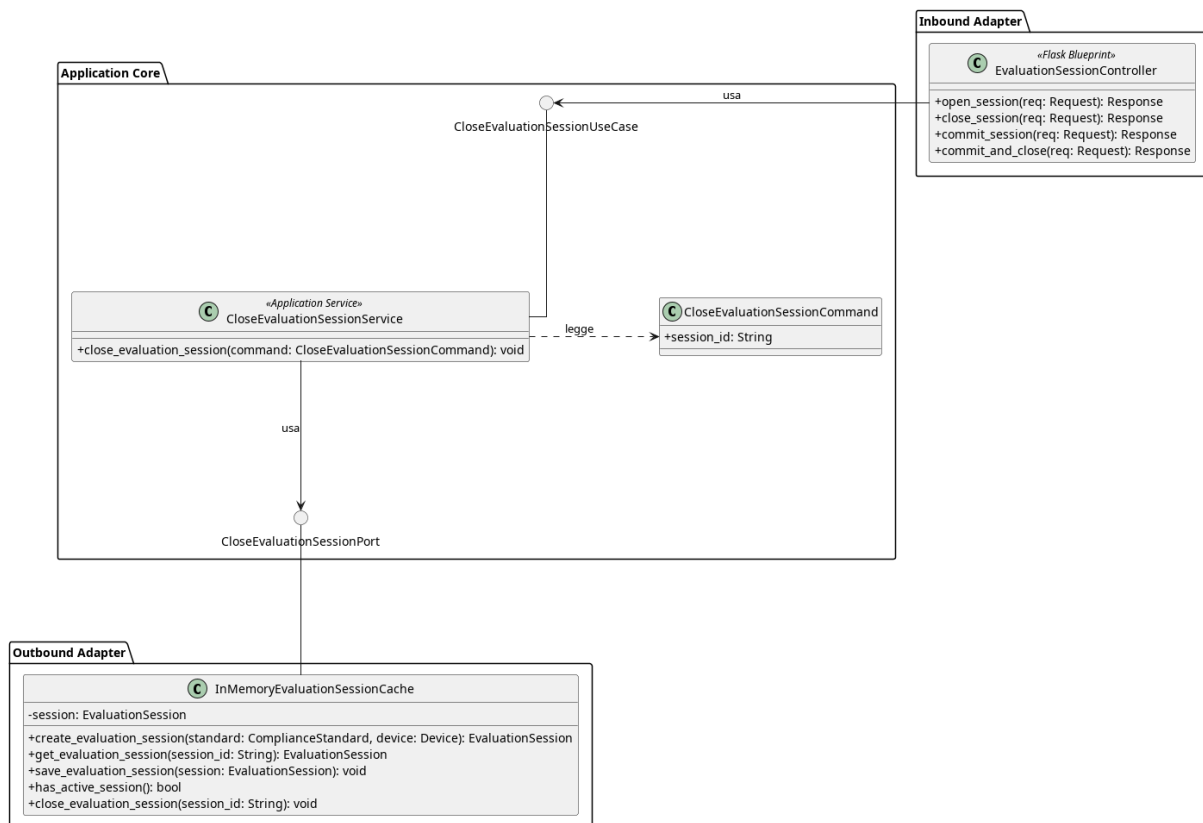


Figura 144: Caso d'uso CloseEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato alla chiusura e all'eliminazione di una sessione di valutazione.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.6.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.6.3.1) CloseEvaluationSessionUseCase

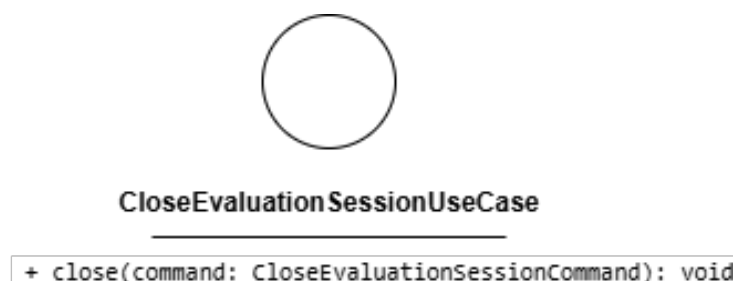


Figura 145: CloseEvaluationSessionUseCase

Descrizione

CloseEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la chiusura definitiva di una sessione di valutazione attiva. Rappresenta l'operazione conclusiva del ciclo di vita della sessione e viene implementata da *CloseEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CloseEvaluationSessionUseCase non definisce attributi.

Metodi

- + close(command: CloseEvaluationSessionCommand): void — firma del metodo delegato all'esecuzione della logica di chiusura della sessione a partire dal comando ricevuto in input.

5.6.3.2) CloseEvaluationSessionService



Figura 146: CloseEvaluationSessionService

Descrizione

CloseEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di chiusura della sessione. Implementa l'interfaccia *CloseEvaluationSessionUseCase*. Una volta eseguite le operazioni di dominio necessarie, richiede la rimozione della sessione dal sistema di persistenza richiamando la porta in uscita *DeleteSessionPort*.

Attributi

- - delete_session_port: DeleteSessionPort — outbound port usata per eliminare la sessione.

Metodi

- + close(command: CloseEvaluationSessionCommand): void — concretizza il contratto definito da *CloseEvaluationSessionUseCase*. Coordina le operazioni di chiusura e inoltra la richiesta di eliminazione della sessione utilizzando i parametri incapsulati nel comando.

5.6.3.3) CloseEvaluationSessionCommand

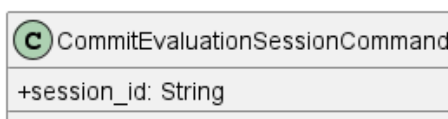


Figura 147: CloseEvaluationSessionCommand

Descrizione

CloseEvaluationSessionCommand incapsula i parametri necessari per richiedere la chiusura di una sessione di valutazione. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- + session_id: String — l'identificativo univoco della sessione di valutazione di cui si richiede la chiusura e l'eliminazione.

Metodi

CloseEvaluationSessionCommand non definisce metodi.

5.6.3.4) DeleteSessionPort

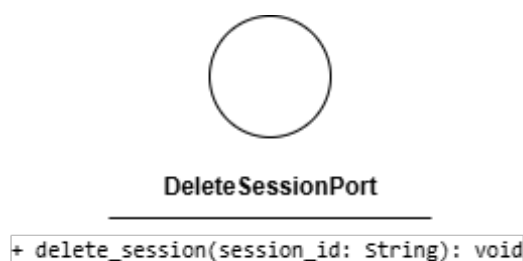


Figura 148: DeleteSessionPort

Descrizione

DeleteSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per l'eliminazione di una sessione di valutazione dal sistema di archiviazione (es. la cache in memoria). Viene utilizzata da *CloseEvaluationSessionService* e implementata nel livello di adapter da *InMemoryEvaluationSessionCache*.

Attributi

DeleteSessionPort non definisce attributi.

Metodi

- + delete_session(session_id: String): void — firma del metodo che si occupa di rimuovere o invalidare lo stato di una specifica sessione di valutazione dal sistema di persistenza.

5.7) Area navigazione

5.7.1) EvaluateDecisionNode

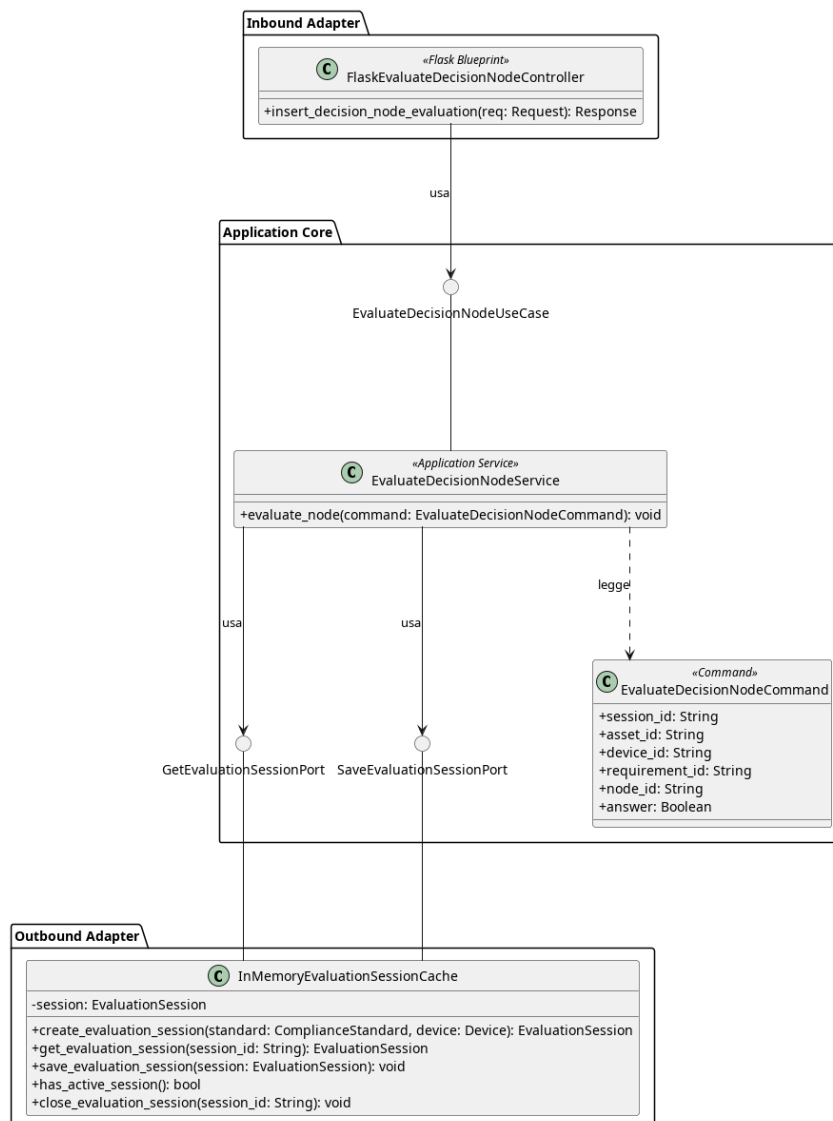


Figura 149: Caso d'uso EvaluateDecisionNode

Il diagramma illustra l'architettura del modulo dedicato alla valutazione di un nodo decisionale durante la verifica di conformità.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *SaveEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.6**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.7.1.1) EvaluateDecisionNodeController

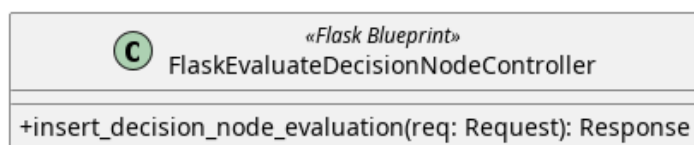


Figura 150: EvaluateDecisionNodeController

Descrizione

EvaluateDecisionNodeController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP per la valutazione di un nodo decisionale e le inoltra al livello applicativo.

Attributi

- - `evaluate_decision_node_use_case`: *EvaluateDecisionNodeUseCase* — out-bound port usata per valutare un nodo

Metodi

- + `insert_decision_node_evaluation(req: Request): Response` — riceve la richiesta HTTP, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.7.1.2) EvaluateDecisionNodeUseCase



Figura 151: EvaluateDecisionNodeUseCase

Descrizione

EvaluateDecisionNodeUseCase è l’interfaccia (Inbound Port) che definisce il contratto per la valutazione di un nodo decisionale. Viene implementata da *EvaluateDecisionNodeService* e utilizzata da *FlaskEvaluateDecisionNodeController*.

Attributi

EvaluateDecisionNodeUseCase non definisce attributi.

Metodi

- + `evaluate_node(command: EvaluateDecisionNodeCommand): void` — firma del metodo delegato all’esecuzione della logica di valutazione a partire dai dati contenuti nel comando.

5.7.1.3) EvaluateDecisionNodeCommand

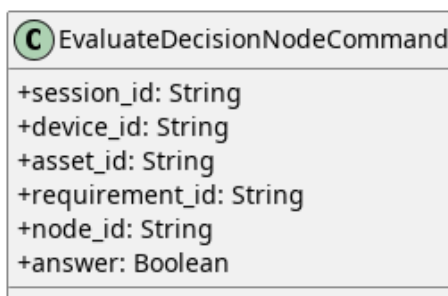


Figura 152: EvaluateDecisionNodeCommand

Descrizione

EvaluateDecisionNodeCommand è il Command Object che veicola i dati necessari alla valutazione di un nodo decisionale dal controller al service.

Attributi

- `+ session_id: String` — identificativo della sessione di valutazione attiva.
- `+ device_id: String` — identificativo del dispositivo oggetto di valutazione.
- `+ asset_id: String` — identificativo dell'Asset oggetto di valutazione.
- `+ requirement_id: String` — identificativo del requisito in fase di valutazione.
- `+ node_id: String` — identificativo del nodo decisionale.
- `+ answer: Boolean` — valore della risposta fornita per il nodo decisionale.

Metodi

EvaluateDecisionNodeCommand non definisce metodi propri.

5.7.1.4) EvaluateDecisionNodeService

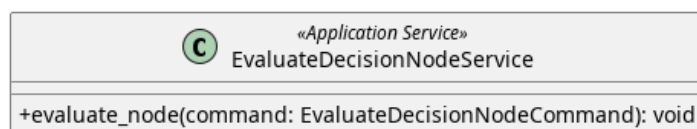


Figura 153: EvaluateDecisionNodeService

Descrizione

EvaluateDecisionNodeService è il service applicativo appartenente all'Application Core responsabile della logica di valutazione di un nodo decisionale. Implementa l'interfaccia *EvaluateDecisionNodeUseCase*. Recupera la sessione attiva tramite *GetEvaluationSessionPort*, individua l'Asset corrispondente all'interno del dispositivo, registra la risposta sul nodo decisionale e persiste la sessione aggiornata tramite *SaveEvaluationSessionPort*. In caso di sessione non trovata, asset non trovato o errore di salvataggio, propaga un'eccezione di tipo *EvaluateNodeFailure*.

Attributi

- `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per recuperare la sessione di valutazione.
- `save_evaluation_session_port`: `SaveEvaluationSessionPort` — outbound port usata per salvare le modifiche applicate alla sessione.

Metodi

- `+ evaluate_node(command: EvaluateDecisionNodeCommand): void` — concretizza il contratto definito da *EvaluateDecisionNodeUseCase*. Recupera la sessione attiva, individua l'Asset nel dispositivo, registra la risposta al nodo decisionale e persiste la sessione aggiornata.

5.7.2) InsertJustification

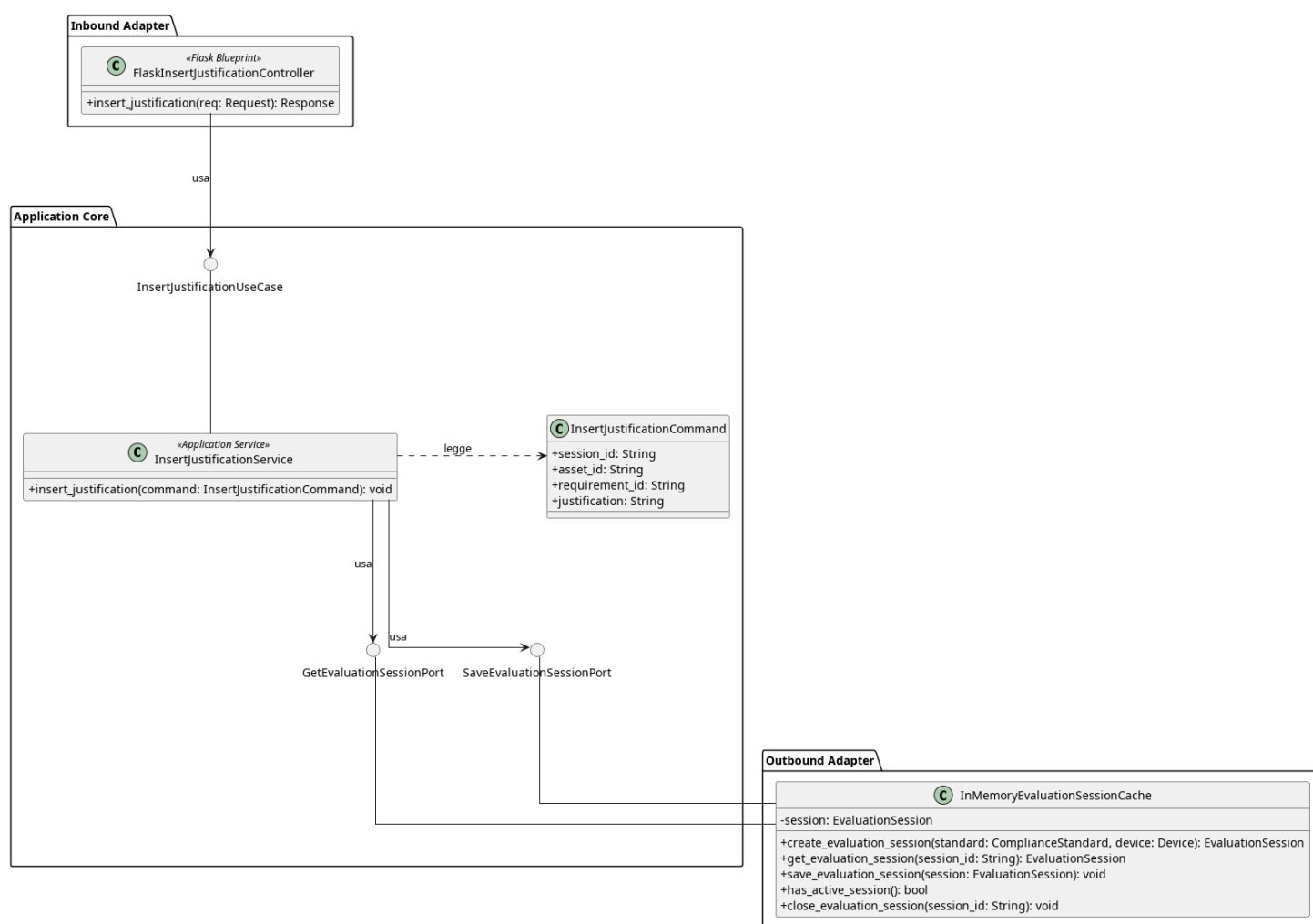


Figura 154: Caso d'uso InsertJustification

Il diagramma illustra l'architettura del modulo dedicato all'inserimento di una giustificazione testuale per un requisito di conformità durante la sessione di valutazione.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.7**.

- Per la definizione di *SaveEvaluationSessionPort*, vedere la sezione **Sezione 5.3.1.6**.

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso.

5.7.2.1) FlaskInsertJustificationController

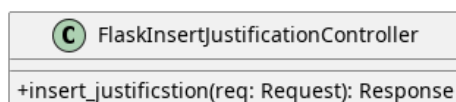


Figura 155: FlaskInsertJustificationController

Descrizione

FlaskInsertJustificationController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di inserimento di una giustificazione per un requisito e le inoltra al livello applicativo.

Attributi

- `insert_justification_use_case`: `InsertJustificationUseCase` — inbound port usata per inserire la giustificazione.

Metodi

- `+ insert_justification(req: Request): Response` — riceve la richiesta HTTP di inserimento della giustificazione, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.7.2.2) InsertJustificationUseCase

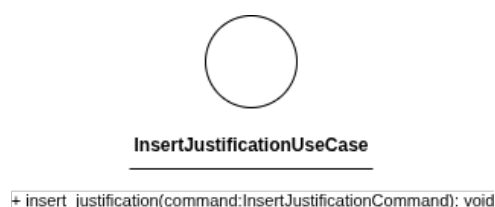


Figura 156: InsertJustificationUseCase

Descrizione

InsertJustificationUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'inserimento di una giustificazione testuale associata a un requisito di conformità. Viene implementata da *InsertJustificationService* e utilizzata da *FlaskInsertJustificationController*.

Attributi

InsertJustificationUseCase non definisce attributi.

Metodi

- + insert_justification(command: InsertJustificationCommand): void — firma del metodo delegato all'esecuzione della logica di inserimento della giustificazione a partire dai dati contenuti nel comando.

5.7.2.3) InsertJustificationCommand

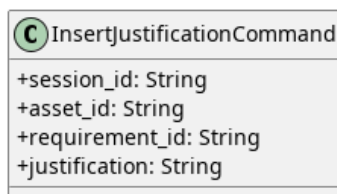


Figura 157: InsertJustificationCommand

Descrizione

InsertJustificationCommand è l'oggetto che veicola i dati necessari all'inserimento di una giustificazione dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione attiva.
- + asset_id: String — identificativo dell'Asset oggetto di valutazione.
- + requirement_id: String — identificativo del requisito a cui si associa la giustificazione.
- + node_id: String — identificativo del nodo decisionale correlato.
- + justification: String — testo della giustificazione da inserire.

Metodi

InsertJustificationCommand non definisce metodi.

5.7.2.4) InsertJustificationService



Figura 158: InsertJustificationService

Descrizione

InsertJustificationService è il service applicativo appartenente all'Application Core responsabile della logica di inserimento di una giustificazione per un requisito di conformità. Implementa l'interfaccia *InsertJustificationUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, associa la giustificazione al requisito specificato e persiste la sessione aggiornata tramite *SaveEvaluationSessionPort*.

Attributi

- `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per recuperare la sessione di valutazione.
- `save_evaluation_session_port`: `SaveEvaluationSessionPort` — outbound port usata per salvare le modifiche applicate alla sessione.

Metodi

- `+ insert_justification(command: InsertJustificationCommand): void` — concretizza il contratto definito da *InsertJustificationUseCase*. Recupera la sessione attiva, individua il requisito corrispondente ai parametri incapsulati nel comando, registra la giustificazione fornita e persiste la sessione aggiornata.

5.8) Architettura Frontend: Widget Semplici

L'approccio server-side rendering adottato nel progetto è integrato da componenti e widget reattivi realizzati con Vue 3 (Composition API). Per ogni pagina, Flask rende-
rizza il template Jinja con i dati statici, mentre le parti interattive vengono delegate a «isole» Vue indipendenti, montate su specifici punti del DOM.

Il sistema frontend gestisce solo l'aspetto grafico e la reattività, altre operazioni vengono realizzate tramite chiamate API.

Per mantenere il codice aderente ai principi SOLID — in particolare Single Responsibility e Dependency Inversion — l'architettura frontend si sviluppa su tre livelli:

3. Componenti Dumb

Componenti UI generici e riutilizzabili, privi di qualsiasi conoscenza del dominio applicativo. Ricevono dati e callback esclusivamente tramite props ed emettono eventi verso il padre. Non accedono a store, non eseguono chiamate di rete, non conoscono la struttura degli URL.

2. Widget Smart

Isole applicative che orchestrano uno o più componenti dumb per realizzare una funzionalità di dominio specifica. Contengono la logica di business (chiamate HTTP, validazione, gestione del redirect post-azione) e compongono i componenti dumb passando loro i dati necessari. Ogni widget è autocontenuto nella propria cartella.

1. Entry Point (Mount Point)

Livello di integrazione tra il mondo Flask/Jinja e Vue. Ogni entry point legge i dati iniettati dal server tramite attributi `data-*` del DOM, li converte nel formato atteso dal widget e monta l'applicazione Vue sul nodo HTML corrispondente. Non contiene logica di business né di presentazione.

Logica Condivisa (Shared) Trasversalmente ai tre livelli, la cartella `shared` contiene logica riutilizzabile che non è legata a un singolo widget né costituisce un componente visuale:

- **Composable** — Funzioni che incapsulano stato reattivo e logica riutilizzabile tra widget diversi. Seguono la convenzione Vue use*. Esempio: useFormModel, che gestisce lo stato dei campi, la validazione client-side e l'integrazione con errori server-side.
- **Definizioni di dominio** — Oggetti che descrivono la struttura dei dati condivisi tra widget. Esempio: deviceFormFields e assetFormFields, che definiscono campi e regole di validazione dei rispettivi form, condivisi tra i widget di creazione e modifica.
- **Utilità** — Funzioni pure senza dipendenze da Vue, utilizzabili sia dai widget che dagli entry point. Esempio: navigationGuard, che gestisce il blocco di navigazione per le pagine con dati non salvati.

Questa separazione garantisce che i componenti dumb siano riutilizzabili tra widget diversi, che i widget siano testabili in isolamento, e che la dipendenza dagli URL e dai dati del server sia confinata esclusivamente nel livello di integrazione.

5.8.1) Shared

5.8.1.1) Validation Rule

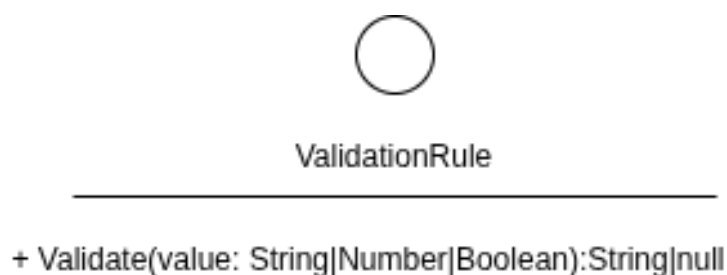


Figura 159: Shared - ValidationRule

Descrizione:

Viene utilizzato per la validazione dei dati inseriti dall'utente. Rappresenta un contratto, un oggetto deve esporre un metodo validate che accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

Attributi:

Le interfacce non hanno attributi

Metodi:

- + validate(value:String|Number|Boolean):String|null: Accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

5.8.1.2) Field Definition

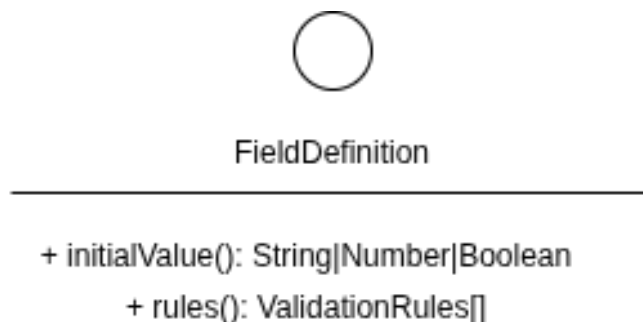


Figura 160: Shared - FieldDefinition

Descrizione: Rappresenta un contratto strutturale che definisce i campi che una oggetto deve esporre per poter essere usato nel model di un form.

Attributi: L'interfaccia non ha attributi.

Metodi:

- + initialValue():String|Number|Boolean:Il valore primitivo di default con cui il campo deve essere inizializzato nel DOM e a cui deve essere ripristinato in fase di reset.
- + rules(): ValidationRule[] :Un array di elementi che implementano l'interfaccia ValidationRule, contenente l'elenco delle funzioni di validazione associate al campo.

5.8.1.3) Use Form Model

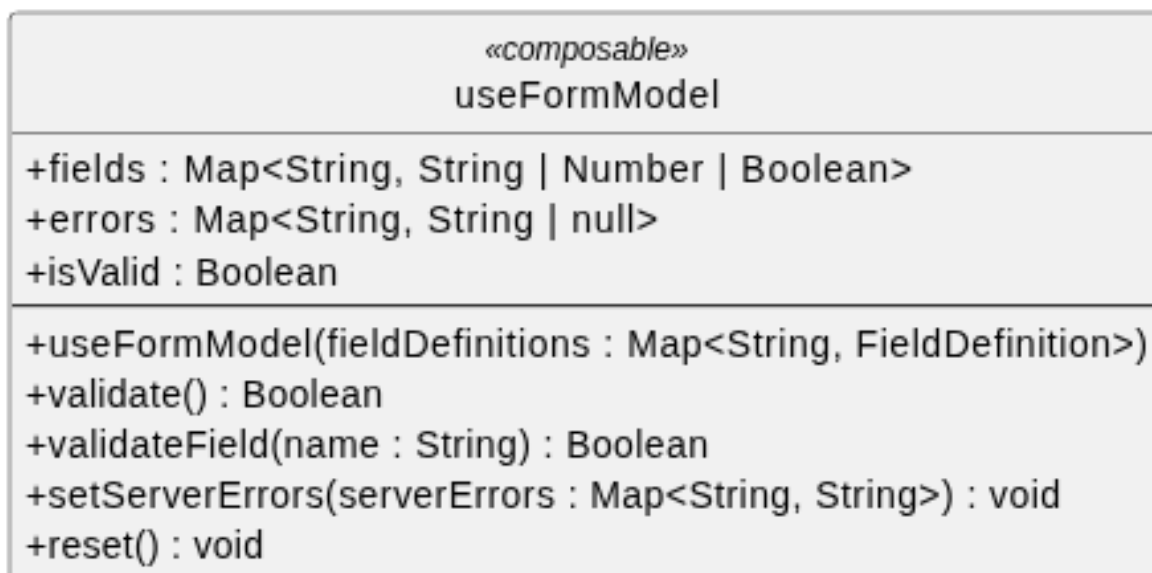


Figura 161: Shared - UseFormModel

Descrizione:

Inizializza e gestisce lo stato reattivo di un form di dominio a partire dalle sue definizioni. Si occupa della validazione client-side e dell'integrazione degli errori inviati dal server, isolando completamente la logica di business dal template visivo.

Attributi:

- - `fieldDefinitions` : `Map<String, FieldDefinition>`: Conserva internamente le field definition ricevute alla costruzione e le usa per usarlo nelle funzioni.
- + `fields`: `Map<String, String | Number | Boolean>` : Mappa reattiva che memorizza i valori correnti inseriti dall'utente per ciascun campo del form.
- + `errors`: `Map<String, String | null>`: Mappa reattiva che associa a ogni campo il relativo messaggio di errore (impostato a null se il campo è valido).
- + `isValid`: `Boolean` : Proprietà calcolata (computed). Restituisce true solo se tutti gli elementi all'interno di `errors` sono pari a null.

Metodi:

- + `useFormModel(fieldDefinitions: Map<String, FieldDefinition>)` : Costruttore/Funzione di inizializzazione. Riceve la configurazione dei campi e genera le strutture reattive per `fields` ed `errors` impostando i valori iniziali.
- + `validate()` : `Boolean` : Esegue la validazione su tutti i campi del form. Restituisce true se l'intero modulo è valido, altrimenti aggiorna la mappa degli errori e restituisce false.
- + `validateField(name: String)` : `Boolean` : Invocato per convalidare un singolo campo (es. all'evento di **blur** o di **input**). Applica le regole in ordine sequenziale e si interrompe al primo fallimento.
- + `setServerErrors(serverErrors: Map<String, String>)` : `void` : Riceve una mappa di errori generati dalle API di Flask (backend) e li inietta direttamente nello stato `errors` per mostrarli all'utente.
- + `reset()` : `void` : Svuota tutti i messaggi di errore e ripristina i valori di `fields` allo stato iniziale definito in `FieldDefinition`.

5.8.1.4) Definizioni di Dominio (Constants)

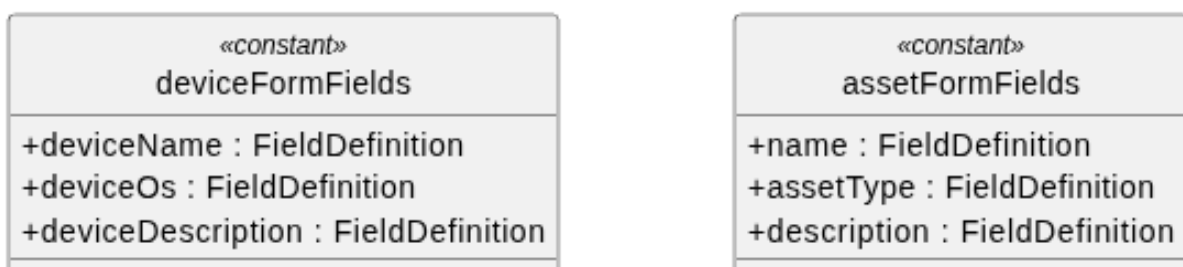


Figura 162: Shared - Form Fields Configurations

Descrizione:

Oggetti JavaScript esportati come costanti, che fungono da singolo punto di configurazione per le regole di validazione degli input utente nei form.

Rappresentano una `map<String, FieldDefinition>`, questi oggetti isolano le regole di validazione) e i valori iniziali dal resto del codice. Essendo condivisi, garantiscono una rigorosa simmetria di validazione tra le interfacce di creazione e di modifica.

Attributi:

Ogni attributo di questi oggetti rappresenta uno specifico campo del dominio e rispetta rigorosamente la forma definita da `FieldDefinition`:

Per `deviceFormFields`:

- + `deviceName`: `FieldDefinition`
- + `deviceOs`: `FieldDefinition`
- + `deviceDescription`: `FieldDefinition`

Per `assetFormFields`:

- + `name`: `FieldDefinition`
- + `assetType`: `FieldDefinition`
- + `description`: `FieldDefinition`

Metodi:

Trattandosi di oggetti di pura configurazione statica dei dati, non espongono alcun metodo operativo.

5.8.2) Component

5.8.2.1) AsyncButton

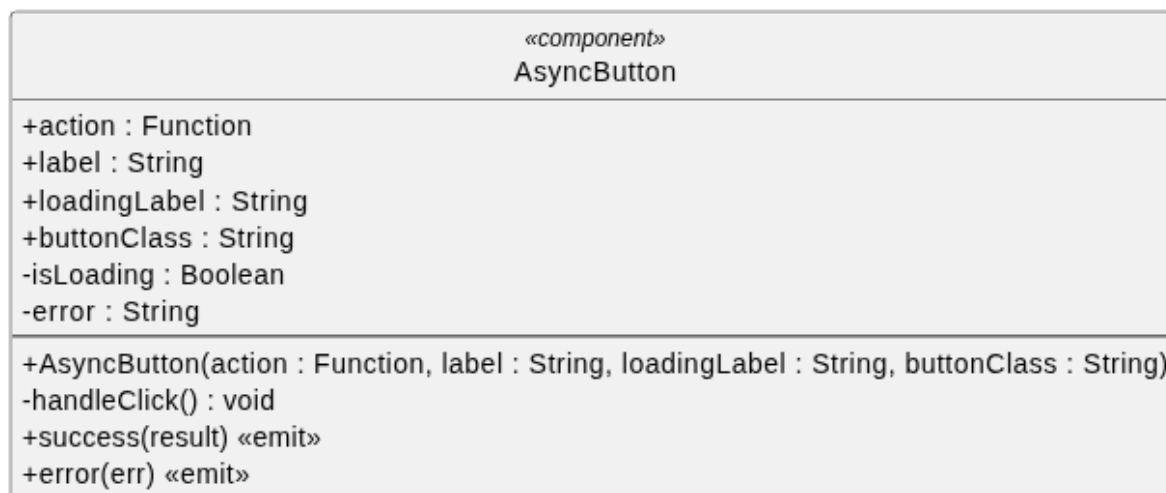


Figura 163: Componente - AsyncButton

Descrizione

Componente generico che esegue una funzione asincrona al click, gestendo lo stato di caricamento e la cattura degli errori.

Non possiede alcuna conoscenza del dominio applicativo: riceve la funzione da eseguire come parametro e comunica l'esito tramite eventi.

Attributi

- - `action: Function` : Funzione da eseguire al click del pulsante
- - `Label: String` : Testo del bottone.
- - `loadingLabel: String` : Testo del bottone durante l'esecuzione della funzione.
- - `buttonClass: String` : Stringa contenente l'elenco delle classi da applicare al pulsante
- - `isLoading : Boolean` — indica se la funzione asincrona è in esecuzione. Durante il caricamento il bottone è disabilitato per prevenire click multipli.
- `error : String` — contiene il messaggio dell'ultimo errore verificatosi, null se l'ultima esecuzione ha avuto successo.

Metodi

- + `AsyncButton(action:Function, label:string, loadingLabel:String, buttonClass:String)` — costruttore. Riceve la funzione asincrona da eseguire, il testo del bottone nei due stati (default e caricamento), e una classe CSS opzionale per lo stile.
- `handleClick()` — metodo privato invocato al click. Verifica che non sia già in corso un'esecuzione, imposta lo stato di caricamento, esegue la funzione asincrona e gestisce il risultato.
- + `success(result)` — evento emesso al completamento con successo della funzione asincrona. Trasporta il valore restituito dalla funzione.
- + `error(err)` — evento emesso in caso di fallimento. Trasporta l'eccezione catturata.

5.8.2.2) BaseModal

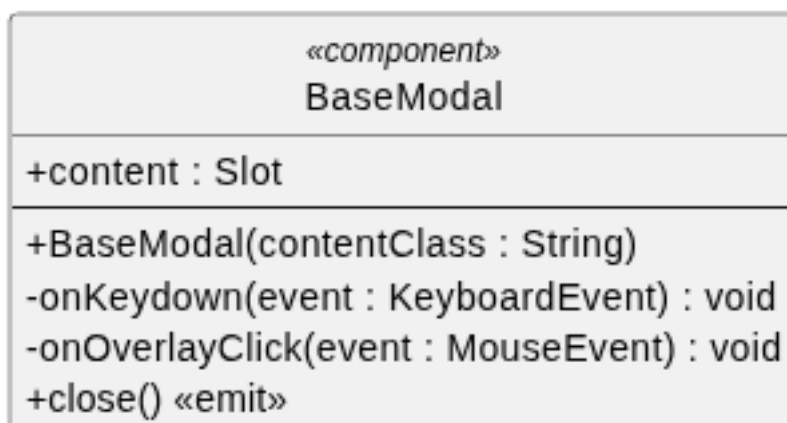


Figura 164: Componente - BaseModal

Descrizione Componente generico che fornisce l'infrastruttura per qualsiasi finestra di dialogo. Non possiede attributi interni né conosce il contenuto che ospita, interamente delegato a uno slot.

La decisione di quando il modale debba apparire o scomparire è responsabilità del componente padre, che lo monta e smonta tramite direttiva condizionale.

Attributi:

- - `ContentClass: String` : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - `Content: Slot` : slot di default che accetta il contenuto del modale. Il componente padre inietta tramite questo slot titolo, messaggio, bottoni e qualsiasi altro elemento necessario. Il `BaseModal` non ha alcuna conoscenza di ciò che lo slot contiene.

Metodi:

- + `BaseModal(contentClass:String)` : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - `onKeyDown(Event:KeyboardEvent)` : metodo privato registrato come listener globale al mount. Intercetta la pressione del tasto Escape e emette l'evento di chiusura.
- - `onOverlayClick(event)` — metodo privato invocato al click sull'overlay. Emette l'evento di chiusura solo se il click avviene sull'overlay stesso e non su un elemento figlio.
- `close()` — evento emesso quando l'utente richiede la chiusura del modale (tramite Escape o click sull'overlay). Il padre decide come reagire.

5.8.2.3) FormField

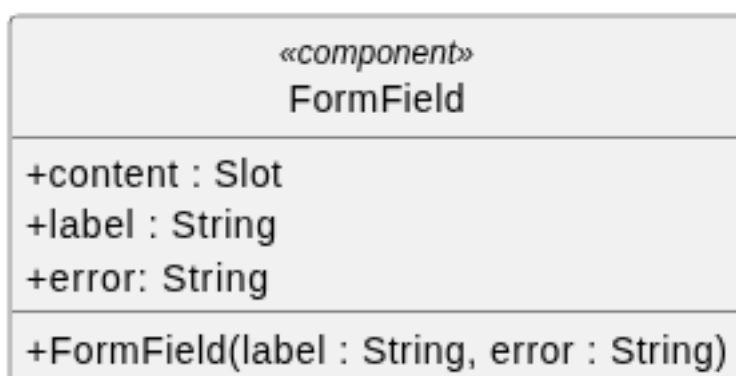


Figura 165: Componente - FormField

Descrizione:

Componente generico che funge da wrapper per un singolo campo di un form. Si occupa di mostrare la label associata al campo e l'eventuale messaggio di errore di

validazione. Non conosce il tipo di input che ospita — il campo vero e proprio (text, select, textarea, radio) è delegato allo slot.

Attributi:

- - label: String : Testo da mostrare nella label del campo di input.
- - error: String : Testo da mostrare in caso di errore.
- - Content: Slot : slot di default che accetta l'input vero e proprio. Il widget padre inietta tramite questo slot il campo appropriato con il relativo binding ai dati.

Metodi:

- + FormField(label:String, error: String):

Costruttore. Riceve il testo della label e l'eventuale messaggio di errore. Se error è null il campo è considerato valido e il messaggio non viene mostrato.

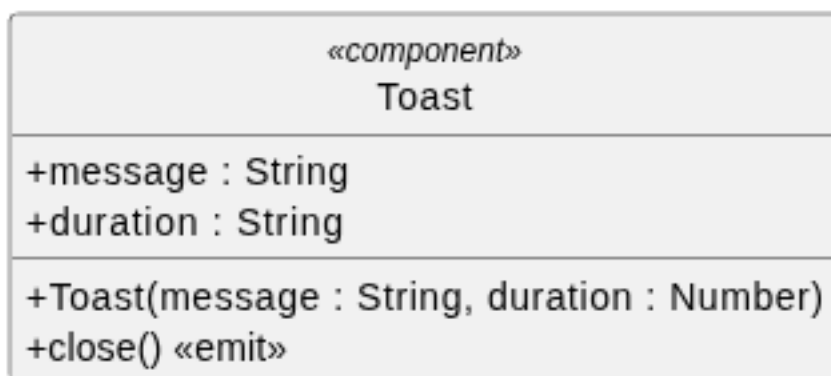
5.8.2.4) Toast

Figura 166: Componente - Toast

Descrizione:

Componente generico che mostra un messaggio temporaneo a schermo sotto forma di notifica. Scompare automaticamente dopo un tempo definito senza richiedere interazione da parte dell'utente. Non mantiene stato interno — il padre lo monta quando serve mostrare un messaggio e lo smonta quando riceve l'evento di chiusura.

Attributi:

- - message:String: messaggio da mostrare.
- - duration:Number: durata a schermo del componente.

Metodi:

- + Toast(message:String, duration:Number): costruttore. Riceve il testo del messaggio e la durata in millisecondi prima della chiusura automatica. Al mount del componente viene avviato un timer che, allo scadere, emette l'evento di chiusura.

- + close():evento emesso alla scadenza del timer. Il padre reagisce smontando il componente.

5.8.2.5) FileDropZone

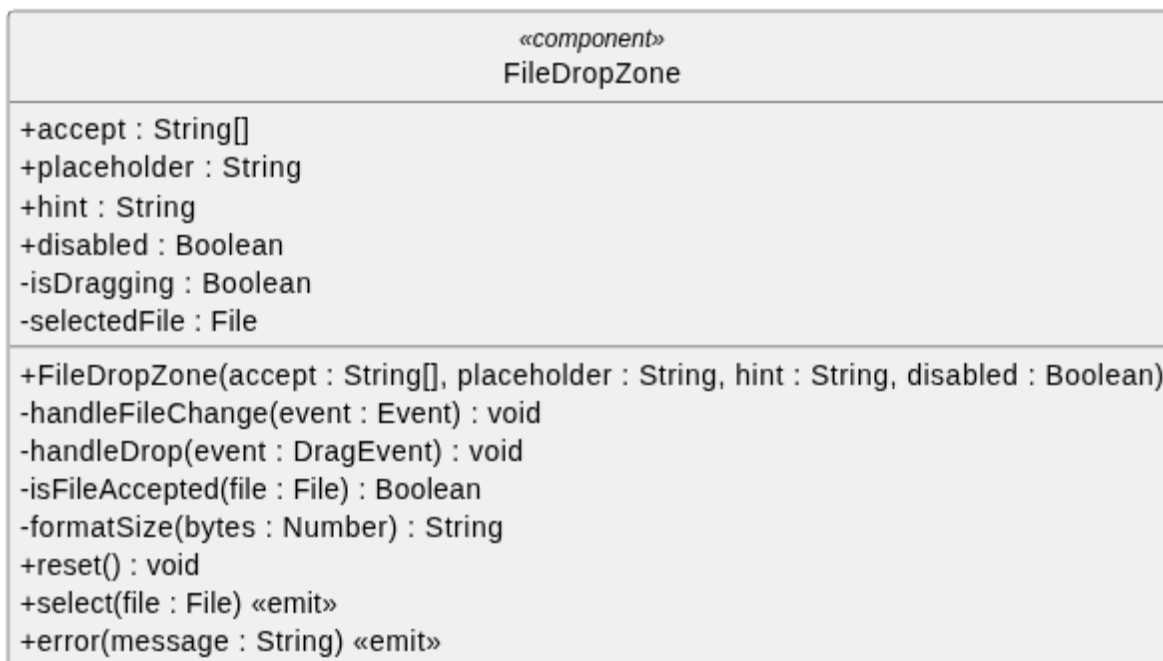


Figura 167: Componente - FileDropZone

Descrizione:

Componente generico che realizza un'area di caricamento file con supporto per drag & drop e selezione tramite click. Gestisce il feedback visivo durante il trascinamento, valida il tipo di file in base alle estensioni accettate e comunica il file selezionato al padre tramite evento. Non conosce cosa verrà fatto del file — l'upload o l'elaborazione sono responsabilità del widget che lo utilizza.

Attributi:

- - accept:String[]: Elenco di estensioni supportate.
- - placeholder: String: Testo del placeholder.
- - hint: String: Suggerimento testuale sui formati accettati.
- - disabled: Boolean: Flag per disabilitare l'interazione.
- - isDragging: Boolean: Indica se un file è attualmente trascinato sopra l'area di drop. Utilizzato per il feedback visivo.
- - selectedFile: File: Riferimento al file selezionato dall'utente, null se nessun file è stato scelto.

Metodi:

- + FileDropZone(accept: String[], placeholder: String, hint: String, disabled: Boolean):Costruttore. Riceve le estensioni accettate, il testo placeholder, un suggerimento sui formati supportati e un flag per disabilitare l'interazione.

- - `handleFileChange(event:Event)`: Metodo privato invocato alla selezione di un file tramite il file picker nativo del browser.
- - `handleDropEvent(event:DragEvent)`: Metodo privato invocato al rilascio di un file trascinato sull'area di drop. Valida il file e lo accetta o emette un errore.
- - `isFileAccepted(file:File)`: Metodo privato che verifica se l'estensione del file è tra quelle accettate.
- - `formatSize(bytes:Number)`: Metodo privato che converte una dimensione in byte in formato leggibile (Bytes, KB, MB).
- + `reset()`: Metodo pubblico esposto al padre tramite template ref. Resetta lo stato del componente rimuovendo il file selezionato.
- + `select(file:File)`: Evento emesso quando un file valido viene selezionato o trascinato.
- + `error(message:String)`: Evento emesso quando il file trascinato non supera la validazione del formato.

5.8.2.6) Device Form

«Component» DeviceForm
+fields: Map<String, String Number Boolean> +errors: Map<String, String null> -fieldComponents: FormField[]
+DeviceForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 168: Component - DeviceForm

Descrizione:

Componente di presentazione specifico per il dominio dei dispositivi. Si occupa esclusivamente di renderizzare l'interfaccia utente del modulo e di stabilire un binding bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo dispositivo o modificandone uno esistente, e non esegue alcuna logica di validazione o salvataggio.

Attributi:

- + `fields`: Object: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo.
- + `errors`: Object: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + `DeviceForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)`

5.8.2.7) Asset Form

«Component» AssetForm
+fields: Map<String, String Number Boolean> +errors: Map<String, String null> -fieldComponents: FormField[]
+AssetForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 169: Component - AssetForm

Descrizione: Componente di presentazione (Dumb Component) delegato al rendering del modulo per la gestione degli Asset. Si occupa esclusivamente di renderizzare l'interfaccia utente del modulo e di stabilire un binding bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo asset o modificandone uno esistente, e non esegue alcuna logica di validazione o salvataggio.

Attributi:

- + fields: Object: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo.
- + errors: Object: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + AssetForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)

5.8.3) Widget

5.8.3.1) AssetCreateWidget

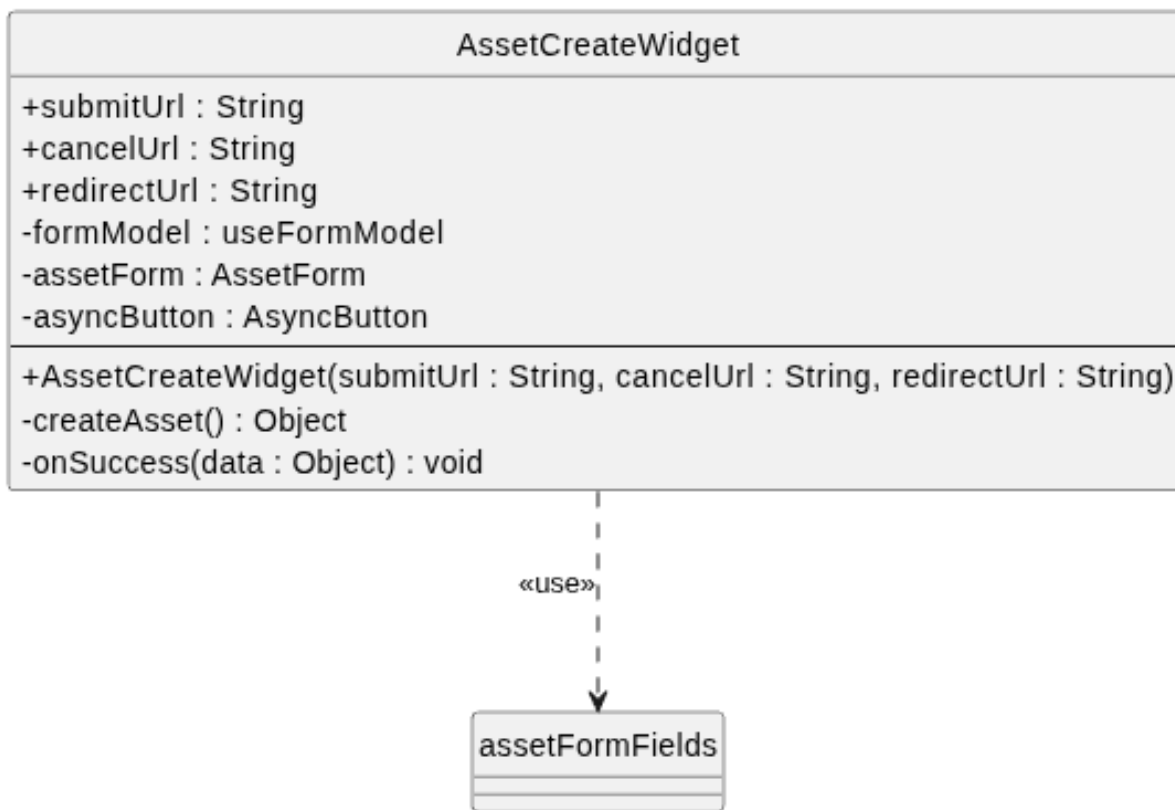


Figura 170: Widget - AssetCreateWidget

Descrizione: Rappresenta un'isola applicativa responsabile di orchestrare la creazione di un nuovo Asset. Agisce come controller di facciata: funge da ponte tra il livello di presentazione e il livello di comunicazione col backend. Detiene la logica di business specifica per questa casistica, ma delega il rendering grafico e la gestione reattiva ai componenti e ai composabile sottostanti.

Attributi:

- `+ submitUrl: String`: L'endpoint Flask a cui inviare il payload POST.
- `+ cancelUrl: String`: L'URL di fallback in caso di annullamento dell'operazione.
- `+ redirectUrl: String`: L'URL a cui reindirizzare l'utente in caso di successo.
- `- formModel: UseFormModel : Model` di riferimento per il componente.
- `- assetForm:AssetForm`: Componente che mostra graficamente il form degli asset.
- `- confirmButton:AsyncButton`: Componente che mostra graficamente il pulsante di conferma.

Metodi:

- `AssetCreateWidget(submitUrl : String, cancelUrl : String, redirectUrl : String)`: Costruttore che riceve esternamente gli url a cui corrispondono le azioni.

- - `createAsset(): Promise<Object>` : Metodo asincrono invocato dal bottone di salvataggio. Esegue la validazione invocando il composabile, compone il payload JSON e gestisce la chiamata di rete, catturando e smistando eventuali errori server-side, ritorna un oggetto json contenente la risposta.
- - `onSuccess(data: Object): void` : Callback eseguita al completamento positivo della chiamata.

5.8.3.2) AssetUpdateWidget

5.8.3.3) AssetDeleteWidget

5.9) Architettura Frontend: Modulo Decision Tree

Il frontend per la valutazione dei requisiti è progettato per essere reattivo e disaccoppiato. Per evitare di sovraccaricare il server a ogni interazione, una porzione della logica di dominio viene eseguita direttamente lato client, orchestrata da uno State Management centralizzato.

L'architettura si articola su quattro pilastri:

1. **Core di Valutazione (EvaluationEngine)**: Il client non si limita a mostrare dati passivi, ma ricalcola istantaneamente il percorso attivo dell'albero in base alle risposte dell'utente, offrendo un feedback visivo immediato tramite una struttura a oggetti polimorfica.
2. **Gestione dello Stato (DecisionTreeStore)**: Basato su Pinia, funge da «Single Source of Truth». Mantiene in memoria le risposte, lo stato dell'interfaccia, il nodo selezionato e delega i calcoli complessi all'EvaluationEngine.
3. **Interfaccia Utente (UI)**: Suddivisa in due macro-aree cooperanti. Il **Tree Canvas** disegna la topologia dell'albero delegando i calcoli geometrici al D3LayoutEngine, mentre la **Tree Sidebar** funge da pannello interattivo per leggere le domande e inserire le risposte, comunicando esclusivamente con lo Store.
4. **Integrazione Backend (EvaluationApiClient)**: Astrattizza la comunicazione HTTP con il server, assicurando che lo stato locale dell'applicazione sia costantemente sincronizzato con il database remoto in modo asincrono.

Nei paragrafi successivi verranno analizzati nel dettaglio i diagrammi delle classi dei singoli sottosistemi.

5.9.1) Architettura Interattiva della Sidebar

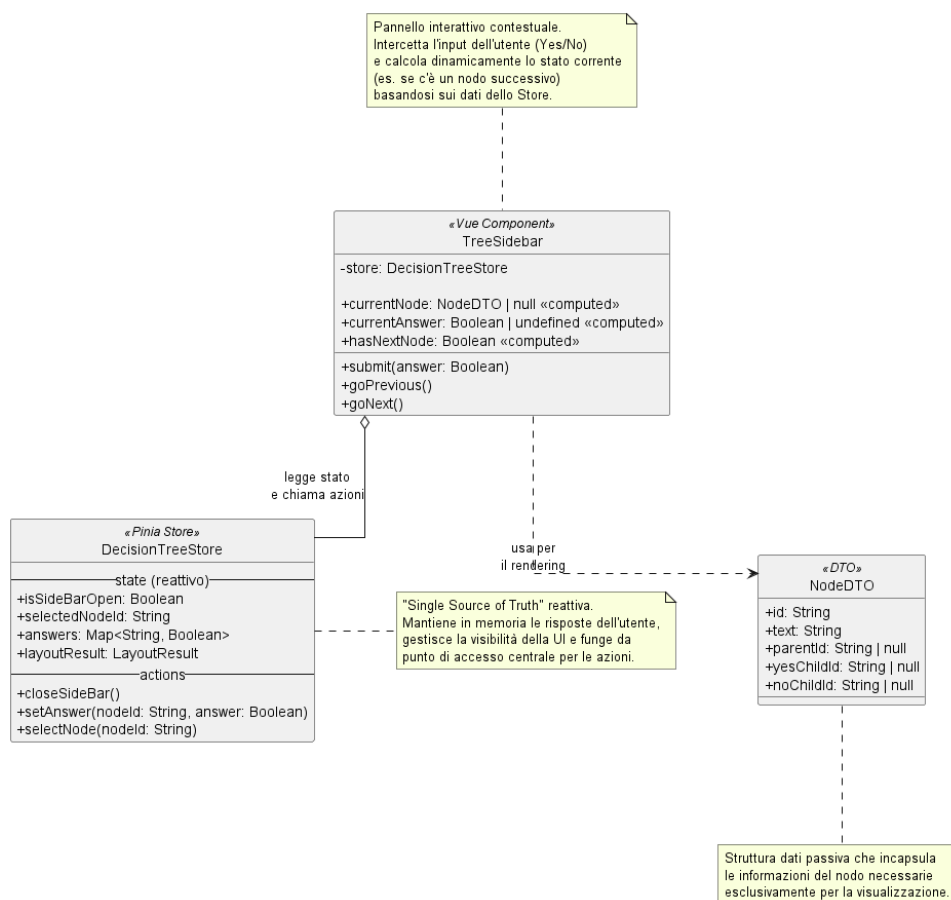


Figura 171: Architettura del componente TreeSidebar e del suo Store

Descrizione

Il diagramma illustra la relazione e la separazione delle responsabilità tra l'interfaccia utente laterale (*TreeSidebar*), il gestore dello stato globale (*DecisionTreeStore*) e l'oggetto di trasferimento dati (*NodeDTO*). Il componente visivo si interfaccia esclusivamente con lo Store per leggere lo stato reattivo e invocare le azioni di aggiornamento, utilizzando il DTO in modo puramente passivo per estrarre le informazioni testuali e topologiche necessarie al rendering a schermo.

Di seguito il dettaglio dei singoli componenti coinvolti nel flusso.

5.9.1.1) DecisionTreeStore

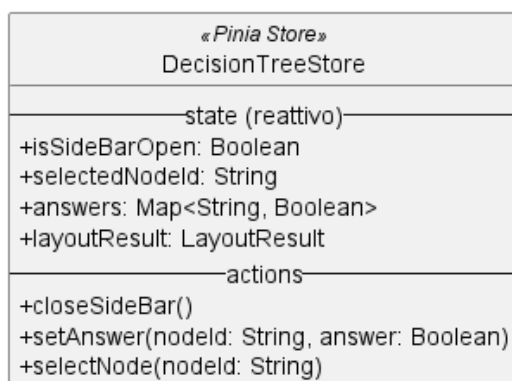


Figura 172: DecisionTreeStore

Descrizione

DecisionTreeStore è il modulo basato su Pinia che funge da «Single Source of Truth» (singola fonte di verità) per il frontend. Mantiene in memoria in modo reattivo le risposte dell'utente, gestisce la visibilità della UI e accentra il punto di accesso per tutte le modifiche di stato.

Stato Reattivo (State)

- *+ isSideBarOpen: Boolean* — flag che determina se il pannello laterale è attualmente aperto o nascosto.
- *+ selectedNodeId: String* — l'identificativo del nodo correntemente selezionato o cliccato dall'utente.
- *+ answers: Map<String, Boolean>* — dizionario che associa l'ID di ogni nodo decisionale alla rispettiva risposta (Yes/No) fornita dall'utente.
- *+ layoutResult: LayoutResult* — struttura dati che incapsula i risultati del calcolo geometrico (nodi e archi) necessari a renderizzare l'albero.

Metodi (Actions)

- *+ closeSideBar()* — azione per chiudere il pannello laterale.
- *+ setAnswer(nodeId: String, answer: Boolean)* — registra o aggiorna la risposta dell'utente per un dato nodo decisionale.
- *+ selectNode(nodeId: String)* — azione per impostare un nodo come «selezionato», scatenando gli aggiornamenti reattivi sulla UI.

5.9.1.2) TreeSidebar

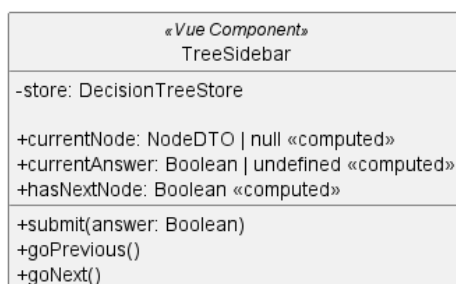


Figura 173: TreeSidebar

Descrizione

TreeSidebar è il componente Vue.js che implementa il pannello laterale interattivo. Il suo compito è intercettare l'input dell'utente e presentare le informazioni del nodo selezionato calcolando dinamicamente la navigazione basandosi sullo Store centrale.

Proprietà Compute (Computed)

- + `currentNode: NodeDTO | null` — recupera dinamicamente le informazioni del nodo selezionato in quel momento.
- + `currentAnswer: Boolean | undefined` — estrae dallo Store la risposta precedentemente data (se presente) per il nodo corrente.
- + `hasNextNode: Boolean` — calcola dinamicamente se esiste un nodo successivo navigabile partendo dalla situazione corrente.

Metodi

- + `submit(answer: Boolean)` — gestisce l'invio della risposta per la domanda a schermo, delegandone il salvataggio all'azione dello Store.
- + `goPrevious()` — innesca la navigazione per tornare al nodo precedente lungo il percorso di valutazione.
- + `goNext()` — innesca la navigazione per avanzare al nodo logico successivo all'interno del flusso.

5.9.1.3) NodeDTO

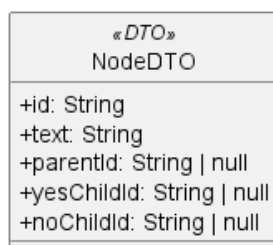


Figura 174: NodeDTO

Descrizione

NodeDTO (Data Transfer Object) è una struttura dati passiva utilizzata esclusivamente per il trasferimento e la formattazione dei dati. Incapsula le informazioni «grezze» del nodo in modo che il componente visivo possa stamparle a schermo senza dover accedere alla complessa logica di dominio.

Attributi

- + id: String — identificativo univoco del nodo.
- + text: String — il contenuto testuale della domanda o dell'esito.
- + parentId: String | null — riferimento all'ID del nodo padre, necessario per calcolare la navigazione a ritroso.
- + yesChildId: String | null — riferimento all'ID del nodo figlio in caso di risposta affermativa.
- + noChildId: String | null — riferimento all'ID del nodo figlio in caso di risposta negativa.

Metodi

NodeDTO non espone metodi, trattandosi di un oggetto dedicato al solo trasporto dati.

5.9.2) Architettura Visiva del Canvas (Tree Canvas)

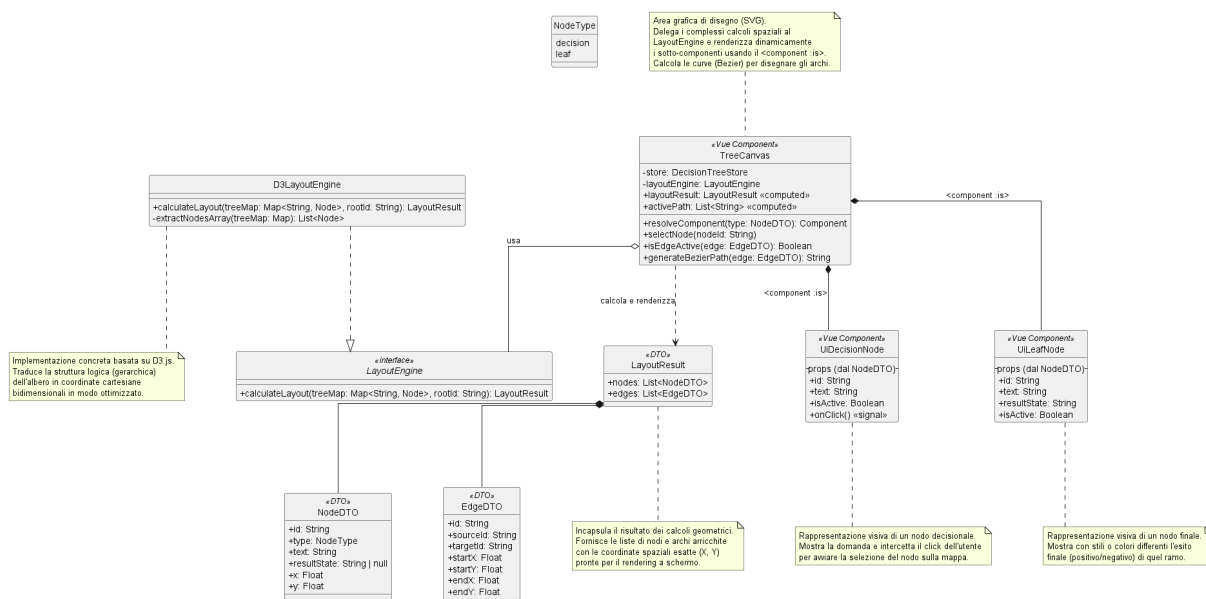


Figura 175: Architettura del componente TreeCanvas e del Layout Engine

Descrizione

Il diagramma illustra l'architettura dedicata alla renderizzazione topologica dell'albero decisionale. Per garantire prestazioni ottimali e codice pulito, il sistema separa nettamente il calcolo matematico delle coordinate visive (delegato a *D3LayoutEngine*) dal rendering effettivo a schermo (gestito dal componente *TreeCanvas* e dai suoi sotto-componenti *UiDecisionNode* e *UiLeafNode*).

Di seguito vengono analizzati nel dettaglio i componenti e le strutture dati che partecipano a questo flusso.

5.9.2.1) LayoutResult e i DTO Geometrici

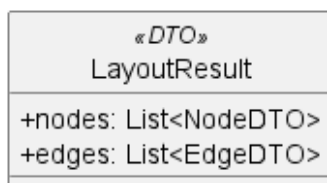


Figura 176: LayoutResult, NodeDTO ed EdgeDTO

Descrizione

Questi Data Transfer Object (DTO) rappresentano le strutture dati arricchite con le informazioni spaziali. A differenza del NodeDTO usato dalla Sidebar, qui i nodi includono coordinate assolute per il posizionamento sulla mappa.

Attributi

- **NodeDTO:**
 - + id: String — identificativo del nodo.
 - + type: NodeType — enumerativo che distingue tra nodo decisionale (decision) e nodo finale (leaf).
 - + text: String — testo da mostrare.
 - + resultState: String | null — eventuale esito finale (se di tipo leaf).
 - + x: Float, + y: Float — coordinate cartesiane calcolate per il centro del nodo.
- **EdgeDTO:**
 - + id: String — identificativo dell'arco.
 - + sourceId: String, + targetId: String — ID dei nodi collegati.
 - + startX, startY, endX, endY: Float — coordinate esatte dei punti di inizio e fine della linea di collegamento.
- **LayoutResult:**
 - + nodes: List<NodeDTO> — lista di tutti i nodi posizionati.
 - + edges: List<EdgeDTO> — lista di tutti gli archi posizionati.

5.9.2.2) LayoutEngine e D3LayoutEngine

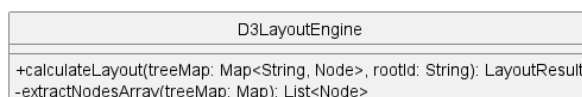


Figura 177: D3LayoutEngine

Descrizione

LayoutEngine è l'interfaccia che definisce il contratto per il calcolo spaziale. *D3LayoutEngine* ne è l'implementazione concreta, che sfrutta la libreria matematica

D3.js per trasformare la struttura logica e gerarchica dell'albero in coordinate cartesiane bidimensionali, evitando sovrapposizioni tra i nodi.

Metodi

- + `calculateLayout(treeMap: Map<String, Node>, rootId: String): LayoutResult` — riceve la mappa logica dei nodi e restituisce il `LayoutResult` contenente nodi e archi con le coordinate X e Y calcolate.
- - `extractNodesArray(treeMap: Map): List<Node>` — metodo di utilità interno per la conversione della mappa in array.

5.9.2.3) TreeCanvas

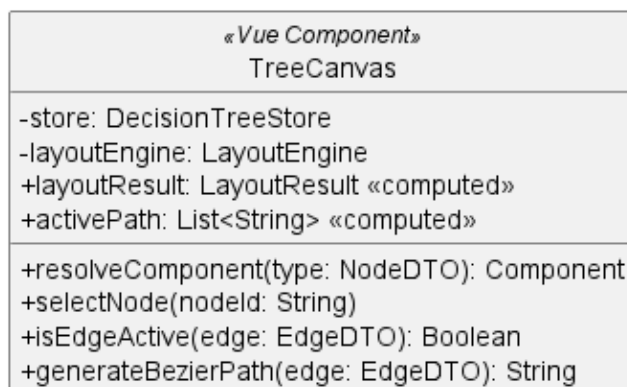


Figura 178: TreeCanvas

Descrizione

TreeCanvas è il componente Vue.js principale per la visualizzazione dell'albero. Interroga lo Store, passa i dati al Layout Engine per il calcolo geometrico e si occupa di orchestrare il rendering dinamico di linee e nodi a schermo.

Proprietà Compute e Stato

- - `store: DecisionTreeStore` — riferimento allo stato globale.
- - `layoutEngine: LayoutEngine` — riferimento al motore matematico.
- + `layoutResult: LayoutResult` — (computed) si aggiorna automaticamente se l'albero cambia, innescando un nuovo ricalcolo geometrico.
- + `activePath: List<String>` — (computed) recupera l'elenco degli ID dei nodi correntemente attivi in base alle risposte dell'utente.

Metodi

- + `resolveComponent(type: NodeDTO): Component` — determina dinamicamente quale sotto-componente Vue renderizzare (*UiDecisionNode* o *UiLeafNode*) in base al tipo del nodo.
- + `selectNode(nodeId: String)` — gestisce il click su un nodo della mappa, notificando lo Store.

- + `isEdgeActive(edge: EdgeDTO): Boolean` — verifica se la linea di collegamento appartiene al percorso attivo per applicare stili di evidenziazione.
- + `generateBezierPath(edge: EdgeDTO): String` — calcola la stringa del tracciato SVG per disegnare una curva morbida tra due nodi.

5.9.2.4) UiDecisionNode e UiLeafNode

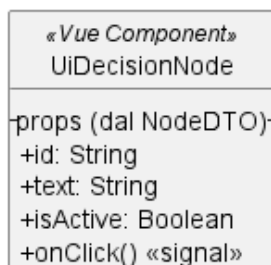


Figura 179: UiDecisionNode e UiLeafNode

Descrizione

UiDecisionNode e *UiLeafNode* sono i micro-componenti visivi montati dinamicamente dal *TreeCanvas*. Si occupano unicamente di stampare a schermo l'interfaccia di un singolo nodo, applicando stili CSS differenti a seconda che il nodo sia attivo o meno, e intercettando le interazioni dell'utente.

Proprietà (Props passate dal padre)

- + `id: String` — identificativo del nodo.
- + `text: String` — testo da mostrare (domanda o esito).
- + `isActive: Boolean` — flag iniettato dal *TreeCanvas* per indicare se il nodo è parte del percorso selezionato.
- + `resultState: String` — (Solo in *UiLeafNode*) indica l'esito finale della valutazione per colorare opportunamente il nodo (es. verde/rosso).

Eventi (Signals)

- + `onClick()` — (Solo in *UiDecisionNode*) emesso quando l'utente fa click sulla forma geometrica del nodo, catturato dal *TreeCanvas*.

5.9.3) Core di Valutazione (Domain Logic Client-side)

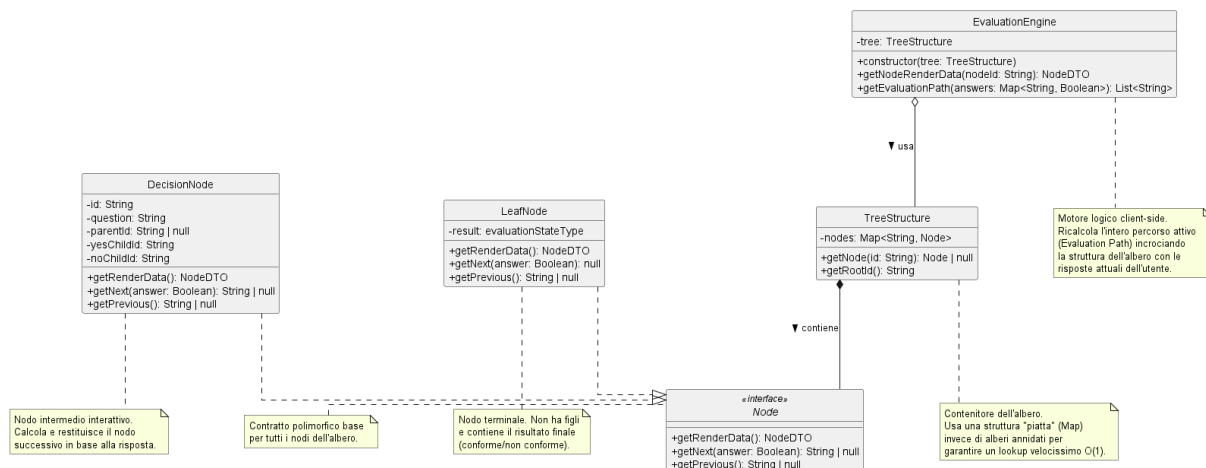


Figura 180: Diagramma delle classi del Motore di Valutazione

Descrizione

Il «Cervello» del modulo frontend risiede in una gerarchia di classi che implementano la logica di navigazione dell'albero decisionale direttamente nel browser. Questo approccio permette di ricalcolare il percorso di valutazione istantaneamente a ogni clic dell'utente, senza richiedere l'intervento del server. Il sistema utilizza il polimorfismo per distinguere il comportamento tra nodi di domanda e nodi di esito.

5.9.3.1) Interfaccia Node

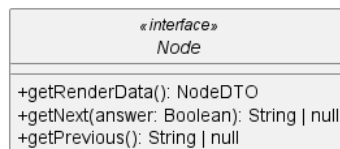


Figura 181: Interfaccia Node

Descrizione

Node è l'interfaccia (o contratto) base che definisce il comportamento comune a tutti i tipi di nodi presenti nell'albero. Garantisce che ogni nodo, indipendentemente dalla sua natura, possa fornire i propri dati di rendering e gestire la navigazione.

Metodi

- + `getRenderData(): NodeDTO` — restituisce i dati necessari al componente visivo per mostrare il nodo.
- + `getNext(answer: Boolean): String | null` — calcola l'ID del nodo successivo in base alla risposta ricevuta.
- + `getPrevious(): String | null` — restituisce l'ID del nodo genitore per permettere la navigazione a ritroso.

5.9.3.2) DecisionNode

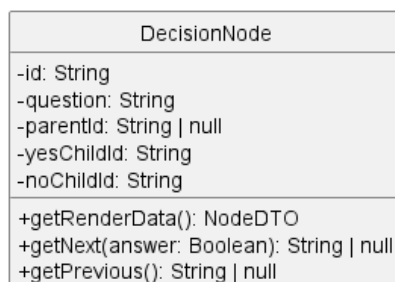


Figura 182: DecisionNode

Descrizione

DecisionNode è la classe che implementa un nodo intermedio (domanda). Contiene la logica decisionale binaria (Sì/No) per determinare quale ramo dell'albero debba essere percorso.

Attributi

- - id, - question: String — identificativo e testo della domanda.
- - parentId: String | null — riferimento al nodo precedente.
- - yesChildId, - noChildId: String — riferimenti ai due possibili nodi successivi.

Metodi

- + getNext(answer: Boolean) — implementazione concreta che restituisce yesChildId se la risposta è true, altrimenti noChildId.

5.9.3.3) LeafNode

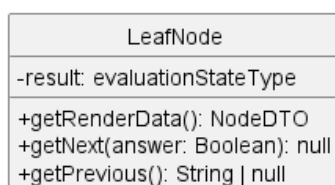


Figura 183: LeafNode

Descrizione

LeafNode rappresenta il punto terminale di un ramo dell'albero (foglia). Non permette ulteriori avanzamenti e contiene lo stato finale della valutazione per quel determinato percorso.

Attributi

- - result: evaluationStateType — definisce l'esito finale (es. «Conforme», «Non Conforme»).

Metodi

- + getNext(answer: Boolean) — restituisce sempre null, segnando la fine del percorso.

5.9.3.4) TreeStructure

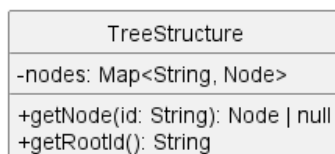


Figura 184: TreeStructure

Descrizione

TreeStructure funge da contenitore organizzato per l'intero albero decisionale. Al fine di massimizzare le prestazioni di ricerca lato client, utilizza una rappresentazione «piatta» basata su una mappa anziché una struttura nidificata.

Attributi

- - nodes: Map<String, Node> — mappa che indicizza tutti i nodi tramite il loro ID univoco per un accesso immediato ($O(1)$).

Metodi

- + getNode(id: String): Node | null — recupera l'oggetto nodo corrispondente all'ID fornito.
- + getRootId(): String — restituisce l'ID del punto di partenza della valutazione.

5.9.3.5) EvaluationEngine

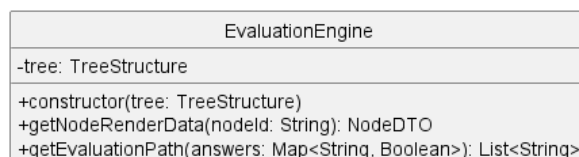


Figura 185: EvaluationEngine

Descrizione

EvaluationEngine è il motore logico principale che orchestra la valutazione. È il componente che traduce la massa di risposte dell'utente in un percorso visivo coerente e navigabile.

Attributi

- - tree: TreeStructure — la struttura dell'albero su cui operare.

Metodi

- + `getEvaluationPath(answers: Map<String, Boolean>): List<String>` — incrocia la mappa delle risposte fornite dall'utente con la struttura dell'albero per generare la lista ordinata di ID che compongono l'attuale «Percorso Attivo» (Active Path).
- + `getNodeRenderData(nodeId: String): NodeDTO` — metodo di utilità per ottenere i dati grafici di un nodo specifico tramite il motore.

5.9.4) DecisionTreeStore

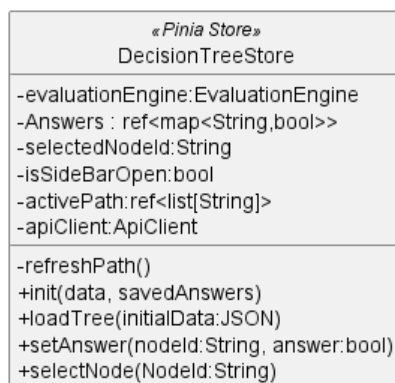


Figura 186: Dettaglio del DecisionTreeStore (Pinia)

Descrizione

DecisionTreeStore rappresenta il cuore reattivo dell'applicazione. È lo Store globale che permette la comunicazione disaccoppiata tra i vari componenti Vue (Canvas e Sidebar). Invece di passare dati tramite «props» e «events» complessi, i componenti leggono e scrivono direttamente in questo store, che si occupa di mantenere la coerenza dei dati.

Attributi (State)

- - `evaluationEngine: EvaluationEngine` — istanza del motore logico utilizzata per i calcoli di navigazione.
- - `answers: ref<Map<String, Boolean>>` — stato reattivo delle risposte fornite dall'utente durante la sessione corrente.
- - `selectedNodeId: String` — identifica il nodo su cui l'utente sta interagendo o che ha cliccato nel Canvas.
- - `activePath: ref<List<String>>` — lista reattiva degli ID dei nodi che compongono il percorso corrente, aggiornata automaticamente al variare delle risposte.
- - `apiClient: ApiClient` — riferimento al client per le chiamate verso le API del backend.

Metodi (Actions)

- + `init(data, savedAnswers)` — inizializza lo store caricando la struttura dell'albero e le eventuali risposte già salvate nel database.
- + `setAnswer(nodeId, answer)` — aggiorna una risposta nello stato locale, invoca il ricalcolo del percorso e avvia la persistenza asincrona tramite l'API.

- + `selectNode(nodeId)` — aggiorna il nodo selezionato, permettendo alla Sidebar di mostrare le informazioni contestuali corrette.
- - `refreshPath()` — metodo privato invocato internamente per aggiornare l'attributo `activePath` interpellando l'*EvaluationEngine*.

5.9.5) EvaluationApiClient

EvaluationApiClient
-baseUrl: String
+saveAnswer(answer: bool, device_id: String, asset_id: String, requirement_id: String, node_id: String): Promise<JSON> +fetchRequirementEvaluationState(device_id: String, asset_id: String, requirement_id: String): Promise<JSON>

Figura 187: Dettaglio dell'EvaluationApiClient

Descrizione

EvaluationApiClient è l'Outbound Adapter del frontend. Questa classe incapsula tutta la logica di comunicazione HTTP, isolando il resto dell'applicazione (Store e Componenti) dai dettagli implementativi delle chiamate API. Utilizza il pattern delle *Promise* per gestire l'asincronia tipica delle richieste di rete.

Attributi

- - `baseUrl: String` — l'indirizzo radice del server API.

Metodi

- + `saveAnswer(answer, device_id, asset_id, requirement_id, node_id): Promise<JSON>` — invia una richiesta asincrona al server per salvare la risposta fornita a un determinato nodo dell'albero. Riceve tutti i parametri di contesto necessari (dispositivo, asset e requisito) per garantire la corretta associazione dei dati nel database.
- + `fetchRequirementEvaluationState(device_id, asset_id, requirement_id): Promise<JSON>` — recupera dal backend lo stato attuale della valutazione per un determinato requisito. Questo metodo è fondamentale durante la fase di inizializzazione per ripristinare il «Percorso Attivo» basandosi sui dati salvati in precedenza.

6) Tracciamento

7) Gestione degli Errori

L'applicazione adotta una strategia di gestione degli errori stratificata. Le eccezioni non attraversano liberamente i confini architetturali, vengono invece intercettate, contestualizzate e tradotte man mano che si propagano dai livelli più profondi verso l'esterno. Per garantire chiarezza d'intenti e manutenibilità del codice, l'implementazione si basa su una netta distinzione semantica definita dai suffissi delle classi:

- **Suffisso **Error* (Livello di Dominio e Infrastruttura):** Viene utilizzato all'interno del Domain Layer (ereditando da una classe base `DomainError`) per segnalare la violazione di regole di business, oppure negli Outbound Adapter per segnalare problemi di interazione con risorse esterne o fallimenti infrastrutturali.
- **Suffisso **Failure* (Livello Applicativo / Casi d'Uso):** Identifica il fallimento di un intero flusso di lavoro orchestrato da un **Application Service**. Queste eccezioni descrivono l'impossibilità di portare a termine un'operazione dal punto di vista dell'utente o del sistema chiamante.

7.0.1) Il Flusso di Traduzione degli Errori (Exception Mapping)

L'intento di questa gestione degli errori è di evitare che dettagli tecnici (come l'uso specifico di MongoDB) inquinino i livelli superiori. Il meccanismo di propagazione si articola in tre fasi:

1. **Adattatori di Output (Infrastruttura):** Sollevano eccezioni custom di tipo `Error` per segnalare l'impossibilità di completare un'operazione di propria competenza. Questo avviene sia intercettando e traducendo errori generati da librerie di terze parti (es. eccezioni dei driver del database), sia generandole direttamente in base alla logica interna dell'adapter (es. risorse non trovate in un sistema di cache in memoria).
2. **Application Services (Casi d'Uso):** Durante l'orchestrazione, intercettano le eccezioni `Error` provenienti dal dominio o dai repository e le sollevano nuovamente (wrapping) traducendole in eccezioni di tipo `Failure`, aggiungendo il contesto specifico dell'operazione in corso.
3. **Adattatori di Input (Controller):** Comunicano esclusivamente con i Service, ricevendo solo eccezioni `Failure`. Il loro unico compito è mappare questi fallimenti nei corretti codici di stato del protocollo di comunicazione (es. HTTP 400 Bad Request, 404 Not Found, 409 Conflict), restituendo al client messaggi chiari e sicuri.